Trust Signature Layer

Proof of Continuity for the AI Internet

A Transport-Independent Trust Envelope Protocol Beneath Applications

---

Founder / Protocol + Hyper-Specific Robust Implementation Specification Draft

May 2026

Hyper-Specific Robust Implementation Variant v3

The internet authenticates accounts. The AI internet must authenticate continuity.

Executive Abstract

The Trust Signature Layer (TSL) is a transport-independent trust envelope protocol for online communication, transactions, and autonomous agents. It is not an application that users join and it is not a social network, inbox, marketplace, wallet, or advertising surface. It allows any message, transaction, API call, or agent action to carry a cryptographic proof of origin, continuity, and revocation status without exposing private content by default. Instead of forcing every platform to build its own identity, reputation, and anti-fraud stack, TSL moves trust beneath application semantics and above ordinary transport delivery. Applications become carriers of signed envelopes and proof links; the canonical trust state lives in cryptographic identities, co-signed receipts, append-only transparency logs, and blockchain-anchored checkpoints.

The protocol's core primitive is Proof of Continuity: a hard-to-fake record of behavioral and cryptographic continuity over time. In a world where generative AI can cheaply produce fake profiles, fake credentials, fake messages, fake social graphs, and fake agents, the scarce resource becomes not identity, but durable, organic, verifiable trajectory. TSL is designed to become the trust substrate for email, messaging, marketplaces, wallets, professional networks, customer support, open-source ecosystems, and AI-agent communication.

Contents

1 Implementation Status: Base Sepolia Release Candidate

This release-candidate document records the current executable state of the reference implementation. It is not a mainnet production certification. It is a public-testnet implementation snapshot for the protocol described in this specification.

Current Release Candidate Status

The current implementation proves the core trust pipeline end to end: canonical objects are signed, committed to Merkle logs, checkpointed, settled on Base Sepolia, and verified through the pure verifier with settlement required. Scoring, ZK, audit consistency, governance policy, and agent delegation remain optional verifier gates rather than requirements for cryptographic validity.

1.1 Implemented Reference Stack

Area	Current Implementation
Core verifier	TypeScript canonical verifier with schema validation, canonical JSON, domain-separated hashes, Ed25519 signatures, Merkle inclusion, checkpoint validation, revocation checks, settlement adapters, and optional policy gates.
Objects	Event commitments, receipts, attestations, revocations, trust assessments, proof bundles, audit findings, governance policies, non-membership proofs, and agent delegations.
Services	Relay, log, resolver, verifier API, checkpoint submitter, scoring provider, auditor, CLI, web verifier, browser-extension skeleton, and agent sidecar.
Storage and queues	PostgreSQL migrations and Redis Streams for the local reference stack, with Merkle nodes and public commitment state persisted separately from private message content.
Settlement	Local Hardhat and Base Sepolia EVM settlement through token-free checkpoint, identity, revocation, provider, and governance registries.
ZK selective disclosure	Groth16 circuits for `identity_age_days >= threshold` and `reciprocal_receipt_count >= threshold`; additional ZK claim types are represented as optional proof-bundle fields.
Parity	Rust and Python parity slices validate canonicalization, hashing, signatures, Merkle roots, and deterministic vectors against the TypeScript reference.
Release gates	Local release command covers build, tests, contract tests, ZK tests, parity, browser bundle, Docker Compose config, deployment validation, and Base mainnet dry-run.

1.2 External Production Gates

Not Yet Mainnet Production Complete

The following gates remain outside the current release-candidate implementation:

- externally governed ZK ceremony artifacts for production use,
- vendor-backed KMS/HSM signing adapters,
- independent security audit and audit-remediation cycle,
- manual one-million-event full-path load acceptance run,
- staffed abuse-review and appeal operations,
- Base mainnet deployment; current mainnet path is dry-run only,
- review or replacement of the production-runtime `ethers -> ws` moderate advisory.

2 The One-Line Pitch

Category Thesis

TSL is the universal trust layer for the AI internet: a protocol that lets any identity, human, business, wallet, or agent prove cryptographic continuity across apps without revealing private messages.

Protocol, Not Application

TSL is not an application that users sign up for, browse, advertise on, or use as a destination. It is a protocol layer and trust-envelope standard that can be embedded beneath existing applications and above transport delivery. Web interfaces, browser extensions, wallets, dashboards, SDKs, and command-line tools are only reference clients for creating, carrying, resolving, and verifying TSL proofs.

The product does not ask the world to join another social network. It does not ask every company to rip out its communication stack. It does not require a new inbox, a new marketplace, or a new identity provider.

Instead, TSL says:

> **Keep every app. Add a trust settlement layer beneath them.**

Every application can carry a TSL envelope. Every envelope can be verified. Every verified event can become part of a privacy-preserving continuity graph. Every continuity graph can be scored, audited, challenged, and selectively disclosed.

The Core Belief

AI makes content cheap. Blockchain makes records durable. Graphs make behavior legible. The next internet trust primitive is **Proof of Continuity**.

3 Why Now

The internet is entering a phase where the cost of manufacturing convincing identity signals is collapsing.

A malicious actor can now generate:

- realistic profile pictures,
- polished bios,
- credible writing styles,
- fake resumes and credentials,
- synthetic references,
- cloned voices,
- fake customer support agents,
- fake sellers and buyers,
- fake software maintainers,
- fake AI agents,
- fake communities of accounts that vouch for each other.

The old trust stack was built for a slower internet:

Old Primitive	Failure Mode in the AI Internet
Username	Easy to impersonate or imitate
Profile picture	AI-generated or stolen
Verified badge	Platform-specific, often pay-to-play or policy-dependent
Email domain	Spoofable, compromised, or unfamiliar to users
Password login	Authenticates access, not trustworthiness
KYC	Expensive, invasive, not useful for pseudonymous ecosystems
Platform reputation	Trapped in silos; disappears when users change platforms
One-time credential	Says little about current behavior or compromise

The coming internet requires a different primitive:

> **Can this identity prove a durable, hard-to-fake pattern of signed behavior over time?**

TSL is the answer.

4 The Category: Trust Settlement

Payments have settlement networks. Cloud has infrastructure layers. Identity has authentication providers. Communication has no equivalent trust settlement layer.

Today, every application tries to solve trust locally:

- Gmail fights phishing inside Gmail.

- LinkedIn fights fake profiles inside LinkedIn.
- Marketplaces score sellers inside their marketplace.
- Wallets warn about scams inside wallet UX.
- AI platforms try to authenticate agents inside their own ecosystem.

This creates duplicated effort, weak portability, and fragmented trust.

TSL introduces a new category:

Trust Settlement

Trust settlement is the process by which claims about identity continuity, key validity, revocation, interaction history, attestations, and risk assessments become portable, verifiable, and independent of any single application.

The protocol does for communication trust what payment rails did for commerce:

- applications initiate interactions,
- the trust layer settles continuity,
- verifiers check proofs,
- users retain portability,
- no single app owns the canonical trust history.

5 Design Goal: Below the Application Layer

The most important architectural decision is that TSL should not become an integrations company. It should become a protocol company.

Wrong Architecture

Build one custom integration for Gmail, another for Discord, another for LinkedIn, another for X, another for Slack, another for marketplaces, and another for agent frameworks. Each app becomes a special case.

Right Architecture

Make applications dumb carriers. The protocol should only require that a message, transaction, or agent action can carry a signed envelope or proof link. Identity, signatures, receipts, revocations, attestations, checkpoints, and verification live below the app.

The application layer should answer:

> How do I deliver this message or action?

The TSL layer should answer:

> Who signed this? Is the key valid? Was the event committed? Has the identity been revoked? Does the interaction fit a trustworthy continuity pattern? What proof can be shown without exposing private content?

6 Layer Placement: Below Applications, Above Transport

TSL should be understood as a trust-envelope layer, not as a replacement for TCP, TLS, HTTP, SMTP, QUIC, or any specific application protocol. It does not deliver packets. It does not own the user interface. It does not require users to visit a new destination. It attaches verifiable trust semantics to messages and actions that are already moving through existing transports.

Layer	Role
Application semantics	Email, chat, marketplace messages, wallet actions, API calls, support tickets, code releases, and AI-agent tool calls.
TSL trust-envelope layer	Signed event commitments, TrustID resolution, receipts, revocation checks, inclusion proofs, checkpoint verification, and optional signed trust assessments.
Transport and security layer	HTTPS, TLS, QUIC, TCP/IP, SMTP, WebSockets, message queues, and other delivery mechanisms.

No Signup Destination

The adoption goal is not to bring users into a new app. The adoption goal is to make existing applications and agents carry verifiable trust envelopes. In the mature version, TSL should feel like TLS for trust: mostly invisible when present, obvious when missing, and independently verifiable when challenged.

A typical flow is:

```
user or agent creates ordinary message/action
  -> TSL client or SDK creates commitment
  -> TSL active key signs commitment
  -> envelope or proof link is attached to the message/action
  -> message travels over existing transport
  -> recipient verifier checks signature, key state, revocation, log inclusion, and
    checkpoint
  -> recipient sees continuity status and explanation
```

7 Protocol Stack

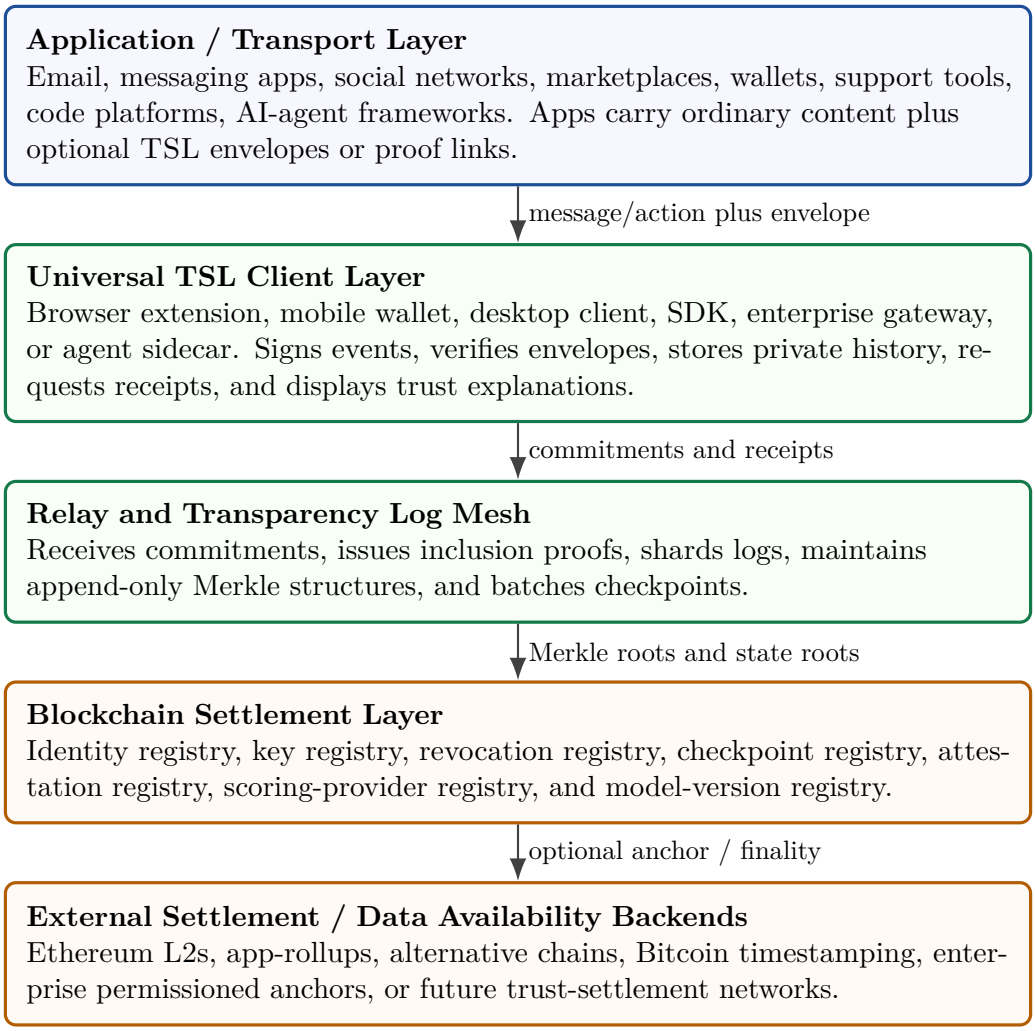

Figure 1: TSL as a trust-envelope layer beneath application semantics and above ordinary transport delivery.

The key separation is simple:

Layer	Responsibility
Application layer	Delivers messages, transactions, and agent calls
Client layer	Signs, verifies, stores private context, and renders trust UX
Relay/log layer	Batches commitments and provides inclusion proofs
Blockchain layer	Settles identity, revocation, checkpoints, attestations, and provider state
AI/graph layer	Estimates behavioral risk from local, disclosed, and aggregate evidence
Governance layer	Defines protocol upgrades, provider rules, appeals, and abuse controls

8 Core Primitive: Proof of Continuity

Proof of Continuity is not a single score. It is a bundle of verifiable signals:

- continuity of cryptographic keys,
- continuity of signed event history,
- continuity of counterparty receipts,
- continuity of graph position,
- continuity of attestations,
- continuity of behavioral trajectory,
- continuity of revocation and recovery state.

An identity can have a valid signature but still be risky. A compromised account can sign with a valid key. A bot can sign messages. A scammer can create a keypair. Therefore, TSL separates authentication from trustworthiness.

Authentication Is Not Trust

A valid signature proves that a key signed an event. It does not prove that the actor is safe, honest, human, uncompromised, or acting in good faith.

The protocol's thesis is:

$$\text{Trust} \approx \text{valid origin} + \text{durable continuity} + \text{organic graph position} + \text{low adversarial similarity}. \quad (1)$$

9 Canonical Objects

9.1 TrustID

A TrustID is a portable cryptographic identity. It may represent a person, business, wallet, software package, AI agent, marketplace seller, organization, or pseudonymous account.

```
{
  "type": "tsl.identity.v1",
  "id": "did:tsl:eip155-8453:0xabc...",
  "controller": "smart_account_or_public_key",
  "created_at": "2026-05-25T00:00:00Z",
  "verification_methods": [
    {
      "id": "#device-key-1",
      "type": "ed25519",
      "public_key": "z6Mk...",
      "status": "active"
    }
  ],
  "recovery": {
    "type": "smart_account_recovery",
    "policy_commitment": "0x..."
  },
  "privacy_policy_commitment": "0x..."
```

```
}
```

The TrustID should be compatible with decentralized identifier patterns, but TSL should not depend on any single DID method. The protocol should expose a resolution abstraction:

```
resolveTrustID(id) -> {
  controller,
  active_keys,
  revoked_keys,
  service_endpoints,
  policy_commitments,
  latest_checkpoint
}
```

9.2 Event Commitment

An event commitment is a signed claim that an interaction occurred. The raw message remains private by default.

```
{
  "type": "tsl.event_commitment.v1",
  "event_class": "message | transaction | attestation | agent_call | code_release",
  "sender": "did:tsl:eip155-8453:0xabc...",
  "receiver_commitment": "H(receiver_id || receiver_salt)",
  "content_commitment": "H(domain || content_hash || content_salt)",
  "metadata_commitment": "H(private_metadata || metadata_salt)",
  "previous_event_commitment": "optional_hash",
  "timestamp": "2026-05-25T00:01:00Z",
  "nonce": "random_256_bit_nonce",
  "disclosure_policy": "commitment_only",
  "signature": "sender_signature_over_canonical_payload"
}
```

The protocol should not make `platform` a required public field. Platform-specific metadata should be private, optional, or selectively disclosed. This avoids turning the protocol into an app-by-app integration system.

9.3 Receipt Commitment

Receipts turn unilateral claims into mutually observed interactions.

```
{
  "type": "tsl.receipt_commitment.v1",
  "event_commitment": "H(tsl.event_commitment.v1)",
  "receiver": "did:tsl:eip155-8453:0xdef...",
  "receipt_class": "received | replied | transacted | completed | disputed",
  "timestamp": "2026-05-25T00:02:00Z",
  "metadata_commitment": "H(private_receipt_metadata || salt)",
  "signature": "receiver_signature_over_receipt"
}
```

A single-signed event proves origin. A co-signed receipt proves interaction. A long history of diverse reciprocal receipts is much harder to fake than a static profile.

9.4 Attestation

An attestation is a signed claim by one identity about another.

```
{
  "type": "tsl.attestation.v1",
  "issuer": "did:tsl:org:trusted-provider",
  "subject": "did:tsl:eip155-8453:0xabc...",
  "attestation_class": "known_business | verified_maintainer | trusted_counterparty |
    scam_warning | org_member",
  "claim_commitment": "H(claim_body || salt)",
  "visibility": "public | selective | private",
  "issued_at": "2026-05-25T00:03:00Z",
  "expires_at": "2026-11-25T00:03:00Z",
  "signature": "issuer_signature"
}
```

Negative attestations require strict abuse controls: rate limits, evidence commitments, appeal channels, issuer accountability, and separation between warnings and definitive claims.

9.5 Revocation Event

```
{
  "type": "tsl.revocation.v1",
  "trust_id": "did:tsl:eip155-8453:0xabc...",
  "revoked_key": "z6MkOld...",
  "replacement_key": "z6MkNew...",
  "reason_class": "rotation | compromise | device_loss | policy_update",
  "effective_at": "2026-05-25T00:04:00Z",
  "signature": "controller_or_recovery_signature"
}
```

Revocation is not an implementation detail. It is central to trust. Without strong revocation, a long-lived identity becomes a long-lived liability.

9.6 Batch Checkpoint

```
{
  "type": "tsl.batch_checkpoint.v1",
  "epoch": "2026-05-25T00:00:00Z/PT5M",
  "shard": "identity_prefix_00af",
  "event_root": "0x...",
  "receipt_root": "0x...",
  "attestation_root": "0x...",
  "revocation_root": "0x...",
  "event_count": 148292,
  "receipt_count": 93301,
  "previous_checkpoint": "0x...",
  "settlement_backend": "eip155:8453",
  "settlement_tx": "0x...",
  "relay_signature": "0x..."
}
```

10 What Goes On-Chain

The blockchain is the settlement and control plane. It is not the database of private communications.

On-chain	Off-chain / Local
TrustID registry	Raw messages
Current public keys	Attachments and media
Key rotation records	Exact private contact lists
Revocation records	Sensitive metadata
Checkpoint roots	Full event payloads
Attestation roots	Full communication graph
Scoring-provider registry	Feature vectors if not disclosed
Model-version registry	Local risk computation inputs
Appeal/governance hooks	Private user settings

10.1 Normative Data-Placement Scope

Because provider, model, and governance registries may exist on-chain, implementations MUST keep a strict data-placement boundary. The blockchain stores control-plane facts and commitments, not private evidence.

Placement	Allowed Data
On-chain	Registry identifiers, active-key commitments, revocation state, checkpoint roots, attestation roots, provider IDs, model-card commitments, evaluation-report commitments, governance-policy commitments, and settlement timestamps.
Off-chain public	Proof bundles, transparency-log entries, inclusion proofs, schemas, public model cards, public evaluation reports, public audit findings, and public governance policies.
Local/private	Raw messages, salts, exact counterparties, private metadata, local relationship labels, private graph, private feature vectors, private notes, and undisclosed consent records.
Never on-chain	Raw content, exact private graph, private salts, protected attributes, IP addresses, user-agent fingerprints, precise location, biometric data, and private disclosure-consent contents.

If an implementation needs to reference private evidence from a public or on-chain object, it MUST use a salted commitment, Merkle root, accumulator commitment, encrypted pointer, or zero-knowledge proof. It MUST NOT publish the private evidence itself.

The chain should answer:

- Who controls this TrustID?
- Which keys are currently valid?
- Was this key revoked?

- Was this batch checkpoint committed before time T?
- Which attestation root was active?
- Which scoring provider signed this risk assessment?
- Which model version produced this assessment?

The chain should not answer:

- What did the message say?
- Who exactly talked to whom?
- What is a user's full contact graph?
- What are the user's private behavioral features?

11 Ledger Architecture

11.1 MVP: Transparency Log with Periodic Anchors

The first production-grade version should use an append-only transparency log with periodic blockchain anchoring.

```
client signs event
  -> relay receives event commitment
  -> relay appends commitment to sharded Merkle log
  -> relay returns inclusion promise
  -> checkpoint root is periodically anchored on-chain
  -> verifier checks Merkle inclusion + chain checkpoint + key state
```

This architecture gives low cost, high throughput, and public auditability.

11.2 Production: Sharded Log Mesh

At scale, one global log is not enough. Use sharded logs by epoch and identity prefix.

$$\text{shard}(i, t) = \text{Hash}(\textsc{TrustID}_i)_{[0:k]} \parallel \text{epoch}(t). \tag{2}$$

Each shard maintains:

- event commitment tree,
- receipt commitment tree,
- attestation commitment tree,
- revocation tree,
- checkpoint chain,
- consistency proofs.

The verifier only needs the relevant shard proof, not the entire network history.

11.3 Scale Path: App-Rollup

When volume grows, TSL should become its own application-specific rollup or trust-settlement network.

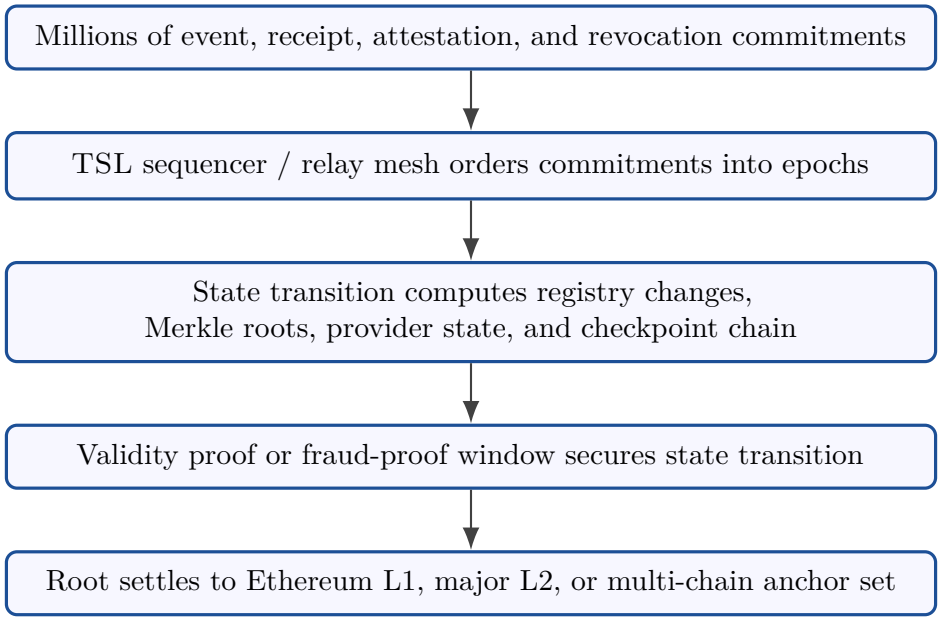

Figure 2: Long-term scale path: TSL app-rollup or trust-settlement network.

11.4 Multi-Backend Verification

The product should not expose chain complexity to users. A verifier should call one function:

```
verifyCommitment(event_commitment, proof, checkpoint) -> VerificationResult
```

Internally, the checkpoint may settle to an Ethereum L2, a TSL rollup, another chain, or a timestamping backend. The app should not care.

12 No-Integration Distribution

No-integration distribution means TSL can propagate through existing channels before platforms add native support. The protocol should succeed first as an envelope, proof link, SDK, and verifier, not as a new place users must visit.

The protocol should work even when platforms do nothing.

12.1 Proof Links

Any message can carry a proof link:

```
https://verify.tsl.network/p/bafy...
tsl://proof/bafy.../event
```

This is the zero-integration wedge. A user or agent can attach a proof link to email, LinkedIn, Discord, Slack, Telegram, X, GitHub, a marketplace chat, API payload, or agent transcript. The proof link is not a destination product; it is a portable verification artifact.

12.2 Standard Envelope

Applications that want native support can embed a standard envelope.

```
{
  "tsl_envelope": {
    "version": "1.0",
    "event_commitment": "0x...",
    "sender": "did:tsl:eip155-8453:0xabc...",
    "signature": "0x...",
    "proof_uri": "ipfs://...",
    "checkpoint": "0x...",
    "disclosure_policy": "commitment_only"
  }
}
```

12.3 Portable Proof Bundle

A proof link SHOULD resolve to a portable proof bundle. The bundle is the implementation anchor for offline verification, browser-extension rendering, enterprise archival, and third-party audit. A verifier MUST be able to ignore optional members it does not understand while still enforcing the version rules for any object it verifies.

```
{
  "type": "tsl.proof_bundle.v1",
  "bundle_id": "0x...",
  "created_at": "2026-05-26T00:00:00Z",
  "envelope": {"type": "tsl.event_commitment.v1"},
  "proof": {"type": "tsl.inclusion_proof.v1"},
  "checkpoint": {"type": "tsl.batch_checkpoint.v1"},
  "identity": {"type": "tsl.identity.v1"},
  "receipts": [],
  "attestations": [],
  "revocations": [],
  "assessment": null,
  "assessment_v2": null,
  "zk_proofs": [],
  "delegations": [],
  "audit_findings": [],
  "governance_policy": {},
  "redaction_manifest": {
    "raw_content_included": false,
    "exact_counterparties_included": false,
    "metadata_fields_redacted": ["platform", "ip_address", "user_agent"]
  }
}
```

The bundle MUST NOT include raw content, exact private counterparties, salts, private graph features, or restricted metadata unless the subject or authorized controller has given explicit disclosure consent

under the privacy consent rules. The `redaction_manifest` is normative for UI and audit: it tells the verifier which fields were intentionally absent, not merely missing.

12.4 Reference Clients and Interfaces

Reference clients and protocol interfaces can be:

- browser extension,
- mobile wallet,
- desktop app,
- enterprise gateway,
- SDK embedded in applications,
- AI-agent sidecar,
- command-line verifier for developers.

The client performs:

- key generation,
- signing,
- local encrypted storage,
- envelope detection,
- inclusion verification,
- revocation checks,
- receipt requests,
- trust-score rendering,
- selective disclosure.

12.5 Relay API

```
POST /v1/commitments
GET  /v1/proofs/{commitment}
GET  /v1/proof-bundles/{bundle_id}
POST /v1/receipts
GET  /v1/identity/{trust_id}
GET  /v1/revocation/{trust_id}
POST /v1/attestations
GET  /v1/checkpoints/{epoch}/{shard}
GET  /v1/scoring-profiles/{profile_id}
GET  /v1/model-cards/{model_id}
POST /v1/delegations/verify
POST /v1/verify
```

This is how TSL avoids app-by-app sprawl. Every product integrates with the same substrate.

13 Identity, Recovery, and Smart Accounts

Human users cannot be expected to manage raw private keys perfectly. TSL should support several custody modes:

Mode	Use Case
Local keypair	Developers, power users, pseudonymous identities
Passkey-backed key	Consumer onboarding and device-bound signing
Smart account	Recovery, policy, session keys, delegated signing
Enterprise account	Organizations, teams, support desks, agent fleets
Agent account	Scoped authority for autonomous agents and bots
Hardware key	High-value identities and maintainers

Smart accounts are especially important because TSL needs:

- key rotation without identity loss,
- social or institutional recovery,
- multi-device signing,
- organization-managed subkeys,
- scoped agent keys,
- delegated session keys,
- revocation of compromised devices,
- sponsored gas for ordinary users.

Recovery Principle

A user should be able to lose a device without losing years of continuity. A user should be able to revoke a compromised key without erasing trustworthy history.

14 Privacy Model

TSL must be private by default. A trust protocol that becomes a surveillance protocol will fail technically, socially, and legally.

14.1 Privacy Levels

Level	Publicly Visible	Private / Local
Level 0: Local only	Nothing	Raw messages, metadata, graph, scoring
Level 1: Commitments	Hashes, timestamps, signatures, roots	Message content, counterparties, metadata
Level 2: Selective proof	Specific disclosed claim	Everything outside the claim
Level 3: Aggregate proof	Counts, ranges, thresholds	Exact counterparties and messages
Level 4: Public identity	Public attestations and score	Any non-disclosed private context

14.2 Commitments

For message m, metadata μ, receiver identity r, and salts s_m, s_μ, s_r:

$$c_m = \text{Hash}(\text{domain} \parallel m \parallel s_m), \tag{3}$$
$$c_\mu = \text{Hash}(\text{metadata-domain} \parallel \mu \parallel s_\mu), \tag{4}$$
$$c_r = \text{Hash}(\text{receiver-domain} \parallel r \parallel s_r). \tag{5}$$

Salting and domain separation prevent common correlation and preimage attacks against predictable content.

14.3 Selective Disclosure

A user should be able to prove statements such as:

- I have controlled this TrustID for more than two years.
- This message belongs to a signed continuity chain.
- I have more than 50 reciprocal receipts.
- This business key is not revoked.
- This agent action was signed under a valid delegation.
- This seller has completed more than 100 transactions without disclosing buyer identities.

14.4 Privacy Consent Enforcement Rules

Privacy policy is not only a statement; it is an implementation gate. A compliant client MUST enforce disclosure consent before exporting proof bundles, uploading feature inputs, or revealing private metadata to a provider.

Control	Required Behavior
Consent record	Store a signed or locally encrypted `tsl.disclosure_consent.v1` record with subject, verifier, field classes, purpose, expiry, and revocation pointer.
Local block	If consent is absent, the client MUST block raw messages, salts, exact counterparties, private notes, protected attributes, IP addresses, user agents, and precise location from leaving local storage.
Upload filter	Provider uploads MUST be produced through an allowlist transform, not a denylist transform. Fields not explicitly allowed are omitted.
Bundle redaction	Proof bundles MUST include a `redaction_manifest` listing omitted sensitive classes and MUST NOT include salts unless the verifier is authorized to open a commitment.
User warning	Before disclosure, UX MUST state what will be revealed, to whom, for what purpose, for how long, and whether the disclosure can increase correlation risk.
Revocation	A subject MAY revoke future disclosures. Past third-party copies cannot be technically erased, so the UX MUST not imply retroactive deletion.

```
{
  "type": "tsl.disclosure_consent.v1",
  "subject": "did:tsl:eip155-8453:0xabc...",
  "verifier_or_provider": "did:tsl:provider:continuity-labs",
  "allowed_field_classes": ["aggregate_counts", "receipt_thresholds", "attestation_status"],
  "forbidden_field_classes": ["raw_content", "exact_counterparties", "protected_attributes",
    "ip_address"],
  "purpose": "anti_phishing_assessment",
  "issued_at": "2026-05-26T00:00:00Z",
  "expires_at": "2026-06-26T00:00:00Z",
  "revocation_pointer": "0x...",
  "signature": "subject_or_controller_signature"
}
```

14.5 Zero-Knowledge Roadmap

Later versions can add zero-knowledge proofs for threshold and set-membership claims:

- prove age over threshold without revealing exact creation time,
- prove reciprocal interaction count without revealing counterparties,
- prove membership in a vetted organization without revealing internal records,
- prove absence from a revocation set,
- prove distance from known malicious clusters using privacy-preserving approximations,
- prove an agent operated within delegated scope.

15 Trust Graph and Behavioral Geometry

Let $G_t = (V_t, E_t)$ be the communication and receipt graph at time t. Nodes are TrustIDs. Edges are signed events, co-signed receipts, attestations, transactions, or agent calls.

Each identity has a behavioral history:

$$H_i(t) = \{e_{i,1}, e_{i,2}, \ldots, e_{i,n(t)}\}. \tag{6}$$

A feature extractor maps the identity, graph, and history into a latent vector:

$$x_i(t) = \phi(v_i, G_t, H_i(t), A_i(t), R_i(t)), \tag{7}$$

where $A_i(t)$ is the attestation set and $R_i(t)$ is revocation/recovery state.

15.1 Feature Families

Feature Family	Example Signals
Cryptographic	valid signatures, key age, rotations, revocations, delegation scope
Temporal	account age, activity cadence, burstiness, dormant reactivation, response latency
Graph	degree, reciprocity, clustering, PageRank, trusted-neighbor ratio, community diversity
Receipts	co-signed interactions, completed transactions, disputes, counterparty diversity
Attestations	issuer quality, claim freshness, negative warnings, organizational membership
Behavioral drift	sudden target change, outbound spike, language/style shift if permitted
Adversarial proximity	similarity to known scam/Sybil/bot clusters
Local context	prior relationship with verifier, personal allowlist, private history

15.2 Trust Manifolds

Let $\mathcal{M}_T$ be the manifold of organic trusted behavior and $\mathcal{M}_A$ be the manifold of adversarial behavior.

$$D_T(i,t) = \operatorname{dist}(x_i(t), \mathcal{M}_T), \tag{8}$$
$$D_A(i,t) = \operatorname{dist}(x_i(t), \mathcal{M}_A), \tag{9}$$
$$\operatorname{Drift}_i(t) = \|x_i(t) - x_i(t - \Delta t)\|. \tag{10}$$

A reference score can be written:

$$S_i(t) = 100 \cdot \operatorname{sigmoid}\left(-\alpha D_T(i,t) + \beta D_A(i,t) - \gamma \operatorname{Drift}_i(t) + \delta \operatorname{Attest}_i(t) + \eta R_i(t)\right). \tag{11}$$

This score must be explainable, probabilistic, and provider-specific. TSL should never claim mathematical omniscience.

Research Math vs. Canonical Implementation

The manifold, sigmoid, entropy, covariance, Mahalanobis-distance, and AUROC/ECE formulas in this section are research and evaluation notation. They describe model intent and audit criteria, not signed wire-format arithmetic. A reference implementation MUST emit signed protocol objects using deterministic fixed-point integers, usually basis points or milli-units, and MUST define exact rounding in the algorithm registry and test vectors. Real-valued equations MAY guide model development, but they are not canonical serialization rules.

15.3 Reference MVP Score

Before advanced graph neural networks, use transparent scoring:

$$\begin{aligned}
\text{score} = {} & 0.20 \cdot \text{crypto_validity} && (12)\\
& + 0.15 \cdot \text{identity_age} && (13)\\
& + 0.15 \cdot \text{reciprocity} && (14)\\
& + 0.15 \cdot \text{trusted_neighbor_ratio} && (15)\\
& + 0.10 \cdot \text{receipt_quality} && (16)\\
& + 0.10 \cdot \text{attestation_quality} && (17)\\
& + 0.10 \cdot \text{temporal_consistency} && (18)\\
& + 0.05 \cdot \text{local_relationship}\,. && (19)
\end{aligned}$$

Map scores to cautious labels:

Score	Label
90–100	Trusted
75–89	Likely trusted
55–74	Medium trust
35–54	Unknown / caution
15–34	Suspicious
0–14	High risk
Special	Revoked / compromised / insufficient evidence

16 Trust Assessment as a Signed Object

A trust score should not be a universal truth. It should be a signed assessment from a model, provider, or local verifier.

```
{
  "type": "tsl.trust_assessment.v1",
  "subject": "did:tsl:eip155-8453:0xabc...",
  "issuer": "did:tsl:scoring-provider:continuity-labs",
  "score": 82,
  "label": "likely_trusted",
```

```
  "model_version": "graph-risk-v0.3.1",
  "evidence_commitment": "0x...",
  "features_disclosed": [
    "valid_signature",
    "reciprocal_receipt_history",
    "low_scam_cluster_similarity"
  ],
  "explanation": [
    "Signature is valid under current key registry",
    "Identity has 31 months of continuity",
    "Counterparty receipts are diverse and reciprocal",
    "No active revocation or compromise warning"
  ],
  "issued_at": "2026-05-25T00:05:00Z",
  "expires_at": "2026-06-25T00:05:00Z",
  "signature": "provider_signature"
}
```

Plural Scoring

The protocol should allow multiple scoring providers, local models, domain-specific models, and user-selected risk policies. The base protocol settles evidence; it should not become one centralized reputation oracle.

17 Trust-Scoring Science: Hyper-Specific Normative Implementation

The base protocol proves signed facts. The scoring layer estimates contextual risk from those facts. A compliant implementation MUST preserve the following invariant:

$$\text{Protocol validity} \neq \text{trustworthiness} \neq \text{humanity} \neq \text{lawfulness}. \tag{20}$$

A TSL verifier MUST first compute cryptographic validity. A trust model MAY then compute a signed assessment. A failed cryptographic gate MUST NOT be repaired by a high score.

Robust Variant

The most robust implementation is not one global reputation score. It is a set of domain-specific, signed, calibrated, expiring assessments over versioned evidence profiles. The protocol settles evidence; scoring providers compete on accuracy, calibration, privacy, auditability, and appeal quality.

17.1 Three Implementation Variants

Variant	Use	Required Properties
MVP transparent scorer	Early demos, proof links, private beta	Hand-weighted features, no raw private data, signed scores, expiration, local override, explicit insufficient-evidence labels.
Production calibrated scorer	Enterprise APIs, marketplaces, agent workflows	Versioned feature registry, domain thresholds, time-split evaluation, calibration report, confidence interval, appeal path, provider registry entry.
High-assurance scorer	Banking, large payments, supply chain, regulated workflows	Independent audit, red-team pass, shadow deployment before promotion, privacy leakage report, reproducible evaluation package, emergency rollback, human review for severe negative labels.

17.2 Hard Gates Before Any Score

The verifier computes a gate vector:

$$g_i = (schema, canon, sig, key, revocation, inclusion, checkpoint, settlement, delegation). \tag{21}$$

The default hard gate is:

$$G_i = schema \wedge canon \wedge sig \wedge key \wedge \neg revocation \wedge inclusion \wedge checkpoint. \tag{22}$$

If the verifier policy requires chain finality, then:

$$G_i^{settled} = G_i \wedge settlement. \tag{23}$$

A scoring provider MUST return one of the following before numeric scoring:

Gate Outcome	Required Behavior
Valid core proof	Numeric trust assessment MAY be computed.
Invalid signature	Return `cryptographic_failure`; no ordinary score.
Revoked key	Return `revoked_or_compromised`; no ordinary score.
Missing inclusion proof	Return `unsettled_or_unproven`; risk score MAY be suppressed.
Settlement required but absent	Return `settlement_missing`; verifier policy decides accept or reject.
Delegation absent for agent action	Return `delegation_missing`; no action authorization.
Sparse evidence but no adverse signal	Return `insufficient_evidence`, not `suspicious`.

17.3 Domain Policies

Every score MUST be bound to a domain policy. The same evidence can imply different action in different domains.

```
{
  "type": "tsl.domain_policy.v1",
  "domain": "agent_payments",
  "policy_version": "agent-payments-2026-05-26",
  "requires_settlement": true,
  "requires_delegation_check": true,
  "requires_content_opening": false,
  "step_up_above_value_minor_units": 100000,
  "max_assessment_age_seconds": 3600,
  "false_positive_cost_class": "medium",
  "false_negative_cost_class": "critical",
  "sparse_identity_default": "unknown_caution",
  "accepted_provider_status": ["active", "probation_allowed"],
  "human_review_required_labels": ["public_high_risk", "fraud_claim"]
}
```

Domain	Primary Failure	Threshold Bias	Default Action
Anti-phishing	False negative	Warn early; tolerate more caution labels	Block or warn on hard proof failures and strong adversarial proximity.
Marketplace trust	Both sides	Balance buyer safety and seller false positives	Use evidence coverage, dispute rates, and appealable warnings.
Agent payments	False negative	Extremely conservative above value thresholds	Require delegation, settlement, and step-up approval.
Open-source supply chain	False positive	Protect legitimate maintainer changes	Use drift as step-up trigger, not automatic accusation.
Professional identity	False positive	Avoid defamation and caste effects	Prefer continuity proofs and private verifier context.
Customer support	False negative	Verify organization authority first	Require org membership and current delegation.

17.4 Scoring Profile Object

A scoring profile is the executable contract between provider, verifier, auditor, and user.

```
{
  "type": "tsl.scoring_profile.v2",
  "profile_id": "did:tsl:provider:continuity-labs/profile/anti-phishing-v2.1.0",
  "provider": "did:tsl:provider:continuity-labs",
  "domain": "anti_phishing",
  "model_family": "transparent_weighted_logistic",
  "model_version": "2.1.0",
  "feature_registry_uri": "ipfs://bafy.../feature_registry.json",
  "feature_registry_commitment": "0x...",
```

```
  "normalization_profile_commitment": "0x...",
  "weight_profile_commitment": "0x...",
  "calibration_profile_commitment": "0x...",
  "threshold_policy_commitment": "0x...",
  "privacy_policy_commitment": "0x...",
  "evaluation_report_commitment": "0x...",
  "training_data_statement_commitment": "0x...",
  "appeal_policy_uri": "https://provider.example/appeals/anti-phishing-v2",
  "issued_at": "2026-05-26T00:00:00Z",
  "valid_after": "2026-05-26T00:00:00Z",
  "expires_at": "2026-08-26T00:00:00Z",
  "signature": "provider_signature"
}
```

17.5 Feature Definition Schema

Every feature MUST be named, typed, normalized, privacy-classified, and attack-modeled.

```
{
  "type": "tsl.feature_definition.v2",
  "feature_id": "graph.reciprocity_bps.v1",
  "name": "reciprocity_bps",
  "family": "graph",
  "raw_definition": "2*min(w_ij,w_ji)/(w_ij+w_ji+eps) aggregated over counterparties",
  "value_type": "integer_bps",
  "range": [0, 10000],
  "direction": "higher_is_safer",
  "required_evidence": ["verified_receipts"],
  "forbidden_inputs": ["raw_message", "exact_private_counterparty_without_consent"],
  "normalizer_id": "bounded_ratio_identity_v1",
  "default_weight_bps": 1200,
  "missing_value_policy": "impute_zero_with_coverage_penalty",
  "stability_window_days": 90,
  "privacy_class": "aggregate",
  "audit_level": "recomputable_from_disclosed_or_provider_visible_evidence",
  "known_attacks": ["receipt_farming", "collusive_counterparties", "replay_attempts"],
  "mitigations": ["counterparty_diversity", "seed_escape", "temporal_decay"]
}
```

17.6 Exact Reference Feature Set

Let i be the subject, t be the assessment time, e be a verified event, c_e be its class, τ_e be its timestamp, and $\omega_e(t)$ be its effective graph weight. Let $\varepsilon = 10^{-9}$.

Feature	Formula	Implementation Rule
Cryptographic gate	$f_{crypto} = \mathbb{I}[G_i = 1]$	Hard gate. If zero, suppress ordinary score.
Identity age	$1 - e^{-\min(a_i,730)/180}$	a_i is age in days since registry creation or verified local creation.

Active key age	$1 - e^{-\min(k_i, 365)/90}$	Recent key age is downweighted unless rotation has valid recovery proof.
Signed event count	$\frac{\log(1+n_i)}{\log(1+1000)}$ clipped	Counts only valid, non-replayed, included events.
Receipt count	$\frac{\log(1+r_i)}{\log(1+300)}$ clipped	Counts only co-signed receipts whose counterparties are distinct after clustering.
Reciprocity	$\frac{\sum_j \min(w_{ij}, w_{ji})}{\sum_j \max(w_{ij}, w_{ji}) + \varepsilon}$	One-way broadcasts do not become strong continuity.
Counterparty diversity	$1 - \sum_j p_{ij}^2$	Uses trust-weighted edge mass; penalizes closed farms.
Community escape	$\frac{\sum_{j:comm(j) \neq comm(i)} w_{ij}}{\sum_j w_{ij} + \varepsilon}$	Low escape is a Sybil warning when paired with density and synchronization.
Trusted neighbor mass	$\frac{\sum_j w_{ij} S_j q_j}{\sum_j w_{ij} + \varepsilon}$	Neighbor score S_j and coverage q_j MUST be model-versioned.
Dispute rate	$\frac{d_i}{r_i + d_i + \varepsilon}$	Disputes require evidence commitments and appeal state.
Attestation quality	$\text{clip}_{0,1} \sum_a Q_a \kappa_a d_a$	Expired and reversed attestations contribute zero or negative weight.
Revocation risk	$\mathbb{I}[\textit{recent compromise}]$	Triggers warning window after compromise event.
Cadence stability	$1 - \text{clip}_{0,1}(MADZ_i/8)$	Robust z-score over activity intervals.
Dormant reactivation	$\mathbb{I}[inactive > 90d \wedge burst > p95]$	Step-up trigger, not automatic fraud.
Adversarial proximity	$e^{-D_A^2/\sigma_A^2}$	Risk penalty; must disclose cluster source class.
Local relationship	private verifier value	MUST remain local unless user discloses.

All clipped features are clipped to $[0, 1]$. Output features stored in protocol objects SHOULD be represented as integer basis points from 0 to 10000.

17.7 Normalization Profiles

For bounded ratios:

$$N_k(z) = \text{clip}_{0,1}(z). \tag{24}$$

For counts:

$$N_k(z) = \text{clip}_{0,1}\left(\frac{\log(1+z) - \log(1+p_{1,k})}{\log(1+p_{99,k}) - \log(1+p_{1,k}) + \varepsilon}\right). \tag{25}$$

For heavy-tailed positive values:

$$N_k(z) = \text{clip}_{0,1}\left(\frac{\text{asinh}(z/s_k)}{\text{asinh}(p_{99,k}/s_k)}\right). \tag{26}$$

For risk penalties where higher means worse:

$$Safe_k(z) = 1 - N_k(z). \tag{27}$$

The normalization profile MUST include the training window, percentiles, missing-value policy, and whether parameters were fitted on a time-split training set.

17.8 Reference Weight Profile

The robust default separates gates, positive evidence, penalties, and uncertainty. This is an implementation profile, not a universal truth.

Component	Weight	Rule
Cryptographic validity	Gate	Required before ordinary scoring.
Identity and key continuity	1400 basis points	Age helps, but cannot dominate because old identities can be compromised.
Receipts and reciprocity	2200 basis points	Co-signed diverse receipts are the core scarce signal.
Counterparty and community diversity	1400 basis points	Prevents farms from converting volume into trust.
Attestation quality	1000 basis points	Useful but issuer-abuse bounded.
Temporal consistency	900 basis points	Detects compromise and laundering.
Trusted-neighbor mass	900 basis points	Useful with coverage weighting and seed governance.
Domain-specific feature	700 basis points	Example: dispute rate for marketplace, delegation scope for agents.
Local relationship	500 basis points	Private verifier context; never globalized by default.
Adversarial proximity penalty	up to 2500 basis points	Strong in anti-phishing, weaker in high-false-positive domains.
Drift penalty	up to 1500 basis points	Usually produces step-up, not permanent label.

The raw score is:

$$z_i = b + \sum_{k \in K^+} w_k N_k(f_{ik}) - \sum_{\ell \in K^-} u_\ell N_\ell(r_{i\ell}) - u_q(1 - q_i). \tag{28}$$

The calibrated score is:

$$S_i = \text{round}(10000 \cdot C(z_i)), \tag{29}$$

where C MUST be a declared monotone calibration function.

17.9 Evidence Coverage and Abstention

Evidence coverage measures whether the provider has enough verified evidence to make a confident claim.

$$q_i = G_i \cdot \min\left(1, \frac{n_{events}}{25}\right)^{0.25} \cdot \min\left(1, \frac{n_{receipts}}{10}\right)^{0.35} \cdot \min\left(1, \frac{n_{counterparties}}{5}\right)^{0.40}. \tag{30}$$

A provider MUST abstain or use caution labels when coverage is low:

Coverage	Label Policy	Meaning
$q_i < 0.25$	Insufficient evidence	Do not infer suspiciousness from newness.
$0.25 \le q_i < 0.50$	Unknown or caution	Score MAY be shown with wide interval.
$0.50 \le q_i < 0.75$	Medium confidence	Ordinary labels allowed except highest trust.
$q_i \ge 0.75$	High confidence	Full label range allowed if calibration supports it.

```
{
  "type": "tsl.evidence_coverage.v1",
  "subject": "did:tsl:eip155-8453:0xabc...",
  "computed_at": "2026-05-26T00:00:00Z",
  "valid_signed_event_count": 184,
  "valid_receipt_count": 62,
  "unique_counterparty_count": 27,
  "distinct_community_count": 6,
  "attestation_count": 4,
  "recent_revocation_count": 0,
  "coverage_bps": 8600,
  "coverage_label": "high",
  "missing_evidence": []
}
```

17.10 Calibration and Confidence Intervals

Providers MUST report calibration on held-out time-split data. Expected calibration error is:

$$ECE = \sum_{b=1}^{B} \frac{|B_b|}{n} \left| acc(B_b) - conf(B_b) \right|. \tag{31}$$

A score assessment SHOULD include a confidence interval. For transparent models, bootstrap over edges, counterparties, and attestations:

$$CI_{95}(S_i) = \left[Q_{0.025}(S_i^*), Q_{0.975}(S_i^*) \right]. \tag{32}$$

For sparse identities, the interval MUST widen by a coverage penalty:

$$width_{adj} = width_{boot} + \lambda_q(1 - q_i). \tag{33}$$

17.11 Thresholds and False-Positive / False-Negative Tradeoff

A label threshold MUST be derived from a domain cost model:

$$L(\theta) = c_{FP}FPR(\theta) + c_{FN}FNR(\theta) + c_A A(Abstain(\theta)) + c_{Appeal}AppealRate(\theta). \tag{34}$$

The reference labels are:

Label	Required Rule	Notes
Trusted	$S \geq 9000$, lower CI ≥ 8000, $q \geq 0.75$	Never shown for sparse identities.
Likely trusted	$7500 \leq S < 9000$, $q \geq 0.50$	Good continuity, not a guarantee.
Medium trust	$5500 \leq S < 7500$	Mixed or moderate evidence.
Unknown caution	$3500 \leq S < 5500$ or low coverage	Default for legitimate new users.
Suspicious	$1500 \leq S < 3500$ and adverse evidence present	Requires adverse signal beyond newness.
High risk	$S < 1500$ or hard negative proof	Reserved for revoked, compromised, scam-like, or severe abuse evidence.
Cryptographic failure	Any failed hard gate	Separate from score.

17.12 Signed Trust Assessment Object

```
{
  "type": "tsl.trust_assessment.v2",
  "assessment_id": "0x...",
  "subject": "did:tsl:eip155-8453:0xabc...",
  "issuer": "did:tsl:provider:continuity-labs",
  "domain": "anti_phishing",
  "scoring_profile_id": "did:tsl:provider:continuity-labs/profile/anti-phishing-v2.1.0",
  "model_version": "2.1.0",
  "gate_result": {
    "schema_valid": true,
    "signature_valid": true,
    "key_active": true,
    "not_revoked": true,
    "included_in_log": true,
    "checkpoint_valid": true,
    "settlement_satisfied": true
  },
  "score_bps": 8420,
  "confidence_interval_bps": [7660, 9010],
  "coverage_bps": 8600,
  "label": "likely_trusted",
```

```
  "reason_codes": [
    "valid_current_key",
    "diverse_reciprocal_receipts",
    "low_dispute_rate",
    "no_recent_compromise_revocation"
  ],
  "risk_codes": [],
  "feature_vector_commitment": "0x...",
  "evidence_coverage_commitment": "0x...",
  "privacy_disclosure_level": "aggregate_only",
  "appeal_uri": "https://provider.example/appeals/0x...",
  "issued_at": "2026-05-26T00:00:00Z",
  "expires_at": "2026-06-26T00:00:00Z",
  "signature": "provider_signature"
}
```

17.13 Assessment Algorithm

```
function assessTrust(subject, evidence, profile, domainPolicy): TrustAssessmentV2 {
  gate = verifyHardGates(subject, evidence, domainPolicy)
  if gate.hasCryptographicFailure():
    return signedFailureAssessment(subject, gate)

  features = extractRegisteredFeatures(evidence, profile.featureRegistry)
  coverage = computeEvidenceCoverage(features, evidence)

  if coverage < domainPolicy.minimumCoverage and !features.hasAdverseEvidence:
    return signedAbstention(subject, gate, coverage, "insufficient_evidence")

  normalized = normalize(features, profile.normalizationProfile)
  rawScore = weightedScore(normalized, profile.weightProfile, coverage)
  calibrated = calibrate(rawScore, profile.calibrationProfile)
  interval = confidenceInterval(evidence, profile)
  label = threshold(calibrated, interval, coverage, profile.thresholdPolicy)

  return signAssessment({subject, gate, calibrated, interval, coverage, label})
}
```

18 Metadata Fingerprint Model: Privacy-Preserving Implementation

Metadata fingerprints are local, scope-limited summaries of continuity-relevant metadata. They are not public tracking identifiers. They MUST NOT become a cross-platform surveillance layer.

18.1 Implementation Variants

Variant	Storage	Disclosure
Local-only	Encrypted client store	Used only for local drift and relationship scoring.
Pairwise verifier	Derived per verifier domain	Stable only between subject and verifier; not reusable globally.
Provider aggregate	Provider receives buckets or proofs	Provider computes features without raw content or exact graph by default.
Public commitment	Salted commitment only	Proves existence or timestamp without opening metadata.
High-assurance private proof	Commitments plus ZK predicates	Proves threshold facts without disclosing raw metadata.

18.2 Allowed, Restricted, and Forbidden Metadata

Input	Default Rule	Notes
Event class	Allowed	Message, receipt, transaction, agent call, code release.
Coarse timestamp bucket	Allowed	Exact timestamp MAY remain local; public log epoch already leaks coarse timing.
Content length bucket	Restricted	Use coarse buckets only; exact size can leak content.
Counterparty commitment	Allowed with salt	Exact counterparty stays local unless disclosed.
Key lineage	Allowed	Key age and rotation history are protocol facts.
Response latency bucket	Restricted	Useful for continuity; risky if exact.
Platform or transport	Local by default	Public platform tags can enable correlation.
Language or style embedding	Opt-in only	Content-derived; should be local or explicitly disclosed.
IP address, precise location, device fingerprint	Forbidden by default	High privacy risk and weak continuity value.
Protected attributes	Forbidden	Race, religion, health, sexuality, political affiliation unless legally required and explicitly scoped.

18.3 Scope-Key Derivation and Rotation

A fingerprint key MUST be scoped. The same metadata tuple MUST NOT produce a reusable global identifier across verifiers.

$$k_{scope} = KDF(k_{master}, \texttt{"tsl-fp-v1"} \parallel context \parallel verifier\_domain \parallel epoch \parallel purpose). \tag{35}$$

Reference purposes are:

Purpose	Rule
Local only	Never disclosed; rotated every 90 days unless user pins a baseline.
Pairwise verifier	Unique per verifier domain; rotated every 180 days or on user request.
Provider ephemeral	Unique per scoring request or short epoch; prevents provider-side long-term linking.
Public commitment	No key disclosure; fresh salt per commitment.

18.4 Bucketization

Metadata MUST be minimized before fingerprinting. A reference tuple is:

$$\mu_t = (event\_class, time\_bucket, length\_bucket, \\ key\_lineage\_bucket, counterparty\_class, receipt\_class). \tag{36}$$

Reference buckets:

Field	Reference Buckets
Time	5 minute, 1 hour, 1 day, or local-only exact time depending on policy.
Content length	0–256 bytes, 257–1024, 1025–4096, 4097–16384, above 16384.
Response latency	below 1 minute, 1–10 minutes, 10–60 minutes, 1–24 hours, above 24 hours.
Counterparty class	local-known, public-org, verified-agent, unknown, private-undisclosed.
Key lineage	same-key, rotated-with-recovery, new-key-unverified, revoked.

18.5 Fingerprint and Commitment Construction

$$F_t = HMAC_{k_{scope}}(\texttt{tsl.metadata.fp.v1} \parallel canon(\mu_t)), \tag{37}$$

$$C_{F,t} = \text{Hash}(\texttt{tsl.metadata.commit.v1} \parallel F_t \parallel s_t), \tag{38}$$

$$R_F = \text{MerkleRoot}(C_{F,1}, \ldots, C_{F,n}). \tag{39}$$

The salt s_t MUST be a fresh 256-bit random value. A public fingerprint commitment MUST NOT include F_t itself.

18.6 Fingerprint Object

```
{
  "type": "tsl.metadata_fingerprint_commitment.v1",
  "subject": "did:tsl:eip155-8453:0xabc...",
  "scope_class": "pairwise_verifier",
  "scope_commitment": "0xhash_of_verifier_domain_and_epoch",
  "bucket_profile": "tsl.bucket_profile.default.v1",
  "fingerprint_commitment": "0x...",
  "salt_commitment": "0x...",
  "created_at_bucket": "2026-05-26T00:00:00Z/PT1H",
  "expires_at": "2026-08-26T00:00:00Z",
```

```
  "disclosure_policy": "selective_or_zk",
  "signature": "subject_signature"
}
```

18.7 Stability and Spoofing

Metadata features are classified by spoofing cost:

Class	Default Weight	Examples
Cryptographic stable	High	Key lineage, revocation status, signed event chain.
Receipt-backed stable	High	Co-signed receipts, completed transactions.
Attestation-backed	Medium	Organization membership, issuer claims.
Cadence-derived	Low to medium	Timing, response latency, burstiness.
Self-reported	Low	User-declared platform, role, or metadata tags.
Content-derived opt-in	Domain-specific	Style or language embeddings; never default global feature.

An adversary can shape cadence and length buckets. Therefore metadata-only evidence MUST NOT be enough for a high-trust label.

18.8 Correlation Risk Budget

A provider or client SHOULD estimate public correlation risk:

$$LeakageRisk = \alpha T_{precision} + \beta V_{volume} + \gamma C_{counterparty} + \delta P_{platform} + \eta U_{uniqueness}. \tag{40}$$

Reference policy:

Risk	Required Mitigation
Low	Ordinary salted commitments allowed.
Medium	Batch commitments, coarsen time, suppress platform tags.
High	Local-only or ZK threshold proof.
Severe	Do not publish; require pairwise verifier proof only.

19 Graph Geometry: Construction Rules, Profiles, and Manifolds

At time t, the trust graph is a typed, temporal, weighted multigraph:

$$G_t = (V_t, E_t, \mathcal{C}, \omega, \tau, \Pi), \tag{41}$$

where Π is the graph profile that defines edge types, weights, decay, community detection, seed rules, and privacy constraints.

19.1 Graph Implementation Variants

Variant	Graph Data	Use
Local verifier graph	User device or enterprise gateway	Personal relationship and private allowlist scoring.
Provider aggregate graph	Provider-visible commitments and disclosed proofs	General risk intelligence without raw private content.
Federated graph features	Multiple providers compute aggregates	Reduces centralized graph ownership.
Private graph proof	Commitments plus ZK or secure aggregation	High-assurance privacy for thresholds and distances.

19.2 Graph Profile Object

```
{
  "type": "tsl.graph_profile.v2",
  "profile_id": "graph-default-2026-05",
  "edge_weight_profile": "default_edge_weights_v2",
  "temporal_decay_profile": "default_decay_v2",
  "community_detection": {
    "algorithm": "leiden",
    "resolution_bps": 10000,
    "min_cluster_size": 20,
    "edge_weight_floor_bps": 500
  },
  "seed_sets": {
    "trusted_seed_policy": "audited_multi_provider_seed_v1",
    "adversarial_seed_policy": "evidence_bound_abuse_seed_v1"
  },
  "negative_edge_policy": {
    "requires_evidence_commitment": true,
    "requires_appeal_uri": true,
    "max_single_negative_weight_bps": 1500,
    "decay_days": 30
  },
  "privacy_policy": {
    "raw_counterparty_upload_required": false,
    "allows_pairwise_private_features": true
  }
}
```

19.3 Allowed Graph Edges

Edges MUST come from verifiable protocol objects or explicit local context. Scraped social links, follows, likes, profile similarity, and unverified contact lists MUST NOT be used as protocol graph edges.

Edge Type	Base Weight	Half-Life	Validation Rule

Unilateral signed event	0.10	90 days	Valid signature, active key, non-replayed nonce.
Received receipt	0.30	180 days	Receiver co-signature over event commitment.
Reply receipt	0.50	180 days	Reciprocal receipt or linked reply commitment.
Completed transaction	1.00	365 days	Completion proof and no unresolved dispute.
Disputed transaction	-0.50	30 days	Evidence commitment and appeal path required.
Positive attestation	$Q_j\kappa$	Claim-specific	Valid issuer, non-expired claim, issuer quality.
Negative attestation	$-Q_j\kappa$	7–90 days	Evidence, appeal, severity limit, reversal tracking.
Delegation edge	Scope-specific	Until expiry	Principal signature and active delegation.
Organization membership	0.70	Until expiry	Issuer status active; membership proof valid.
Code release signature	0.60	365 days	Maintainer or release-bot key active.

The effective edge weight is:

$$\omega_e(t) = \omega_{base}(c_e) \cdot q_{source(e)} \cdot d_{c_e}(t - \tau_e) \cdot \rho_e \cdot \chi_e, \tag{42}$$

where q_{source} is counterparty or issuer quality, d_c is decay, ρ_e is receipt status, and χ_e is an appeal/reversal multiplier.

19.4 Temporal Decay

Use exponential half-life decay:

$$d_c(\Delta) = 2^{-\Delta/h_c}. \tag{43}$$

Immediate state facts override decay. Revocation is not a decayed feature; it is a state transition.

19.5 Reciprocity, Diversity, and Concentration

Directional mass:

$$w_{ij}(t) = \sum_{e:i\to j} \omega_e(t). \tag{44}$$

Pair reciprocity:

$$r_{ij} = \frac{2\min(w_{ij}, w_{ji})}{w_{ij} + w_{ji} + \varepsilon}. \tag{45}$$

Counterparty mass distribution:

$$p_{ij} = \frac{w_{ij} + w_{ji}}{\sum_k (w_{ik} + w_{ki}) + \varepsilon}. \tag{46}$$

Diversity metrics:

$$HHI_i = \sum_j p_{ij}^2, \tag{47}$$

$$D_i = 1 - HHI_i, \tag{48}$$

$$H_i = -\sum_j p_{ij} \log(p_{ij} + \varepsilon), \tag{49}$$

$$N_i^{eff} = e^{H_i}. \tag{50}$$

A high volume of edges with low N^{eff} is weak evidence.

19.6 Community and Escape Metrics

For community assignment $comm(v)$:

$$InternalMass_i = \sum_{j:comm(j)=comm(i)} (w_{ij} + w_{ji}), \tag{51}$$

$$ExternalMass_i = \sum_{j:comm(j)\neq comm(i)} (w_{ij} + w_{ji}), \tag{52}$$

$$Escape_i = \frac{ExternalMass_i}{InternalMass_i + ExternalMass_i + \varepsilon}. \tag{53}$$

For a cluster C:

$$Density(C) = \frac{\sum_{i,j\in C} w_{ij}}{|C|(|C|-1) + \varepsilon}, \tag{54}$$

$$Conductance(C) = \frac{cut(C, \bar{C})}{\min(vol(C), vol(\bar{C})) + \varepsilon}, \tag{55}$$

$$SeedEscape(C) = \frac{\sum_{i\in C, j\in S_T} w_{ij}}{\sum_{i\in C, j} w_{ij} + \varepsilon}. \tag{56}$$

Low conductance, high internal density, low seed escape, and synchronized creation are Sybil evidence when observed together.

19.7 Trusted and Adversarial Manifolds

The organic manifold $\mathcal{M}_T$ is a distribution over trajectories with:

- long-lived key lineage or verified recovery,
- diverse reciprocal receipts,
- cross-community escape,
- low unresolved dispute rate,
- stable cadence with explainable changes,
- attestations from issuers with low reversal rates.

The adversarial manifold $\mathcal{M}_A$ is a distribution over trajectories with:

- synchronized identity creation,
- dense internal receipts and low external escape,
- shared issuer or device patterns where disclosure permits,
- abrupt target-community shifts,
- high outbound burstiness,
- proximity to evidence-bound abuse clusters.

Distance features MAY include:

$$D_T(i) = (x_i - \mu_T)^\top \Sigma_T^{-1} (x_i - \mu_T), \tag{57}$$

$$D_A(i) = \min_{a \in \mathcal{A}} \operatorname{dist}(x_i, x_a), \tag{58}$$

$$D_{diff}(i, S) = 1 - PPR_S(i), \tag{59}$$

$$D_{cluster}(i) = \min_{C \in \mathcal{C}_A} \operatorname{dist}(x_i, \mu_C). \tag{60}$$

These are statistical features, not proof of fraud.

Fixed-Point Graph Outputs

Graph math MAY be estimated with real-valued methods during research. Any graph feature written into a signed or hashed TSL object MUST be converted to integer fields such as `reciprocity_bps`, `counterparty_hhi_bps`, or `effective_counterparty_count_milli`. Floating-point values MUST NOT appear in signed graph-profile, graph-feature, Sybil, drift, model-card, or evaluation-report objects.

19.8 Graph Feature Output

```
{
  "type": "tsl.graph_feature_vector.v1",
  "subject": "did:tsl:eip155-8453:0xabc...",
  "graph_profile_id": "graph-default-2026-05",
  "computed_at": "2026-05-26T00:00:00Z",
  "reciprocity_bps": 8120,
  "counterparty_diversity_bps": 7740,
```

```
  "effective_counterparty_count_milli": 18600,
  "community_escape_bps": 6320,
  "trusted_neighbor_mass_bps": 7010,
  "cluster_concentration_bps": 1280,
  "adversarial_proximity_bps": 420,
  "privacy_level": "aggregate_no_exact_counterparties",
  "feature_commitment": "0x..."
}
```

20 Sybil Detection and New Identity Handling

Sybil detection in TSL is not a binary claim that an identity is fake. It is a cost-aware estimate that a set of identities is controlled, coordinated, or inflated in a way that should reduce trust or require step-up verification.

20.1 Threat Budget Tiers

Tier	Capability	Examples
B0	Cheap fake identities	Scripted accounts, no durable receipts.
B1	Internal receipt farm	Controlled identities co-sign each other.
B2	Purchased aged identities	Old accounts or keys acquired from others.
B3	Compromised real identities	Stolen keys, session hijack, device compromise.
B4	Corrupt or weak issuer	False attestations from issuer-like accounts.
B5	Relay, provider, or auditor collusion	Checkpoint manipulation, model abuse, selective visibility.

20.2 Sybil Resistance Assumptions

The protocol assumes:

1. uncompromised signatures cannot be forged,
2. diverse reciprocal receipts from honest regions are costly to obtain,
3. audited issuers face penalties for false attestations,
4. append-only checkpoints prevent undetectable history rewriting,
5. at least one honest auditor or verifier observes conflicting public checkpoint views,
6. scoring providers are plural and challengeable.

20.3 Cluster Metrics

For a candidate cluster C:

$$CreationSync(C) = 1 - \frac{MAD(created\_at(C))}{T_{sync} + \varepsilon}, \tag{61}$$

$$InternalRatio(C) = \frac{\sum_{i,j \in C} w_{ij}}{\sum_{i \in C, j} w_{ij} + \varepsilon}, \tag{62}$$

$$ExternalDiversity(C) = 1 - \sum_{q} P_{Cq}^2, \tag{63}$$

$$IssuerReuse(C) = \max_{m} \frac{|\{i \in C : issuer(i) = m\}|}{|C|}, \tag{64}$$

$$ReceiptSymmetry(C) = \frac{\sum_{i,j \in C} \min(w_{ij}, w_{ji})}{\sum_{i,j \in C} \max(w_{ij}, w_{ji}) + \varepsilon}, \tag{65}$$

$$SeedEscape(C) = \frac{\sum_{i \in C, j \in S_T} w_{ij}}{\sum_{i \in C, j} w_{ij} + \varepsilon}. \tag{66}$$

Reference Sybil cluster risk:

$$\begin{aligned} Risk(C) = \sigma(&\alpha_0 + \alpha_1 InternalRatio + \alpha_2 CreationSync + \alpha_3 IssuerReuse \\ &+ \alpha_4 ReceiptSymmetry - \alpha_5 SeedEscape - \alpha_6 ExternalDiversity). \end{aligned} \tag{67}$$

20.4 Cost-of-Attack Model

An attacker seeking score S^* must buy or create evidence:

$$Cost(S^*) = C_{ids} + C_{time} + C_{external\_receipts} + C_{attestations} + C_{compromise} + C_{evasion}. \tag{68}$$

The robust design objective is:

$$Cost(S^*) > ExpectedBenefit(S^*) \tag{69}$$

for high-risk actions in target domains.

20.5 Collusion Simulation Profile

Every production scorer SHOULD test at least these synthetic attacks:

```
{
  "type": "tsl.sybil_simulation_profile.v1",
  "attack_scenarios": [
    {"name": "dense_internal_farm", "identities": 1000, "internal_receipt_rate_bps": 8000},
    {"name": "slow_aged_farm", "identities": 500, "aging_days": 180, "low_rate_edges": true},
    {"name": "bridge_attack", "identities": 200, "honest_bridge_edges": 20},
    {"name": "issuer_collusion", "identities": 300, "weak_issuer_fraction_bps": 2500},
    {"name": "compromised_real_accounts", "identities": 50, "drift_after_day": 90}
  ],
  "success_metric": "fraction_reaching_likely_trusted_label",
  "required_detection_rate_bps": 8500
}
```

20.6 Bootstrap Rules for New Identities

New identities MUST be handled as low-evidence, not inherently malicious.

Stage	Requirements	Allowed Label Ceiling
Fresh	Valid key, no receipts	Unknown caution.
Seeded	Verified organization, inviter, or local relationship	Medium trust unless receipts exist.
Emerging	At least 5 reciprocal receipts and 3 counterparties	Likely trusted MAY be possible in narrow local context.
Established	At least 25 events, 10 receipts, 5 counterparties	Full label range possible subject to calibration.
High-value actor	Strong attestations or long history	Full label range, stricter drift monitoring.

20.7 Sybil Assessment Object

```
{
  "type": "tsl.sybil_assessment.v1",
  "subject": "did:tsl:eip155-8453:0xabc...",
  "cluster_id_commitment": "0x...",
  "computed_at": "2026-05-26T00:00:00Z",
  "adversary_tier_assumed": "B2",
  "cluster_size_bucket": "100-500",
  "cluster_concentration_bps": 2200,
  "trusted_escape_bps": 7100,
  "internal_receipt_ratio_bps": 2200,
  "creation_sync_bps": 900,
  "issuer_reuse_bps": 300,
  "external_diversity_bps": 6900,
  "attack_cost_minor_units": 2500000,
  "risk_score_bps": 1820,
  "risk_label": "low",
  "privacy_level": "cluster_commitment_only",
  "signature": "provider_signature"
}
```

21 Behavioral Drift and Compromise Detection

Behavioral drift is a change in trajectory. It is not automatically bad. The robust implementation treats drift as a step-up signal unless it is combined with hard evidence of compromise or adversarial activity.

21.1 Drift Windows

Window	Reference Duration	Purpose
Immediate	1 hour	Burst, spam, unusual agent tool use.
Short	24 hours	Compromise indicators and sudden target change.
Medium	30 days	New community, new transaction pattern, new cadence.
Long	180 days	Baseline continuity and seasonal behavior.
Dormant	90 or more inactive days	Reactivation risk; requires special handling.

21.2 Drift Vector

The drift vector is grouped so explanations are actionable:

$$\Delta_i = (\Delta_{key}, \Delta_{graph}, \Delta_{action}, \Delta_{cadence}, \Delta_{claim}, \Delta_{agent}, \Delta_{local}). \tag{70}$$

Component	Examples
Key drift	New active key, recovery event, device-key churn, post-revocation behavior.
Graph drift	New counterparties, old counterparties absent, new community, seed distance change.
Action drift	New transaction type, new code release behavior, new outbound message class.
Cadence drift	Burstiness, dormant reactivation, unusual time bucket, response latency shift.
Claim drift	New negative attestation, expired org membership, conflicting attestations.
Agent drift	New tools, parameter ranges, subdelegation attempts, value escalation.
Local drift	Verifier-specific relationship break, local denylist, unusual direct context.

21.3 Baseline Model

For feature vector $x_i(t)$, compute a robust baseline from a long window:

$$\mu_i^{base} = median(x_i(t - W_L), \ldots, x_i(t - W_M)), \tag{71}$$

$$\Sigma_i^{base} = robustCov(x_i), \tag{72}$$

$$D_i^{Maha} = \sqrt{(x_i^{cur} - \mu_i^{base})^\top (\Sigma_i^{base} + \lambda I)^{-1} (x_i^{cur} - \mu_i^{base})}. \tag{73}$$

For sparse identities, use cohort baselines and widen uncertainty.

21.4 Drift Score

$$DriftRisk_i = \text{clip}_{0,1}\left(\frac{D_i^{Maha}}{D_{crit}}\right) \cdot q_i + DormantPenalty_i + KeyPenalty_i. \tag{74}$$

Reference thresholds:

Risk	Threshold	Action
Low	below 2500 basis points	No special action.
Moderate	2500–5000	Show explanation; reduce confidence.
High	5000–7500	Step-up verification for high-value actions.
Severe	above 7500	Block or quarantine in high-risk domains until recovery or review.

21.5 Compromise vs. Legitimate Change

Signal	More Like Compromise	More Like Legitimate Change
Key change	No recovery proof, followed by burst	Signed rotation with recovery policy.
Dormant reactivation	Immediate outbound requests or payments	Step-up proof and gradual return.
Graph shift	Old counterparties vanish, new targets appear	New org membership or disclosed role change.
Agent tool change	New high-risk tool or value escalation	New delegation signed by principal.
Attestations	Negative claims unresolved	Positive verified migration attestation.

21.6 Drift Report Object

```
{
  "type": "tsl.drift_report.v1",
  "subject": "did:tsl:eip155-8453:0xabc...",
  "computed_at": "2026-05-26T00:00:00Z",
  "baseline_window_days": 180,
  "observation_window_days": 1,
  "drift_score_bps": 6120,
  "drift_label": "severe",
  "components": {
    "key_drift_bps": 7200,
    "graph_drift_bps": 5800,
    "action_drift_bps": 4300,
    "cadence_drift_bps": 7600,
    "claim_drift_bps": 0,
    "agent_drift_bps": 0
  },
```

```
  "reason_codes": ["dormant_reactivation", "new_key_without_prior_local_history", "
    outbound_burst"],
  "action": "step_up",
  "signature": "provider_signature"
}
```

22 Attestation Trust Model

An attestation proves that an issuer made a claim. It does not prove the claim is true. A robust TSL implementation treats attestations as evidence with issuer quality, claim class, decay, conflict handling, and appeal mechanics.

22.1 Issuer Quality

Issuer quality is domain-specific:

$$Q_j^d = \sigma(\beta_0 + \beta_1 Audit_j + \beta_2 Accuracy_j + \beta_3 Age_j + \beta_4 StakeOrBond_j - \beta_5 Reversal_j - \beta_6 Dispute_j - \beta_7 Conflict_j). \tag{75}$$

Where no token or bond exists, $StakeOrBond$ is replaced by contractual accountability, public audit status, or enterprise trust tier. Token ownership alone MUST NOT increase issuer quality.

22.2 Attestation Classes and Controls

Class	Evidence Requirement	Abuse Control
Organization member	Membership record commitment	Expiration, issuer revocation, subject consent where needed.
Known business	Business registry or domain proof	Periodic renewal and issuer audit.
Verified maintainer	Repository or release-key proof	Code-release drift monitoring.
Trusted counterparty	Prior receipt or contract proof	Weight capped; no global trust inflation.
Private warning	Local evidence	Not public by default; expiration.
Provider risk flag	Evidence commitment and provider signature	Appeal URI, short expiry, audit sampling.
Public negative claim	Strong evidence commitment and high issuer quality	Appeal before high-impact propagation where feasible.
Compromise warning	Revocation or recovery evidence	Immediate visibility; short review loop.
Fraud label	Adjudicated evidence	Human review, reversal mechanics, defamation-aware policy.

22.3 Attestation Decay

Positive attestations decay unless the claim type has explicit validity:

$$Value(a,t) = Q_{issuer(a)} \cdot \kappa_{class(a)} \cdot 2^{-(t-t_a)/h_a} \cdot StateMultiplier(a,t). \tag{76}$$

Negative attestations decay faster unless renewed with evidence:

$$NegValue(a,t) = -Q_{issuer(a)} \cdot \kappa_{class(a)} \cdot 2^{-(t-t_a)/h_a} \cdot AppealMultiplier(a,t). \tag{77}$$

If an attestation is reversed, its contribution becomes zero and the issuer reversal rate increases.

22.4 Conflict Resolution

For claim class c:

$$PosMass_c = \sum_{a \in A_c^+} Value(a,t), \tag{78}$$

$$NegMass_c = \sum_{a \in A_c^-} |NegValue(a,t)|, \tag{79}$$

$$Conflict_c = \frac{\min(PosMass_c, NegMass_c)}{PosMass_c + NegMass_c + \varepsilon}. \tag{80}$$

High conflict does not automatically mean high risk. It means the verifier must show the conflict and possibly require review.

22.5 Attestation Object V2

```
{
  "type": "tsl.attestation.v2",
  "attestation_id": "0x...",
  "issuer": "did:tsl:org:issuer",
  "subject": "did:tsl:eip155-8453:0xabc...",
  "claim_class": "org_member",
  "claim_polarity": "positive",
  "severity": "none",
  "claim_commitment": "0x...",
  "evidence_commitment": "0x...",
  "evidence_policy": "selective_disclosure_or_auditor_review",
  "visibility": "selective",
  "appeal_uri": "https://issuer.example/appeal/0x...",
  "issued_at": "2026-05-26T00:00:00Z",
  "valid_after": "2026-05-26T00:00:00Z",
  "expires_at": "2026-11-26T00:00:00Z",
  "revocation_pointer": "0x...",
  "signature": "issuer_signature"
}
```

22.6 Appeal and Reversal Mechanics

```
{
  "type": "tsl.attestation_appeal.v1",
  "appeal_id": "0x...",
  "attestation_id": "0x...",
  "subject": "did:tsl:eip155-8453:0xabc...",
  "appeal_reason_class": "incorrect_claim",
  "counter_evidence_commitment": "0x...",
  "requested_action": "retract_or_downgrade",
  "submitted_at": "2026-05-26T00:00:00Z",
  "status": "submitted",
  "signature": "subject_signature"
}
```

A public negative claim MUST expose an appeal state: `submitted`, `under_review`, `upheld`, `reversed`, `expired`, or `escalated`. In ordinary prose, these machine labels should be rendered without implying guilt before adjudication.

23 Privacy-Preserving Analytics and Zero-Knowledge Roadmap

The privacy roadmap MUST be staged. TSL adoption should not depend on advanced private graph proofs in the MVP. The robust path is selective disclosure first, threshold proofs second, private graph analytics last.

23.1 Privacy Implementation Variants

Stage	Primitive	Use
P0	Salted commitments	Commit content, metadata, counterparties without disclosure.
P1	Merkle inclusion proofs	Prove event or receipt exists in a committed set.
P2	Selective disclosure	Open one claim without opening the whole graph.
P3	Threshold ZK	Prove age, count, rate, or value bounds.
P4	Set membership and non-membership	Prove membership in org set or absence from revocation/abuse set.
P5	Delegation-scope proof	Prove parameters satisfy policy without revealing all parameters.
P6	Private graph analytics	Approximate distance or escape metrics using commitments or secure aggregation.

23.2 Circuit Registry

```
{
  "type": "tsl.zk_circuit_registry_entry.v1",
  "circuit_id": "tsl.receipt_count_threshold.v1",
  "statement": "subject has at least N valid reciprocal receipts before time T",
  "proving_system": "plonkish_or_groth16_declared_by_backend",
```

```
  "public_inputs": ["subject_commitment", "threshold", "epoch_root", "circuit_version"],
  "private_witness": ["receipt_leaves", "counterparty_salts", "merkle_paths"],
  "output": "boolean plus proof bytes",
  "soundness_bits_min": 100,
  "privacy_notes": "Does not reveal counterparties or exact count beyond threshold.",
  "auditor_commitment": "0x..."
}
```

23.3 Required Circuit Interfaces

Circuit	Public Claim	Private Witness
Age threshold	Identity age exceeds T	Creation proof, salt, registry path.
Receipt count threshold	Reciprocal receipts exceed N	Receipt leaves, salts, counterparties, paths.
Dispute-rate bound	Dispute rate below r	Completed and disputed receipt leaves.
Set membership	Subject is in approved set	Membership leaf, salt, Merkle path.
Non-membership	Subject absent from revocation or abuse set	Sparse Merkle non-membership path.
Organization membership	Subject has active org claim	Attestation witness and issuer path.
Delegation scope	Action parameters are within scope	Parameter values, policy witness, delegation path.
Private graph distance	Distance or escape exceeds threshold	Committed local neighborhood, aggregate proof, seed commitments.

23.4 Example Threshold Proof Request

```
{
  "type": "tsl.zk_proof_request.v1",
  "circuit_id": "tsl.receipt_count_threshold.v1",
  "subject": "did:tsl:eip155-8453:0xabc...",
  "public_statement": {
    "minimum_reciprocal_receipts": 50,
    "valid_before": "2026-05-26T00:00:00Z",
    "receipt_root": "0x..."
  },
  "disclosure_policy": "do_not_reveal_counterparties",
  "verifier_domain": "marketplace.example"
}
```

23.5 Metadata Privacy Leakage Analysis

A privacy report MUST consider:

- timing leakage from checkpoint epochs,
- volume leakage from repeated commitments,

- uniqueness leakage from unusual bucket combinations,
- counterparty correlation through repeated pairwise proofs,
- public commitment reuse across domains,
- provider-side linkage through scoring requests,
- appeal and negative-claim disclosure risks.

Reference leakage score:

$$PLS = 1 - \prod_{k}(1 - l_k), \tag{81}$$

where l_k are normalized leakage components. High-assurance profiles MUST publish mitigation for any $PLS > 0.35$.

24 Model Governance and Provider Accountability

Scoring providers are not protocol truth authorities. They are accountable issuers of probabilistic assessments.

24.1 Provider Lifecycle

$$registered \rightarrow shadow \rightarrow active \rightarrow probation \rightarrow suspended \rightarrow revoked. \tag{82}$$

Models follow:

$$draft \rightarrow offline\_evaluated \rightarrow shadow \rightarrow canary \rightarrow active \rightarrow deprecated \rightarrow retired. \tag{83}$$

A provider on probation MAY issue assessments only if verifier policy accepts probationary providers.

24.2 Provider Registration Object

```
{
  "type": "tsl.provider_registration.v1",
  "provider_id": "did:tsl:provider:continuity-labs",
  "legal_or_pseudonymous_name_commitment": "0x...",
  "public_keys": [{"key_id": "#assessment-key-1", "type": "ed25519", "public_key": "0x..."}],
  "supported_domains": ["anti_phishing", "marketplace", "agent_delegation"],
  "provider_status": "active",
  "audit_policy_uri": "https://provider.example/audit-policy",
  "appeal_policy_uri": "https://provider.example/appeals",
  "privacy_policy_commitment": "0x...",
  "emergency_contact_commitment": "0x...",
  "registered_at": "2026-05-26T00:00:00Z",
  "signature": "provider_controller_signature"
}
```

24.3 Model Card Schema

```
{
  "type": "tsl.model_card.v2",
  "model_id": "anti-phishing-v2.1.0",
  "provider": "did:tsl:provider:continuity-labs",
  "domain": "anti_phishing",
  "intended_use": "consumer and enterprise phishing warning",
  "not_intended_for": ["employment_decisions", "credit_decisions", "law_enforcement"],
  "feature_registry_commitment": "0x...",
  "training_data_statement": {
    "time_window": "2025-01-01/2026-04-30",
    "label_sources": ["partner_reports", "appeal_adjudication", "red_team"],
    "private_data_used": false,
    "protected_attribute_use": "not_used"
  },
  "metrics": {
    "auroc_bps": 9430,
    "auprc_bps": 7110,
    "ece_bps": 310,
    "sparse_identity_false_positive_rate_bps": 180,
    "p95_latency_ms": 92,
    "appeal_reversal_rate_bps": 120
  },
  "known_limitations": ["low coverage for very new identities", "not proof of intent"],
  "red_team_report_commitment": "0x...",
  "privacy_report_commitment": "0x...",
  "signed_changelog_commitment": "0x...",
  "signature": "provider_signature"
}
```

24.4 Challenge and Appeal Process

```
{
  "type": "tsl.model_challenge.v1",
  "challenge_id": "0x...",
  "challenger": "did:tsl:eip155-8453:0xabc...",
  "provider": "did:tsl:provider:continuity-labs",
  "assessment_id": "0x...",
  "challenge_class": "incorrect_label",
  "evidence_commitment": "0x...",
  "requested_remedy": "reissue_assessment_or_explain",
  "submitted_at": "2026-05-26T00:00:00Z",
  "status": "submitted",
  "signature": "challenger_signature"
}
```

24.5 Provider Accountability Metrics

$$ProviderQuality = \sigma(\theta_0 + \theta_1 Calibration + \theta_2 RedTeam + \theta_3 AuditPass \tag{84}$$
$$- \theta_4 Reversal - \theta_5 UnresolvedAppeal - \theta_6 PrivacyIncident), \tag{85}$$
$$ReversalRate = \frac{reversed\_assessments}{challenged\_assessments + \varepsilon}, \tag{86}$$
$$TimelyAppealRate = \frac{appeals\_closed\_within\_SLA}{appeals\_submitted + \varepsilon}. \tag{87}$$

Sanctions MAY include downgrade to probation, model retirement, assessment-key revocation, public audit finding, bond slashing if a bond module exists, or removal from default verifier policy.

24.6 Release Controls

A model update MUST NOT become active unless:

$$EvalPass \wedge CalibrationPass \wedge PrivacyPass \wedge RedTeamPass \wedge RollbackPlanExists. \tag{88}$$

Production models SHOULD be deployed through shadow and canary phases. Canary exposure SHOULD be below 10% before promotion unless emergency mitigation is declared.

25 Evaluation Framework

The evaluation framework must test accuracy, calibration, fairness to sparse identities, latency, privacy leakage, robustness under attacks, and longitudinal drift.

25.1 Benchmark Dataset Families

Dataset	Source	Purpose
Historical benign continuity	Verified logs and partner opt-in data	Estimate ordinary trajectory distribution.
Confirmed abuse	Evidence-bound phishing, fraud, scam, compromised identity cases	Positive adversarial labels.
Appeal adjudication	Reversed and upheld assessments	Measure label quality and provider abuse.
Sparse legitimate identities	New users, privacy-preserving users, low-volume users	Prevent new-user punishment.
Synthetic Sybil farms	Simulated collusion graphs	Stress-test cluster metrics.
Compromise simulations	Key theft and dormant reactivation scripts	Test drift detection.

Agent misuse traces	Tool calls, delegation violations, value escalation	Test agent authorization.
Negative controls	Similar-looking but benign communities	Measure false positives.

25.2 Ground Truth Labeling

Labels MUST be time-indexed:

$$y_i(t) \in \{benign, abusive, compromised, sybil, unknown, disputed\}. \tag{89}$$

A label is valid only for a time window and evidence class. Human adjudication SHOULD use at least two independent reviewers for high-impact negative labels, with a tie-breaker for disagreement.

25.3 Metrics

$$AUROC = \Pr(S^+ > S^-), \tag{90}$$

$$Precision = \frac{TP}{TP + FP}, \tag{91}$$

$$Recall = \frac{TP}{TP + FN}, \tag{92}$$

$$FPR_{sparse} = \frac{FP_{sparse}}{FP_{sparse} + TN_{sparse}}, \tag{93}$$

$$ECE = \sum_b \frac{|B_b|}{n} |acc(B_b) - conf(B_b)|, \tag{94}$$

$$Coverage = \frac{assessments\_not\_abstained}{total\_eligible}, \tag{95}$$

$$Leakage = PLS, \tag{96}$$

$$Latency_{p95} = Q_{0.95}(verification\_time). \tag{97}$$

25.4 Promotion Gates by Domain

Promotion gates are expressed in fixed-point basis points for machine-readable reports. For example, `auroc_bps = 9200` means AUROC 0.92.

Domain	Min AUROC bps	Max ECE bps	Additional Gate
Anti-phishing	9200	400	Recall at high-risk threshold at least 8500 bps.
Marketplace	8800	500	Sparse seller false-positive rate below 300 bps.
Agent payments	9400	300	Zero known out-of-scope payment approvals in red-team set.
Open-source	8600	500	Maintainer-drift false-positive review below 200 bps.
Professional identity	8400	500	Public negative label reversal rate below 100 bps.
Customer support	9000	400	Org-delegation verification false negative below 100 bps.

25.5 Red-Team Matrix

A production scorer MUST be evaluated against:

- dense Sybil receipt farms,
- slow-aged farms,
- purchased aged identities,
- compromised old identities,
- false positive attacks on sparse legitimate users,
- malicious negative attestations,
- issuer collusion,
- model poisoning,
- relay equivocation attempts,
- privacy correlation attempts,
- agent delegation bypass attempts.

25.6 Evaluation Report Object

```
{
  "type": "tsl.evaluation_report.v1",
  "model_id": "anti-phishing-v2.1.0",
  "provider": "did:tsl:provider:continuity-labs",
  "dataset_commitments": ["0x...", "0x..."],
  "time_split": {"train_end": "2026-03-31", "test_start": "2026-04-01"},
  "metrics": {
    "auroc_bps": 9430,
    "auprc_bps": 7110,
    "ece_bps": 310,
    "sparse_identity_fpr_bps": 180,
    "red_team_detection_bps": 8820,
    "p95_latency_ms": 92,
    "privacy_leakage_bps": 2100
  },
```

```
  "promotion_gate_passed": true,
  "auditor_signature": "optional_auditor_signature",
  "provider_signature": "provider_signature"
}
```

26 Economic and Game-Theoretic Model

The economic objective is to make trustworthy evidence cheaper for honest users than for attackers, while making abusive strategies unprofitable at scale.

26.1 Implementation Variants

Variant	Incentive Mechanism	Notes
Token-free MVP	Subscriptions, API fees, enterprise contracts	Core validity independent of economics.
Bonded provider	Fiat, stablecoin, or token bond	Provider can lose bond after audited abuse.
Relay fee market	Per-checkpoint or per-proof fees	Does not affect proof validity.
Auditor reward	Bug bounty or fraud proof payout	Rewards detection of equivocation and provider abuse.
Full economic module	Staking, slashing, rewards	Optional adapter; never defines trustworthiness.

26.2 Honest Receipt Incentives

Receipt value should reward quality, not raw count:

$$ReceiptValue_{ij} = \omega_c \cdot r_{ij} \cdot D_i \cdot D_j \cdot Q_j \cdot d_c(t - \tau) \cdot AppealState. \tag{98}$$

This makes closed-cluster receipt farming asymptotically low value:

$$\lim_{InternalRatio(C) \to 1} MarginalTrustGain(e_{internal}) = 0. \tag{99}$$

26.3 Spam Cost Model

$$Profit_{spam} = N \cdot p_{success} \cdot V_{victim} - C_{message} - C_{identity} - C_{receipt} - C_{attestation} - p_{detect} Penalty. \tag{100}$$

The protocol should reduce $p_{success}$, increase $C_{identity}$, increase $C_{receipt}$, and increase p_{detect} without imposing excessive costs on legitimate new users.

26.4 Sybil Farming Economics

A farm strategy $\mathcal{S}$ succeeds if it reaches a target domain threshold:

$$Success(\mathcal{S}) = \mathbb{I}[S_i^d \geq \theta_d \wedge q_i \geq q_d]. \tag{101}$$

Expected attack ROI:

$$ROI(\mathcal{S}) = \frac{P(Success) \cdot Benefit - Cost(\mathcal{S}) - P(Detected) \cdot Penalty}{Cost(\mathcal{S}) + \varepsilon}. \tag{102}$$

A model promotion gate SHOULD require $ROI(\mathcal{S}) < 0$ for reference attack profiles in high-risk domains.

26.5 Reputation Laundering

For an aged identity sold at price P_{sale}:

$$Profit_{launder} = P_{sale} + AttackBenefit - C_{aging} - C_{receipts} - C_{attestations} - P_{drift}Penalty. \tag{103}$$

Defenses:

- ownership-sensitive key-change windows,
- value-based step-up checks,
- post-sale graph drift detection,
- attestation revalidation after control change,
- local relationship preservation so a new controller cannot inherit private trust automatically.

26.6 Negative Claim Abuse

$$Cost_{bad\_negative} = P_{appeal\_success} \cdot Penalty + ReputationLoss + AuditCost + LegalRisk. \tag{104}$$

The system is robust only if:

$$Cost_{bad\_negative} > Benefit_{sabotage}. \tag{105}$$

Therefore public negative claims require high issuer quality, evidence commitment, expiration, appeal, and reversal tracking.

27 Formal Security Model

27.1 Assumptions

The protocol relies on:

- existential unforgeability of declared signature schemes,

- collision and preimage resistance of declared hashes,
- soundness of Merkle inclusion and consistency proofs,
- finality assumptions of settlement backends,
- secure key custody or timely revocation after compromise,
- at least one honest auditor or verifier observing public equivocation,
- honest execution of local client privacy controls,
- transparent provider governance for scoring outputs.

27.2 Adversary Capabilities

The adversary may create unlimited identities, generate arbitrary content, collude among identities, compromise keys, delay relay submissions, attempt log equivocation, corrupt weak issuers, poison model training data, submit malicious negative claims, exploit public timing leakage, and attempt agent delegation bypass.

The adversary cannot forge signatures for uncompromised keys, find hash collisions, or rewrite finalized checkpoints without violating stated assumptions.

27.3 Security Games

Game	Adversary Goal	Win Condition
Origin forgery	Create valid event for uncompromised key	Verifier accepts signature under active key not controlled by adversary.
Commitment binding	Open one commitment two ways	Same commitment verifies for two different contents or metadata values.
Inclusion forgery	Claim log inclusion falsely	Invalid Merkle path accepted for a settled root.
Checkpoint equivocation	Present conflicting roots	Two roots for same epoch-shard avoid auditor or verifier detection.
Revocation safety	Use revoked key after effective time	Verifier accepts post-revocation event under policy requiring revocation check.
Unlinkability	Link two commitments beyond policy	Adversary distinguishes same subject across scopes better than leakage budget permits.
Delegation scope	Prove out-of-scope action as allowed	Verifier accepts action outside effective delegation scope.
Assessment integrity	Forge provider score	Verifier accepts unsigned or wrong-provider assessment as valid.

27.4 Guarantees vs. Estimates

Mechanism	What It Guarantees
Signature	A holder of the private key signed the canonical payload, assuming no compromise.
Registry	Key state and provider state according to settled registry facts.
Hash commitment	Opened value matches committed value if salt and value are disclosed.
Merkle proof	Commitment is included in a stated root if proof verifies.
Checkpoint	Root was recorded under settlement assumptions.
Auditor gossip	Equivocation becomes detectable if conflicting views are observed.
Attestation	Issuer made a signed claim; truth depends on evidence and issuer quality.
Trust score	Calibrated estimate of risk for a domain; never proof of safety or intent.
Graph model	Statistical similarity and continuity evidence; not cryptographic truth.

27.5 Formal Invariants

1. Invalid signatures MUST fail regardless of score.
2. Revoked keys MUST fail for post-revocation events.
3. A verifier MUST be able to recompute canonical bytes before signature verification.
4. Public commitments MUST NOT require raw content disclosure.
5. Scoring provider signatures MUST be checked independently of relay signatures.
6. Agent action authorization MUST check effective scope and revocation.
7. Sparse evidence MUST NOT be mapped directly to maliciousness without adverse evidence.

28 Agent Delegation Semantics

AI agents need verifiable authority, not just identity. A valid agent signature proves that an agent key signed an action. It does not prove that the principal authorized the action.

28.1 Delegation Implementation Variants

Variant	Use	Rule
Simple delegation	Low-risk read-only tools	Signed allow policy, short expiry.
Value-constrained delegation	Purchases and payments	Amount caps, counterparties, approval thresholds.
Workflow delegation	Enterprise workflows	Project, role, and tool constraints.
Subdelegation chain	Multi-agent systems	Parent must explicitly allow subdelegation; effective scope is intersection.
ZK-constrained delegation	Private parameters	Prove committed parameters satisfy limits.

28.2 Permission Language Requirements

The permission language MUST be:

- decidable,
- canonicalizable,
- monotone under scope intersection,
- parameter-aware,
- revocable,
- explainable to human and machine verifiers.

28.3 Scope Grammar

```
Policy        := { Effect, Principal, Delegate, Resources, Actions, Constraints,
    Subdelegation }
Effect        := "allow" | "deny"
Resource      := Namespace ":" Path
Action        := Namespace "." Verb
Constraint    := TimeConstraint | ValueConstraint | CounterpartyConstraint | ToolConstraint |
     RateConstraint | ApprovalConstraint
Subdelegation := { allowed: boolean, max_depth: integer, allowed_actions: Action[] }
Decision      := allow only if at least one allow matches and no deny matches
```

28.4 Delegation Policy Object

```
{
  "type": "tsl.delegation_policy.v2",
  "policy_id": "0x...",
  "principal": "did:tsl:org:acme",
  "delegate": "did:tsl:agent:invoice-agent-7",
  "effect": "allow",
  "valid_from": "2026-05-26T00:00:00Z",
  "valid_until": "2026-06-26T00:00:00Z",
  "resources": ["invoice:approved_vendor/*", "purchase_order:project_alpha/*"],
  "actions": ["invoice.read", "invoice.request_approval", "purchase_order.draft"],
  "constraints": {
    "max_value_minor_units": 500000,
    "currency": "USD",
    "requires_human_approval_above_minor_units": 100000,
    "allowed_tools": ["invoice_api", "vendor_registry"],
    "denied_tools": ["wire_transfer_execute"],
    "allowed_counterparty_set_commitment": "0x...",
    "rate_limit": {"max_actions": 100, "window_seconds": 86400}
  },
  "subdelegation": {
    "allowed": false,
    "max_depth": 0
  },
  "parent_policy_id": null,
  "revocation_pointer": "0x...",
  "nonce": "0x...",
```

```
  "signature": "principal_signature"
}
```

28.5 Agent Action Object

```
{
  "type": "tsl.agent_action.v2",
  "action_id": "0x...",
  "agent": "did:tsl:agent:invoice-agent-7",
  "principal": "did:tsl:org:acme",
  "action": "invoice.request_approval",
  "resource": "invoice:approved_vendor/12345",
  "tool": "invoice_api",
  "parameters_commitment": "0x...",
  "parameter_disclosure_policy": "selective_or_zk",
  "delegation_chain_root": "0x...",
  "issued_at": "2026-05-26T00:00:00Z",
  "nonce": "0x...",
  "signature": "agent_signature"
}
```

28.6 Effective Scope

For chain $D_0, D_1, \ldots, D_n$:

$$Scope_{eff} = \bigcap_{k=0}^{n} Scope(D_k). \tag{106}$$

An action is allowed only if:

$$Allowed(a, ctx, D_{0:n}) = \bigwedge_{k=0}^{n} Signed(D_k) \wedge Active(D_k, t) \wedge \neg Revoked(D_k, t) \tag{107}$$

$$\wedge\, SubdelegationValid(D_{0:n}) \wedge a \in Scope_{eff} \wedge Constraints(ctx, Scope_{eff}). \tag{108}$$

Deny rules override allow rules.

28.7 Authorization Algorithm

```
function verifyAgentAction(action, delegationChain, parameters, policy): AgentDecision {
  verifyAgentSignature(action)
  verifyCanonicalActionCommitment(action, parameters)

  effectiveScope = universeScope()
  for delegation in delegationChain:
    verifyPrincipalSignature(delegation)
    assert activeAt(delegation, action.issued_at)
```

```
    assert !revokedAt(delegation, action.issued_at)
    assert subdelegationAllowedByParent(delegation)
    effectiveScope = intersect(effectiveScope, delegation.scope)

  if denyMatches(action, effectiveScope): return deny("explicit_deny")
  if !allowMatches(action, effectiveScope): return deny("no_matching_allow")
  if !constraintsSatisfied(action, parameters, effectiveScope): return deny("
    constraint_violation")
  if policy.requireSettlement: verifyInclusionAndCheckpoint(action)

  return allow("inside_scope")
}
```

28.8 Proof That an Action Was Inside Scope

A verifier checks:

1. agent signature over canonical action,
2. principal signatures over delegation policies,
3. delegation chain inclusion or disclosure,
4. active time window for every delegation,
5. revocation state for every delegation,
6. subdelegation permission and depth,
7. action, resource, tool, value, counterparty, and rate constraints,
8. event inclusion and settlement if policy requires it.

For private values, use a circuit with public claim:

$$CommittedAmount \leq MaxAmount \wedge Counterparty \in ApprovedSet \wedge Tool \in AllowedTools. \quad (109)$$

28.9 Agent-Specific Failure Labels

Failure	Meaning
`agent_signature_invalid`	Agent did not sign the action or payload changed.
`delegation_missing`	No valid principal authorization was provided.
`delegation_expired`	Action occurred outside allowed time.
`delegation_revoked`	Principal revoked authority before action.
`scope_violation`	Action or resource is outside allowed scope.
`constraint_violation`	Amount, counterparty, tool, rate, or approval rule failed.
`subdelegation_not_allowed`	Agent attempted unauthorized subdelegation.
`settlement_missing`	Policy required checkpoint settlement but proof was missing.

29 Threat Model

29.1 Sybil Farms

Attackers create many identities that interact with each other.

Defenses:

- diverse counterparty weighting,
- community-escape metrics,
- trust-weighted attestations,
- rate limits for new identities,
- cluster anomaly detection,
- economic or proof-of-personhood optional add-ons,
- decay functions for low-quality internal edges.

29.2 Reputation Laundering

Attackers slowly build reputation, then sell or weaponize accounts.

Defenses:

- behavioral drift detection,
- key/device continuity checks,
- sudden target-community change alerts,
- high-value action re-verification,
- receipt-quality weighting,
- post-compromise recovery and warning flows.

29.3 Compromised Identity

An attacker steals a valid signing key.

Defenses:

- smart-account recovery,
- device-scoped keys,
- revocation registry,
- anomaly-triggered step-up verification,
- short-lived agent/session keys,
- delegated signing policies.

29.4 Metadata Correlation

Observers infer private relationships from public commitments.

Defenses:

- blinded receiver commitments,
- salting and domain separation,
- batching and delayed publication,
- optional local-only mode,
- aggregate disclosure,
- private set intersection for contact verification,
- zero-knowledge proofs for thresholds.

29.5 Malicious Negative Attestations

Bad actors use warnings or reports to harass competitors.

Defenses:

- issuer reputation,
- evidence commitments,
- appeal process,
- rate limits,
- separation of private warnings from public labels,
- defamation-aware governance,
- human review for high-impact actions.

29.6 Model Poisoning

Attackers manipulate training data or graph features.

Defenses:

- robust training sets,
- adversarial evaluation,
- provider audit logs,
- model cards,
- signed model versions,
- shadow models,
- user-selectable providers,
- local policy overrides.

30 Use Cases

30.1 Anti-Phishing

A message says it is from a bank. The TSL client verifies whether the sender's TrustID matches the user's prior verified bank relationship, whether the key is active, whether the proof is included in a checkpoint, and whether current behavior resembles prior continuity.

Example Warning

This message has a valid signature, but not from the TrustID you previously used with this bank. The identity is three days old, has no reciprocal receipts with you, and has elevated outbound activity. Treat as high risk.

30.2 AI-Agent Trust

AI agents will call tools, negotiate with other agents, spend money, sign documents, and interact with humans. Each agent needs scoped identity, delegated authority, and continuity history.

TSL can answer:

- Is this the same agent I interacted with before?
- Which human, company, or wallet delegated authority to it?
- What actions is it allowed to perform?
- Has its behavior changed sharply?
- Is this agent connected to malicious clusters?

30.3 Marketplace Reputation

Sellers should not lose all reputation when they move platforms. Buyers should not have to trust platform-specific badges. TSL makes completed transactions and counterparty receipts portable.

30.4 Professional Trust

Founders, recruiters, engineers, contractors, and creators can prove long-term professional continuity without exposing private messages.

30.5 Open-Source Supply Chain

Package maintainers and release bots can sign releases, issues, pull requests, and maintainer attestations. A suspicious package release can be evaluated against maintainer continuity, key state, and behavioral drift.

30.6 Customer Support

Businesses can sign support interactions. Customers can verify that a support agent is authorized and that a message belongs to the company's continuity chain.

31 Killer Demos

Demo 1: The Phishing Email That Fails

A realistic fake email from a bank arrives. Gmail does not block it. TSL shows: wrong TrustID, no prior relationship, young key, no company attestation, scam-like outbound pattern. The user sees one clear warning before clicking.

Demo 2: Portable Seller Reputation

A marketplace seller leaves one platform. On a new marketplace, they selectively prove 500 completed transactions, 2 years of continuity, and low dispute rate without revealing buyer identities.

Demo 3: Agent-to-Agent Verification

An AI procurement agent receives an invoice from another agent. It verifies delegation scope, company TrustID, invoice continuity, and revocation status before initiating payment approval.

Demo 4: Compromised Maintainer Alert

A trusted open-source maintainer signs a release, but the behavior is abnormal: new device key, dormant account reactivation, unusual package dependency, and sudden outbound maintainer messages. TSL shows a step-up verification warning.

32 Product Surface

The product surface is only the visible edge of the protocol. These badges, web views, extensions, wallets, and dashboards are reference interfaces for a deeper trust-envelope layer. They should never be described as the canonical TSL application, because the canonical object is the verifiable proof itself.

32.1 Consumer Badge

```
TSL Verified
Continuity: Strong
Relationship: Known to you
Risk: Low
Why:
- Signature valid under current key registry
- Identity has 31 months of continuity
- 142 reciprocal receipts
- No active revocation
- Behavior consistent with previous interactions
```

32.2 Caution Badge

```
Unknown TSL Identity
Continuity: Insufficient
```

```
Risk: Caution
Why:
- Signature is valid
- Identity is 2 days old
- No prior relationship with you
- No trusted attestations disclosed
```

32.3 High-Risk Badge

```
High Risk
Continuity: Broken
Why:
- Key was revoked 6 hours ago
- Message signed with revoked key
- Sender claims to be a known business but uses a new TrustID
- Similar behavior observed in flagged phishing cluster
```

33 MVP Roadmap

33.1 Phase 0: Protocol Spec

Deliverables:

- canonical object schemas,
- canonical serialization rules,
- signing and verification library,
- event commitment format,
- receipt format,
- revocation semantics,
- transparency-log spec,
- verification API.

33.2 Phase 1: Universal Proof Link Reference Client

Build a reference client where a user or agent can:

- create a TrustID,
- sign a message or claim,
- generate a proof link,
- share the proof anywhere,
- allow a recipient to verify and co-sign a receipt.

This phase needs no platform integration and no destination application. Existing communication channels simply carry proof links.

33.3 Phase 2: Browser Extension

The extension detects TSL proof links and embedded envelopes on common web surfaces. It does not need deep API integration. It only needs to parse, verify, and render.

33.4 Phase 3: Transparency Log and Anchoring

Build:

- append-only Merkle log,
- inclusion proofs,
- consistency proofs,
- checkpoint contract,
- revocation registry,
- identity registry.

33.5 Phase 4: Trust Assessment Engine

Start with transparent scoring and explanations. Add graph intelligence only after enough data exists.

33.6 Phase 5: Agent SDK

Ship SDKs for AI agents, wallets, and enterprise workflows.

```
npm install @tsl/proof
pip install tsl-proof
cargo add tsl-proof
```

34 Reference APIs

34.1 Create TrustID

```
POST /v1/identity/create

Request:
{
  "public_key": "z6Mk...",
  "controller_type": "local | smart_account | enterprise | agent",
  "recovery_policy_commitment": "0x..."
}

Response:
{
  "trust_id": "did:tsl:eip155-8453:0xabc...",
  "created_at": "2026-05-25T00:00:00Z",
  "registry_tx": "0x..."
}
```

34.2 Commit Event

```
POST /v1/commitments

Request:
{
  "event_commitment": "0x...",
  "sender": "did:tsl:eip155-8453:0xabc...",
  "signature": "0x...",
  "timestamp": "2026-05-25T00:01:00Z"
}

Response:
{
  "accepted": true,
  "relay": "relay-3.tsl.network",
  "log_index": 184293,
  "epoch": "2026-05-25T00:00:00Z/PT5M",
  "shard": "00af",
  "inclusion_promise": "0x..."
}
```

34.3 Get Proof

```
GET /v1/proofs/0xEVENTCOMMITMENT

Response:
{
  "event_commitment": "0x...",
  "checkpoint": {
    "epoch": "2026-05-25T00:00:00Z/PT5M",
    "shard": "00af",
    "root": "0x...",
    "settlement_tx": "0x..."
  },
  "merkle_proof": ["0x...", "0x...", "0x..."],
  "consistency_proof": ["0x...", "0x..."]
}
```

34.4 Verify

```
POST /v1/verify

Request:
{
  "message": "optional raw message if user chooses to disclose",
  "envelope": { "...": "..." },
  "proof": { "...": "..." },
  "verifier_context": {
    "local_relationship": "optional_private_context"
  }
}
```

```
Response:
{
  "signature_valid": true,
  "key_active": true,
  "included_in_log": true,
  "checkpoint_settled": true,
  "revoked": false,
  "risk_label": "likely_trusted",
  "score": 82,
  "explanation": [
    "Signature valid",
    "Key active",
    "Event included in checkpoint",
    "Strong reciprocal receipt history"
  ]
}
```

35 Smart Contracts

35.1 Registry Interfaces

```
interface ITrustIDRegistry {
    function register(bytes32 trustId, bytes32 controller, bytes32 policyCommitment) external
    ;
    function rotateKey(bytes32 trustId, bytes32 oldKey, bytes32 newKey, bytes calldata proof)
     external;
    function revokeKey(bytes32 trustId, bytes32 key, uint8 reason, bytes calldata proof)
    external;
    function getActiveKey(bytes32 trustId) external view returns (bytes32);
    function isRevoked(bytes32 trustId, bytes32 key) external view returns (bool);
}

interface ICheckpointRegistry {
    function submitCheckpoint(
        uint64 epoch,
        bytes32 shard,
        bytes32 eventRoot,
        bytes32 receiptRoot,
        bytes32 attestationRoot,
        bytes32 revocationRoot,
        uint256 eventCount,
        bytes calldata relaySignature
    ) external;

    function getCheckpoint(uint64 epoch, bytes32 shard) external view returns (Checkpoint);
}

interface IProviderRegistry {
    function registerProvider(bytes32 providerId, bytes32 publicKey, bytes32 policyCommitment
    ) external;
    function registerModel(bytes32 providerId, bytes32 modelId, bytes32 modelCardCommitment)
    external;
```

```
    function revokeProvider(bytes32 providerId, uint8 reason) external;
}
```

35.2 Contract Principle

Contracts should remain minimal. They settle state roots and registry facts. Complex scoring, graph computation, and privacy-sensitive operations happen off-chain.

36 Governance

TSL must avoid becoming one centralized blacklist or social-credit oracle.

Layer	Governance Principle
Protocol schema	Open standard and versioned compatibility
Reference clients	Open-source implementation
Identity registry	Neutral infrastructure
Scoring providers	Plural, inspectable, and user-selectable
Negative claims	Evidence-bound, appealable, and issuer-accountable
Model updates	Signed model cards and changelogs
User UX	Probabilistic, explanatory labels rather than absolute accusations
Enterprise policies	Configurable without fragmenting protocol verification

Ethical Constraint

The protocol should prove continuity, not enforce conformity. It should help users evaluate risk, not create an irreversible global reputation caste system.

37 Business Model

The base protocol should be open. The company can monetize products and infrastructure around it.

Revenue Line	Description
Enterprise verification API	Banks, marketplaces, SaaS, recruiting, support platforms
Agent trust SDK	AI-agent platforms, tool providers, autonomous commerce systems
Risk intelligence	Scam cluster feeds, fraud intelligence, trust analytics
Hosted relay infrastructure	Managed logs, checkpoints, monitoring, uptime guarantees
Consumer premium	Identity recovery, advanced privacy proofs, professional profile
Marketplace verification	Seller/buyer trust receipts and portable transaction history
Developer tooling	Maintainer signing, package trust, CI/CD release verification

38 Go-to-Market

The wedge should be a problem where trust failure is painful and proof links can work without permission from incumbents.

38.1 Best Initial Wedges

1. **AI-agent verification:** new category, little incumbent lock-in, urgent need for delegated trust.
2. **Marketplace seller/buyer trust:** obvious economic value and portable reputation story.
3. **Anti-phishing for businesses:** strong enterprise willingness to pay.
4. **Open-source maintainer continuity:** developer credibility and viral proof-of-concept.
5. **Professional proof links:** founders, recruiters, contractors, creators, and consultants.

38.2 Avoid as First Wedge

Do not begin by asking Gmail, LinkedIn, or major social platforms for permission. They can become later distribution channels. The first wedge should prove value without platform cooperation.

38.3 Adoption Loop

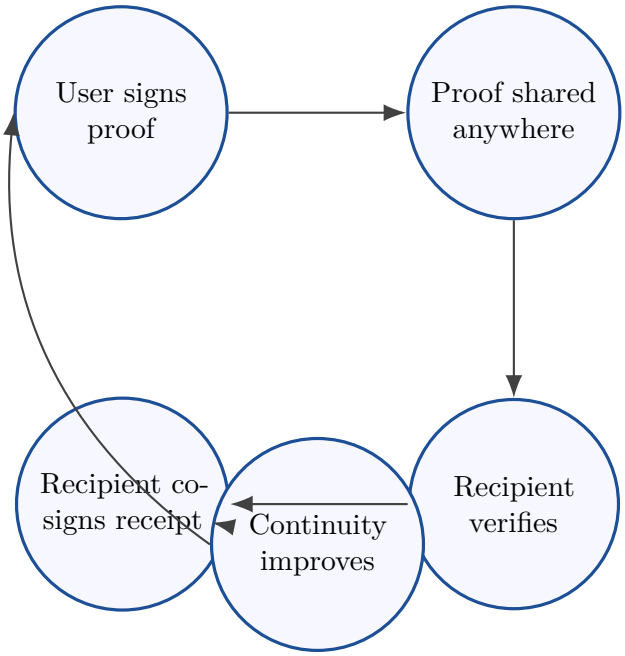

Figure 3: Every verification can create a receipt; every receipt increases continuity; stronger continuity makes future verification more valuable.

39 Competitive Positioning

Category	What It Solves	What It Misses
KYC providers	Legal identity verification	Privacy, pseudonymity, behavioral continuity, portability across contexts
Password/auth providers	Account access	Trustworthiness, graph behavior, counterparty receipts
Email security	Phishing detection	Cross-platform continuity and user-owned proofs
Platform reputation	Local marketplace trust	Portability and cryptographic ownership
DID wallets	Identifier control	Behavioral trust, receipts, graph intelligence
Blockchains	Durable settlement	Communication trust semantics and privacy UX
AI scam detectors	Pattern detection	Verifiable identity history and user-owned evidence
TSL	Portable continuity trust	Must solve adoption, privacy, governance, and UX carefully

40 Technical Differentiation

TSL combines eight capabilities that are usually separate:

1. cryptographic identity,
2. message/event signatures,
3. counterparty receipts,
4. append-only transparency logs,
5. blockchain-anchored checkpoints,
6. key revocation and recovery,
7. privacy-preserving graph intelligence,
8. signed, plural trust assessments.

The novelty is not "put messages on a blockchain." That is the wrong framing. The novelty is:

> **Build a portable behavioral trust substrate whose evidence is cryptographically auditable and privacy-preserving by default.**

41 Minimum Security Requirements

A production implementation requires:

- audited cryptographic libraries,

- canonical serialization to prevent signature ambiguity,
- domain-separated commitments,
- replay protection with nonces and timestamps,
- robust key rotation semantics,
- revocation propagation,
- transparency-log consistency proofs,
- rate limits and anti-spam controls,
- privacy threat modeling,
- user-controlled disclosure,
- signed model versions,
- appeal mechanisms,
- abuse review for high-impact labels.

42 Reference Implementation Plan

42.1 Repository Structure

```
tsl/
  specs/
    identity.schema.json
    event_commitment.schema.json
    receipt_commitment.schema.json
    attestation.schema.json
    revocation.schema.json
    checkpoint.schema.json
  packages/
    tsl-core-ts/
    tsl-core-rust/
    tsl-core-python/
  clients/
    web-verifier/
    browser-extension/
    cli/
    agent-sidecar/
  services/
    relay/
    transparency-log/
    checkpoint-submitter/
    provider-registry/
  contracts/
    TrustIDRegistry.sol
    RevocationRegistry.sol
    CheckpointRegistry.sol
    ProviderRegistry.sol
  models/
    scoring-reference/
    graph-features/
    anomaly-detection/
  docs/
    whitepaper.tex
```

```
protocol.md
threat-model.md
```

42.2 First 90 Days

Period	Deliverables
Days 1–15	Protocol schemas, signing library, canonical serialization, TrustID generation
Days 16–30	Proof-link reference client, message signing, basic verification, local encrypted store
Days 31–45	Receipt flow, append-only log prototype, inclusion proofs
Days 46–60	Revocation registry, checkpoint contract, first blockchain anchor
Days 61–75	Browser extension, trust badge UX, explanation engine
Days 76–90	Agent SDK, marketplace demo, phishing demo, open-source maintainer demo

43 Engineering Implementation Specification

This section converts the protocol architecture into an engineer-executable backend specification. The reference implementation must be treated as a protocol stack, not as a monolithic application. A web app, extension, dashboard, or enterprise console is only a client of the protocol.

Implementation Boundary

The canonical product is the protocol: object schemas, serialization rules, cryptographic verification, identity resolution, revocation semantics, Merkle proof verification, checkpoint settlement, relay behavior, and signed trust assessments. Applications are optional interfaces and carriers. No single website, dashboard, wallet, or extension is the protocol.

43.1 Normative Language

The following words are used with implementation meaning:

Term	Meaning
MUST	Required for protocol compatibility. Implementations that violate a MUST are non-compliant.
SHOULD	Strong recommendation. Implementations may differ only with a documented reason.
MAY	Optional extension that must not break compatibility.
LOCAL	Data or computation controlled by the user, device, organization, or private verifier.
PUBLIC	Data visible in logs, registries, chain state, or public APIs.

43.2 Decentralization Invariants

A compliant implementation MUST preserve these invariants:

1. A valid TSL proof MUST be verifiable without the original hosted website.
2. A valid TSL proof MUST NOT require the user to join a destination application or social network.
3. Raw message content MUST NOT be required for public log inclusion.
4. The canonical event validity MUST NOT depend on a trust score.
5. The canonical event validity MUST NOT depend on a future token, staking tier, subscription plan, or payment method.
6. Any compatible relay MAY accept commitments if it follows the relay validation rules.
7. Any compatible verifier MAY verify signatures, revocation state, inclusion proofs, and settled checkpoints.
8. Scoring providers MUST be plural and signed; the base protocol MUST NOT require a single centralized scoring oracle.
9. Revocation MUST be checkable from registry state and signed revocation events.
10. Checkpoint proofs MUST be portable across settlement backends through a common verification adapter.

43.3 Required Implementation Modules

The reference repository SHOULD contain these modules:

```
tsl/
  specs/
    json-schema/
    openapi/
    test-vectors/
  packages/
    core-ts/             # canonicalization, hashes, signatures, schemas
    core-rust/           # high-assurance verifier and log implementation
    core-python/         # scoring and data tooling bindings
    verifier-ts/         # browser/node verification package
    client-sdk-ts/       # signing, proof links, local store
    agent-sdk-python/    # AI-agent identity and delegated signing
  services/
    relay-node/          # commitment intake, receipts, proofs, gossip
    log-node/            # Merkle log, shard manager, checkpoint builder
    checkpoint-submitter/# submits roots to settlement backend
    resolver-node/       # identity, key, revocation, provider resolver
    verifier-api/        # hosted verifier wrapper around pure library
    scoring-provider/    # optional signed assessment provider
    auditor-node/        # log consistency and equivocation checks
  contracts/
    src/
      TrustIDRegistry.sol
      RevocationRegistry.sol
      CheckpointRegistry.sol
      ProviderRegistry.sol
      GovernanceRegistry.sol
    test/
    script/
```

```
  clients/
    web-verifier/
    browser-extension/
    cli/
    agent-sidecar/
  infra/
    docker-compose.yml
    k8s/
    terraform/
  docs/
    implementation-spec.tex
    protocol.md
    threat-model.md
    operations.md
```

44 Canonical Data and Cryptography

44.1 Encoding Rules

Every signed or hashed object MUST be converted into canonical bytes before signing or hashing. The reference implementation uses deterministic JSON canonicalization with these rules:

1. Objects are encoded as UTF-8 JSON.
2. Object keys are sorted lexicographically by Unicode code point.
3. Whitespace outside strings is removed.
4. Arrays preserve order.
5. Strings use standard JSON escaping.
6. Integers MUST be encoded in base-10 without leading zeroes.
7. Floating point values MUST NOT appear in signed core objects. Scores MAY appear as integers in basis points.
8. Timestamps MUST be RFC3339 UTC strings ending in `Z`, or unsigned integer milliseconds since Unix epoch where explicitly specified.
9. Binary values MUST be encoded as lowercase hex with `0x` prefix or multibase strings, depending on field definition.
10. Unknown fields MUST be rejected for core object types unless the object version declares an extension namespace.

No Ambiguous Signatures

The same semantic event must have exactly one byte representation before signing. If two libraries serialize the same event differently, the protocol is broken. Canonicalization tests are mandatory.

44.2 Domain Separation

Every hash MUST include a domain tag. The tag prevents a hash intended for one object type from being reused as another.

```
TSL_IDENTITY_V1        = "tsl.identity.v1"
```

```
TSL_EVENT_V1           = "tsl.event_commitment.v1"
TSL_RECEIPT_V1         = "tsl.receipt_commitment.v1"
TSL_ATTESTATION_V1     = "tsl.attestation.v1"
TSL_REVOCATION_V1      = "tsl.revocation.v1"
TSL_CHECKPOINT_V1      = "tsl.batch_checkpoint.v1"
TSL_MERKLE_LEAF_V1     = "tsl.merkle.leaf.v1"
TSL_MERKLE_NODE_V1     = "tsl.merkle.node.v1"
TSL_ASSESSMENT_V1      = "tsl.trust_assessment.v1"
TSL_ASSESSMENT_V2      = "tsl.trust_assessment.v2"
TSL_PROOF_BUNDLE_V1    = "tsl.proof_bundle.v1"
TSL_DELEGATION_V2      = "tsl.delegation_policy.v2"
TSL_AGENT_ACTION_V2    = "tsl.agent_action.v2"
```

The generic hash function is:

$$\mathrm{Hash}_T(x) = \mathrm{Hash}(\mathrm{utf8}(T) \parallel \texttt{0x00} \parallel x). \tag{110}$$

44.3 Reference Crypto Suite

The MVP crypto suite SHOULD use:

Primitive	Reference Choice
Signature	Ed25519 for local keys and agent keys
Smart account signature	EIP-712-compatible signature or account abstraction validation where applicable
Hash	SHA-256 for broad compatibility; BLAKE3 MAY be offered as an extension
Commitment salt	256-bit random salt from a cryptographically secure random generator
Nonce	256-bit random nonce per event
Merkle tree	Binary append-only tree with domain-separated leaves and internal nodes
Encryption at rest	XChaCha20-Poly1305 or AES-256-GCM for local private data

44.4 Core TypeScript Interfaces

```
export type Hex32 = `0x${string}`;      // exactly 32 bytes encoded as lowercase hex
export type HexSig = `0x${string}`;     // signature bytes
export type TrustID = string;           // did:tsl:<method>:<identifier>
export type RFC3339 = string;

export interface VerificationMethodV1 {
  id: string;
  type: "ed25519" | "secp256k1" | "smart_account";
  public_key: string;
  status: "active" | "revoked" | "expired";
  created_at: RFC3339;
  expires_at?: RFC3339;
}

export interface EventCommitmentV1 {
```

```
  type: "tsl.event_commitment.v1";
  event_class: "message" | "transaction" | "attestation" | "agent_call" | "code_release";
  sender: TrustID;
  signing_key_id: string;
  receiver_commitment?: Hex32;
  content_commitment: Hex32;
  metadata_commitment?: Hex32;
  previous_event_commitment?: Hex32;
  timestamp: RFC3339;
  nonce: Hex32;
  disclosure_policy: "local_only" | "commitment_only" | "selective" | "public";
  signature: HexSig;
}
```

45 Canonical Object Schemas

This section defines the minimum JSON Schema shape for production validation. The full repository SHOULD contain machine-readable schema files under `specs/json-schema/`.

45.1 Identity Schema

```
{
  "$id": "https://spec.tsl.network/schemas/identity.v1.json",
  "type": "object",
  "required": ["type", "id", "controller", "created_at", "verification_methods"],
  "additionalProperties": false,
  "properties": {
    "type": { "const": "tsl.identity.v1" },
    "id": { "type": "string", "pattern": "^did:tsl:" },
    "controller": { "type": "string" },
    "created_at": { "type": "string", "format": "date-time" },
    "verification_methods": {
      "type": "array",
      "minItems": 1,
      "items": {
        "type": "object",
        "required": ["id", "type", "public_key", "status", "created_at"],
        "additionalProperties": false,
        "properties": {
          "id": { "type": "string" },
          "type": { "enum": ["ed25519", "secp256k1", "smart_account"] },
          "public_key": { "type": "string" },
          "status": { "enum": ["active", "revoked", "expired"] },
          "created_at": { "type": "string", "format": "date-time" },
          "expires_at": { "type": "string", "format": "date-time" }
        }
      }
    },
    "recovery": { "type": "object" },
    "privacy_policy_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" }
  }
}
```

45.2 Event Commitment Schema

```
{
  "$id": "https://spec.tsl.network/schemas/event_commitment.v1.json",
  "type": "object",
  "required": [
    "type", "event_class", "sender", "signing_key_id",
    "content_commitment", "timestamp", "nonce", "disclosure_policy", "signature"
  ],
  "additionalProperties": false,
  "properties": {
    "type": { "const": "tsl.event_commitment.v1" },
    "event_class": { "enum": ["message", "transaction", "attestation", "agent_call", "
    code_release"] },
    "sender": { "type": "string", "pattern": "^did:tsl:" },
    "signing_key_id": { "type": "string" },
    "receiver_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "content_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "metadata_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "previous_event_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "timestamp": { "type": "string", "format": "date-time" },
    "nonce": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "disclosure_policy": { "enum": ["local_only", "commitment_only", "selective", "public"] },

    "signature": { "type": "string", "pattern": "^0x[0-9a-f]+$" }
  }
}
```

45.3 Receipt Commitment Schema

```
{
  "$id": "https://spec.tsl.network/schemas/receipt_commitment.v1.json",
  "type": "object",
  "required": ["type", "event_commitment", "receiver", "signing_key_id", "receipt_class", "
    timestamp", "signature"],
  "additionalProperties": false,
  "properties": {
    "type": { "const": "tsl.receipt_commitment.v1" },
    "event_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "receiver": { "type": "string", "pattern": "^did:tsl:" },
    "signing_key_id": { "type": "string" },
    "receipt_class": { "enum": ["received", "replied", "transacted", "completed", "disputed"]
     },
    "timestamp": { "type": "string", "format": "date-time" },
    "metadata_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "signature": { "type": "string", "pattern": "^0x[0-9a-f]+$" }
  }
}
```

45.4 Checkpoint Schema

```
{
```

```
  "$id": "https://spec.tsl.network/schemas/batch_checkpoint.v1.json",
  "type": "object",
  "required": [
    "type", "epoch_start_ms", "epoch_duration_ms", "shard",
    "event_root", "receipt_root", "attestation_root", "revocation_root",
    "event_count", "receipt_count", "previous_checkpoint", "relay_id", "relay_signature"
  ],
  "additionalProperties": false,
  "properties": {
    "type": { "const": "tsl.batch_checkpoint.v1" },
    "epoch_start_ms": { "type": "integer", "minimum": 0 },
    "epoch_duration_ms": { "type": "integer", "minimum": 1 },
    "shard": { "type": "string", "pattern": "^[0-9a-f]{4,16}$" },
    "event_root": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "receipt_root": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "attestation_root": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "revocation_root": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "event_count": { "type": "integer", "minimum": 0 },
    "receipt_count": { "type": "integer", "minimum": 0 },
    "previous_checkpoint": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "settlement_backend": { "type": "string" },
    "settlement_tx": { "type": "string" },
    "relay_id": { "type": "string", "pattern": "^did:tsl:" },
    "relay_signature": { "type": "string", "pattern": "^0x[0-9a-f]+$" }
  }
}
```

46 Backend Service Architecture

46.1 Service Map

Service	Port	Responsibility
relay-node	8080	Accept commitments, receipts, attestations, revocations; validate and enqueue them.
log-node	8081	Maintain sharded append-only Merkle logs; produce inclusion and consistency proofs.
resolver-node	8082	Resolve TrustIDs, active keys, revoked keys, providers, and model versions.
checkpoint-submitter	worker	Periodically submit checkpoint roots to settlement contracts.
verifier-api	8083	Hosted wrapper around pure verifier library. Not required for protocol validity.
scoring-provider	8084	Optional feature extraction, scoring, and signed trust assessments.
auditor-node	8085	Monitor logs, verify checkpoints, detect equivocation, and publish audit reports.

46.2 Reference Data Flow

```
1. Client creates event object without signature.
2. Client canonicalizes event payload.
3. Client signs canonical payload hash with active private key.
4. Client submits event commitment to any relay.
5. Relay validates schema, signature, timestamp, nonce, and key status.
6. Relay writes accepted item to durable queue.
7. Log node appends commitment to shard for current epoch.
8. Log node returns inclusion promise immediately if configured.
9. At epoch close, log node computes Merkle roots and checkpoint.
10. Checkpoint submitter anchors checkpoint root to settlement backend.
11. Verifier checks signature, key status, inclusion proof, checkpoint settlement, and
    revocation.
```

46.3 Queue Topics

```
tsl.commitments.accepted.v1
tsl.receipts.accepted.v1
tsl.attestations.accepted.v1
tsl.revocations.accepted.v1
tsl.checkpoints.ready.v1
tsl.checkpoints.settled.v1
tsl.audit.findings.v1
```

The queue implementation MAY be Kafka, NATS JetStream, Redpanda, Redis Streams, or another durable queue. Message payloads MUST be canonical objects or pointers to immutable stored canonical bytes.

47 Database Implementation

The reference backend uses PostgreSQL for relay, resolver, and log-node state. Local clients SHOULD use SQLite or IndexedDB for private data. The public protocol does not require a specific database engine.

47.1 PostgreSQL Schema

```
CREATE TABLE trust_identities (
    trust_id TEXT PRIMARY KEY,
    controller TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL,
    identity_document JSONB NOT NULL,
    identity_hash TEXT NOT NULL CHECK (identity_hash ~ '^0x[0-9a-f]{64}$'),
    latest_checkpoint_hash TEXT,
    created_block_number BIGINT,
    created_tx_hash TEXT,
    inserted_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE verification_keys (
```

```
    trust_id TEXT NOT NULL REFERENCES trust_identities(trust_id),
    key_id TEXT NOT NULL,
    key_type TEXT NOT NULL,
    public_key TEXT NOT NULL,
    status TEXT NOT NULL CHECK (status IN ('active','revoked','expired')),
    created_at TIMESTAMPTZ NOT NULL,
    expires_at TIMESTAMPTZ,
    revoked_at TIMESTAMPTZ,
    revocation_reason TEXT,
    PRIMARY KEY (trust_id, key_id)
);

CREATE TABLE revocations (
    revocation_hash TEXT PRIMARY KEY CHECK (revocation_hash ~ '^0x[0-9a-f]{64}$'),
    trust_id TEXT NOT NULL,
    key_id TEXT NOT NULL,
    reason_class TEXT NOT NULL,
    effective_at TIMESTAMPTZ NOT NULL,
    replacement_key_id TEXT,
    canonical_body BYTEA NOT NULL,
    signature TEXT NOT NULL,
    accepted_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE event_commitments (
    commitment_hash TEXT PRIMARY KEY CHECK (commitment_hash ~ '^0x[0-9a-f]{64}$'),
    sender_trust_id TEXT NOT NULL,
    signing_key_id TEXT NOT NULL,
    event_class TEXT NOT NULL,
    content_commitment TEXT NOT NULL,
    receiver_commitment TEXT,
    metadata_commitment TEXT,
    previous_event_commitment TEXT,
    event_timestamp TIMESTAMPTZ NOT NULL,
    nonce TEXT NOT NULL,
    disclosure_policy TEXT NOT NULL,
    canonical_body BYTEA NOT NULL,
    signature TEXT NOT NULL,
    relay_id TEXT NOT NULL,
    shard TEXT NOT NULL,
    epoch_start_ms BIGINT NOT NULL,
    log_index BIGINT,
    accepted_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(sender_trust_id, signing_key_id, nonce)
);

CREATE INDEX idx_event_commitments_sender_time ON event_commitments(sender_trust_id,
    event_timestamp DESC);
CREATE INDEX idx_event_commitments_epoch_shard ON event_commitments(epoch_start_ms, shard,
    log_index);

CREATE TABLE receipt_commitments (
    receipt_hash TEXT PRIMARY KEY CHECK (receipt_hash ~ '^0x[0-9a-f]{64}$'),
    event_commitment TEXT NOT NULL,
    receiver_trust_id TEXT NOT NULL,
    signing_key_id TEXT NOT NULL,
```

```
    receipt_class TEXT NOT NULL,
    receipt_timestamp TIMESTAMPTZ NOT NULL,
    metadata_commitment TEXT,
    canonical_body BYTEA NOT NULL,
    signature TEXT NOT NULL,
    relay_id TEXT NOT NULL,
    shard TEXT NOT NULL,
    epoch_start_ms BIGINT NOT NULL,
    log_index BIGINT,
    accepted_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE attestations (
    attestation_hash TEXT PRIMARY KEY CHECK (attestation_hash ~ '^0x[0-9a-f]{64}$'),
    issuer_trust_id TEXT NOT NULL,
    subject_trust_id TEXT NOT NULL,
    attestation_class TEXT NOT NULL,
    visibility TEXT NOT NULL,
    issued_at TIMESTAMPTZ NOT NULL,
    expires_at TIMESTAMPTZ,
    claim_commitment TEXT NOT NULL,
    canonical_body BYTEA NOT NULL,
    signature TEXT NOT NULL,
    accepted_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE checkpoints (
    checkpoint_hash TEXT PRIMARY KEY CHECK (checkpoint_hash ~ '^0x[0-9a-f]{64}$'),
    epoch_start_ms BIGINT NOT NULL,
    epoch_duration_ms BIGINT NOT NULL,
    shard TEXT NOT NULL,
    event_root TEXT NOT NULL,
    receipt_root TEXT NOT NULL,
    attestation_root TEXT NOT NULL,
    revocation_root TEXT NOT NULL,
    event_count BIGINT NOT NULL,
    receipt_count BIGINT NOT NULL,
    previous_checkpoint TEXT NOT NULL,
    relay_id TEXT NOT NULL,
    relay_signature TEXT NOT NULL,
    settlement_backend TEXT,
    settlement_tx TEXT,
    settlement_status TEXT NOT NULL DEFAULT 'pending',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    settled_at TIMESTAMPTZ,
    UNIQUE(epoch_start_ms, shard)
);

CREATE TABLE merkle_nodes (
    epoch_start_ms BIGINT NOT NULL,
    shard TEXT NOT NULL,
    tree_kind TEXT NOT NULL CHECK (tree_kind IN ('event','receipt','attestation','revocation
    ')),
    level INTEGER NOT NULL,
    node_index BIGINT NOT NULL,
    node_hash TEXT NOT NULL CHECK (node_hash ~ '^0x[0-9a-f]{64}$'),
```

```
    PRIMARY KEY (epoch_start_ms, shard, tree_kind, level, node_index)
);

CREATE TABLE provider_registry_cache (
    provider_id TEXT PRIMARY KEY,
    public_key TEXT NOT NULL,
    policy_commitment TEXT NOT NULL,
    status TEXT NOT NULL,
    latest_model_id TEXT,
    updated_at TIMESTAMPTZ NOT NULL
);

CREATE TABLE trust_assessments (
    assessment_hash TEXT PRIMARY KEY CHECK (assessment_hash ~ '^0x[0-9a-f]{64}$'),
    subject_trust_id TEXT NOT NULL,
    issuer_provider_id TEXT NOT NULL,
    score_bps INTEGER NOT NULL CHECK (score_bps BETWEEN 0 AND 10000),
    label TEXT NOT NULL,
    model_version TEXT NOT NULL,
    evidence_commitment TEXT NOT NULL,
    issued_at TIMESTAMPTZ NOT NULL,
    expires_at TIMESTAMPTZ NOT NULL,
    canonical_body BYTEA NOT NULL,
    signature TEXT NOT NULL
);
```

47.2 v2 Scoring, Graph, Governance, and Agent Tables

The v1 tables above store the baseline protocol. Implementations that claim `TSL-RC2` or higher SHOULD add typed v2 tables or equivalent JSONB stores with canonical-body retention. Raw private evidence MUST NOT be stored in these public-service tables.

```
CREATE TABLE scoring_profiles_v2 (
    profile_id TEXT PRIMARY KEY,
    provider_id TEXT NOT NULL,
    domain TEXT NOT NULL,
    model_version TEXT NOT NULL,
    profile_hash TEXT NOT NULL CHECK (profile_hash ~ '^0x[0-9a-f]{64}$'),
    evaluation_report_commitment TEXT NOT NULL,
    issued_at TIMESTAMPTZ NOT NULL,
    expires_at TIMESTAMPTZ NOT NULL,
    canonical_body BYTEA NOT NULL,
    signature TEXT NOT NULL
);

CREATE TABLE feature_definitions_v2 (
    feature_id TEXT PRIMARY KEY,
    profile_id TEXT NOT NULL REFERENCES scoring_profiles_v2(profile_id),
    feature_name TEXT NOT NULL,
    feature_family TEXT NOT NULL,
    value_unit TEXT NOT NULL,
    normalization_rule JSONB NOT NULL,
    privacy_class TEXT NOT NULL,
    canonical_body BYTEA NOT NULL
```

```
);

CREATE TABLE domain_policies_v1 (
    policy_id TEXT PRIMARY KEY,
    domain TEXT NOT NULL,
    min_coverage_bps INTEGER NOT NULL CHECK (min_coverage_bps BETWEEN 0 AND 10000),
    threshold_policy JSONB NOT NULL,
    false_positive_cost_bps INTEGER,
    false_negative_cost_bps INTEGER,
    canonical_body BYTEA NOT NULL,
    signature TEXT
);

CREATE TABLE evidence_coverage_v1 (
    coverage_hash TEXT PRIMARY KEY CHECK (coverage_hash ~ '^0x[0-9a-f]{64}$'),
    subject_trust_id TEXT NOT NULL,
    signed_event_count BIGINT NOT NULL,
    reciprocal_receipt_count BIGINT NOT NULL,
    unique_counterparty_count BIGINT NOT NULL,
    trusted_counterparty_mass_bps INTEGER NOT NULL,
    coverage_bps INTEGER NOT NULL CHECK (coverage_bps BETWEEN 0 AND 10000),
    computed_at TIMESTAMPTZ NOT NULL,
    canonical_body BYTEA NOT NULL
);

CREATE TABLE trust_assessments_v2 (
    assessment_hash TEXT PRIMARY KEY CHECK (assessment_hash ~ '^0x[0-9a-f]{64}$'),
    subject_trust_id TEXT NOT NULL,
    provider_id TEXT NOT NULL,
    profile_id TEXT NOT NULL,
    score_bps INTEGER NOT NULL CHECK (score_bps BETWEEN 0 AND 10000),
    confidence_low_bps INTEGER NOT NULL,
    confidence_high_bps INTEGER NOT NULL,
    risk_label TEXT NOT NULL,
    evidence_coverage_hash TEXT,
    evidence_commitment TEXT NOT NULL,
    issued_at TIMESTAMPTZ NOT NULL,
    expires_at TIMESTAMPTZ NOT NULL,
    canonical_body BYTEA NOT NULL,
    signature TEXT NOT NULL
);

CREATE TABLE metadata_fingerprint_commitments_v1 (
    fingerprint_commitment TEXT PRIMARY KEY CHECK (fingerprint_commitment ~ '^0x[0-9a-f]{64}$
    '),
    subject_trust_id TEXT NOT NULL,
    scope_class TEXT NOT NULL,
    scope_commitment TEXT NOT NULL,
    bucket_profile TEXT NOT NULL,
    created_at_bucket TEXT NOT NULL,
    expires_at TIMESTAMPTZ NOT NULL,
    disclosure_policy TEXT NOT NULL,
    canonical_body BYTEA NOT NULL,
    signature TEXT NOT NULL
);
```

```
CREATE TABLE graph_feature_vectors_v1 (
    feature_vector_hash TEXT PRIMARY KEY CHECK (feature_vector_hash ~ '^0x[0-9a-f]{64}$'),
    subject_trust_id TEXT NOT NULL,
    graph_profile_id TEXT NOT NULL,
    computed_at TIMESTAMPTZ NOT NULL,
    reciprocity_bps INTEGER,
    counterparty_diversity_bps INTEGER,
    effective_counterparty_count_milli BIGINT,
    community_escape_bps INTEGER,
    trusted_neighbor_mass_bps INTEGER,
    adversarial_proximity_bps INTEGER,
    privacy_level TEXT NOT NULL,
    canonical_body BYTEA NOT NULL
);

CREATE TABLE sybil_assessments_v1 (
    sybil_assessment_hash TEXT PRIMARY KEY CHECK (sybil_assessment_hash ~ '^0x[0-9a-f]{64}$'),

    subject_trust_id TEXT NOT NULL,
    cluster_id_commitment TEXT NOT NULL,
    adversary_tier_assumed TEXT NOT NULL,
    risk_score_bps INTEGER NOT NULL,
    risk_label TEXT NOT NULL,
    computed_at TIMESTAMPTZ NOT NULL,
    canonical_body BYTEA NOT NULL,
    signature TEXT NOT NULL
);

CREATE TABLE drift_reports_v1 (
    drift_report_hash TEXT PRIMARY KEY CHECK (drift_report_hash ~ '^0x[0-9a-f]{64}$'),
    subject_trust_id TEXT NOT NULL,
    drift_score_bps INTEGER NOT NULL,
    drift_label TEXT NOT NULL,
    action TEXT NOT NULL,
    computed_at TIMESTAMPTZ NOT NULL,
    canonical_body BYTEA NOT NULL,
    signature TEXT NOT NULL
);

CREATE TABLE model_cards_v2 (
    model_id TEXT PRIMARY KEY,
    provider_id TEXT NOT NULL,
    model_version TEXT NOT NULL,
    model_card_hash TEXT NOT NULL CHECK (model_card_hash ~ '^0x[0-9a-f]{64}$'),
    evaluation_report_commitment TEXT NOT NULL,
    privacy_report_commitment TEXT NOT NULL,
    issued_at TIMESTAMPTZ NOT NULL,
    canonical_body BYTEA NOT NULL,
    signature TEXT NOT NULL
);

CREATE TABLE evaluation_reports_v1 (
    report_id TEXT PRIMARY KEY,
    model_id TEXT NOT NULL,
    domain TEXT NOT NULL,
    auroc_bps INTEGER,
```

```
    auprc_bps INTEGER,
    ece_bps INTEGER,
    privacy_leakage_bps INTEGER,
    promotion_gate_result TEXT NOT NULL,
    issued_at TIMESTAMPTZ NOT NULL,
    canonical_body BYTEA NOT NULL,
    signature TEXT NOT NULL
);

CREATE TABLE delegation_policies_v2 (
    policy_id TEXT PRIMARY KEY,
    principal_trust_id TEXT NOT NULL,
    delegate_trust_id TEXT NOT NULL,
    valid_from TIMESTAMPTZ NOT NULL,
    valid_until TIMESTAMPTZ NOT NULL,
    revocation_pointer TEXT NOT NULL,
    canonical_body BYTEA NOT NULL,
    signature TEXT NOT NULL
);

CREATE TABLE agent_actions_v2 (
    action_id TEXT PRIMARY KEY,
    agent_trust_id TEXT NOT NULL,
    principal_trust_id TEXT NOT NULL,
    action TEXT NOT NULL,
    resource TEXT NOT NULL,
    tool TEXT,
    parameters_commitment TEXT CHECK (parameters_commitment ~ '^0x[0-9a-f]{64}$'),
    issued_at TIMESTAMPTZ NOT NULL,
    canonical_body BYTEA NOT NULL,
    signature TEXT NOT NULL
);
```

47.3 Local Client Store

Raw messages, exact counterparties, salts, undisclosed metadata, and private graph features remain local.

```
CREATE TABLE local_private_events (
    local_event_id TEXT PRIMARY KEY,
    commitment_hash TEXT NOT NULL,
    raw_message_encrypted BLOB,
    private_metadata_encrypted BLOB,
    receiver_trust_id TEXT,
    content_salt BLOB NOT NULL,
    metadata_salt BLOB,
    receiver_salt BLOB,
    created_at INTEGER NOT NULL
);

CREATE TABLE local_relationships (
    counterparty_trust_id TEXT PRIMARY KEY,
    first_seen_at INTEGER NOT NULL,
    last_seen_at INTEGER NOT NULL,
```

```
    reciprocal_receipt_count INTEGER NOT NULL DEFAULT 0,
    local_label TEXT,
    private_notes_encrypted BLOB
);

CREATE TABLE local_settings (
    key TEXT PRIMARY KEY,
    value_encrypted BLOB NOT NULL
);
```

48 Merkle Log Implementation

48.1 Shard Assignment

The shard for a commitment is computed from the sender TrustID and epoch. For MVP, use the first 16 bits of the SHA-256 hash of the sender TrustID.

$$\text{shard}(\text{TrustID}, t) = \text{hex}_4(\text{Hash}(\text{utf8}(\text{TrustID}))) \parallel : \parallel \text{epoch}(t). \tag{111}$$

The default epoch duration SHOULD be five minutes for production and one minute for test networks.

48.2 Leaf and Node Hashes

For an item commitment C_i, the leaf hash is:

$$L_i = \text{Hash}_{\texttt{TSL\_MERKLE\_LEAF\_V1}}(\text{uint64be}(i) \parallel C_i). \tag{112}$$

For two child hashes a and b, the internal node hash is:

$$N = \text{Hash}_{\texttt{TSL\_MERKLE\_NODE\_V1}}(a \parallel b). \tag{113}$$

If a level has an odd number of nodes, the final node is promoted unchanged to the next level. This rule MUST be consistent across implementations.

48.3 Inclusion Proof Object

```
{
  "type": "tsl.inclusion_proof.v1",
  "tree_kind": "event",
  "commitment": "0x...",
  "leaf_index": 184293,
  "leaf_hash": "0x...",
  "root": "0x...",
  "epoch_start_ms": 1779667200000,
  "epoch_duration_ms": 300000,
  "shard": "00af",
  "path": [
    { "side": "right", "hash": "0x..." },
```

```
    { "side": "left",  "hash": "0x..." }
  ],
  "checkpoint_hash": "0x..."
}
```

48.4 Inclusion Verification Pseudocode

```
function verifyInclusion(proof): boolean {
  h = hashLeaf(proof.leaf_index, proof.commitment)
  if h != proof.leaf_hash: return false

  for step in proof.path:
    if step.side == "left":
      h = hashNode(step.hash, h)
    else if step.side == "right":
      h = hashNode(h, step.hash)
    else:
      return false

  return h == proof.root
}
```

48.5 Consistency Proofs

Each closed checkpoint MUST link to the previous checkpoint for the same shard. The minimum MVP consistency check verifies:

1. checkpoint k includes `previous_checkpoint = hash(k-1)`,
2. checkpoint k is signed by an authorized relay,
3. checkpoint k is settled or pending under the declared settlement policy,
4. no two checkpoints exist for the same $(epoch, shard)$ with different roots.

Auditor nodes SHOULD additionally verify append-only tree consistency and publish signed audit findings.

49 Relay Node Specification

49.1 Commitment Intake Validation

A relay MUST perform these checks before accepting an event commitment:

1. Validate JSON schema.
2. Reject unknown fields.
3. Check timestamp is within the relay acceptance window. Default: ± 10 minutes.
4. Resolve sender TrustID.
5. Confirm signing key exists and was active at event timestamp.
6. Confirm signing key is not revoked at event timestamp.

7. Recompute canonical payload bytes excluding `signature`.
8. Verify signature.
9. Recompute event commitment hash.
10. Enforce replay protection with $(sender, signing\_key\_id, nonce)$.
11. Assign shard and epoch.
12. Persist canonical bytes and enqueue for log append.

49.2 Relay Response States

State	Meaning
accepted	Relay validated the object and queued it for log append.
included	Object has a log index and can receive an inclusion proof.
checkpointed	Object's epoch is closed and root has been checkpointed.
settled	Checkpoint has settlement proof from the declared backend.
rejected	Object failed validation.

49.3 Relay Gossip

To avoid one relay becoming the canonical trust authority, relays SHOULD gossip closed checkpoints and signed log summaries.

```
POST /v1/gossip/checkpoint
POST /v1/gossip/audit-finding
GET  /v1/gossip/peers
GET  /v1/gossip/checkpoints/{epoch}/{shard}
```

A receiving relay MUST verify the checkpoint signature, checkpoint hash, and settlement status before importing it.

50 API Specification

All hosted APIs are wrappers around protocol libraries. A proof that requires the hosted API to be trusted is not a valid decentralized proof.

50.1 HTTP Conventions

- All requests and responses use `application/json` unless returning canonical bytes.
- Idempotent submission SHOULD use `Idempotency-Key`.
- API errors use a standard `error.code`, `error.message`, and optional `error.details`.
- Verifier endpoints MUST return enough information for the client to independently recompute verification.

50.2 Endpoint Summary

Endpoint	Method	Purpose
`/v1/identity/create`	POST	Create and optionally register a TrustID.
`/v1/identity/{trustId}`	GET	Resolve identity document and active key state.
`/v1/keys/rotate`	POST	Submit signed key rotation.
`/v1/keys/revoke`	POST	Submit signed key revocation.
`/v1/commitments`	POST	Submit event commitment.
`/v1/receipts`	POST	Submit receipt commitment.
`/v1/attestations`	POST	Submit signed attestation.
`/v1/proofs/{commitment}`	GET	Fetch inclusion proof and checkpoint data.
`/v1/proof-bundles/{bundleId}`	GET	Fetch portable proof bundle.
`/v1/checkpoints/{epoch}/{shard}`	GET	Fetch checkpoint.
`/v1/verify`	POST	Verify envelope, proof, key state, and optional message.
`/v1/assessments`	POST	Request optional trust assessment.
`/v1/scoring-profiles/{profileId}`	GET	Fetch signed scoring profile.
`/v1/model-cards/{modelId}`	GET	Fetch signed model card.
`/v1/delegations/verify`	POST	Verify delegated agent action.
`/v1/audit/checkpoint`	POST	Submit signed auditor finding.

50.3 Create Identity

```
POST /v1/identity/create

Request:
{
  "type": "tsl.identity_create_request.v1",
  "controller_type": "local",
  "verification_method": {
    "type": "ed25519",
    "public_key": "0x..."
  },
  "recovery_policy_commitment": "0x...",
  "privacy_policy_commitment": "0x..."
}

Response:
{
  "trust_id": "did:tsl:local:0x...",
  "identity_hash": "0x...",
  "registry_status": "registered | pending | local_only",
  "registry_tx": "0x..."
}
```

50.4 Submit Commitment

```
POST /v1/commitments
```

```
Request:
{
  "event": { "type": "tsl.event_commitment.v1", "...": "..." }
}

Response:
{
  "status": "accepted",
  "commitment_hash": "0x...",
  "relay_id": "did:tsl:relay:001",
  "epoch_start_ms": 1779667200000,
  "epoch_duration_ms": 300000,
  "shard": "00af",
  "inclusion_promise": "0xrelaySignedPromise"
}
```

50.5 Verify

```
POST /v1/verify

Request:
{
  "envelope": { "type": "tsl.event_commitment.v1", "...": "..." },
  "proof": { "type": "tsl.inclusion_proof.v1", "...": "..." },
  "checkpoint": { "type": "tsl.batch_checkpoint.v1", "...": "..." },
  "message_disclosure": {
    "raw_message": "optional",
    "content_salt": "optional"
  },
  "verifier_policy": {
    "require_settlement": true,
    "max_clock_skew_ms": 600000,
    "accepted_scoring_providers": ["did:tsl:provider:continuity-labs"]
  }
}

Response:
{
  "verified": true,
  "checks": {
    "schema_valid": true,
    "signature_valid": true,
    "key_active": true,
    "not_revoked": true,
    "content_commitment_matches": true,
    "included_in_log": true,
    "checkpoint_valid": true,
    "checkpoint_settled": true
  },
  "commitment_hash": "0x...",
  "risk_label": "not_assessed",
  "explanation": ["Signature valid", "Key active", "Included in settled checkpoint"]
}
```

50.6 Canonical Error-Code Registry

Every API and verifier library MUST return canonical machine-readable error codes. User-facing copy MAY be localized, but the code, source layer, fatality, and retryability semantics MUST remain stable.

Code	Layer	Fatal	Retry	User-Facing Meaning
TSL_SCHEMA_INVALID	schema	yes	no	The proof object is malformed or unsupported.
TSL_CANONICALIZATION_FAILED	schema	yes	no	The object cannot be serialized deterministically.
TSL_UNSUPPORTED_OBJECT_VERSION	schema	yes	no	This verifier does not support the object version.
TSL_UNKNOWN_REQUIRED_FIELD	schema	yes	no	A required field is unknown or outside an extension namespace.
TSL_SIGNATURE_INVALID	crypto	yes	no	The signature does not match the claimed key.
TSL_KEY_NOT_FOUND	identity	yes	maybe	The signing key is not registered for this identity.
TSL_KEY_EXPIRED	identity	yes	no	The signing key had expired at the relevant time.
TSL_KEY_REVOKED	identity	yes	no	The key was revoked before the relevant event time.
TSL_REVOCATION_STATE_STALE	identity	no	yes	Revocation state is too old for the verifier policy.
TSL_NONCE_REPLAY	relay	yes	no	This sender/key/nonce was already accepted.
TSL_TIMESTAMP_OUT_OF_WINDOW	relay	yes	yes	The timestamp is outside relay acceptance policy.
TSL_INCLUSION_INVALID	log	yes	no	The Merkle proof does not match the checkpoint root.
TSL_CHECKPOINT_INVALID	settlement	yes	no	The checkpoint signature or hash is invalid.
TSL_CHECKPOINT_CONFLICT	settlement	yes	no	Conflicting roots exist for the same epoch and shard.
TSL_SETTLEMENT_MISSING	settlement	policy	yes	Settlement is missing but required by policy.
TSL_PROOF_BUNDLE_REDACTED	privacy	no	no	Some private fields were intentionally redacted.
TSL_DISCLOSURE_CONSENT_REQUIRED	privacy	policy	no	More disclosure requires explicit consent.
TSL_INSUFFICIENT_EVIDENCE	scoring	no	yes	The scorer abstained because evidence coverage is too low.
TSL_PROVIDER_INACTIVE	scoring	policy	maybe	The scoring provider is not active under verifier policy.
TSL_MODEL_NOT_REGISTERED	scoring	policy	maybe	The referenced model version is not registered.
TSL_MODEL_EXPIRED	scoring	no	yes	The assessment expired and should be refreshed.
TSL_ASSESSMENT_SIGNATURE_INVALID	scoring	yes	no	The assessment signature is invalid.
TSL_NEGATIVE_CLAIM_APPEALED	attestation	no	no	A negative claim exists but has active appeal context.
TSL_DELEGATION_MISSING	agent	yes	yes	No valid principal authorization was provided.
TSL_DELEGATION_EXPIRED	agent	yes	no	The action occurred outside delegated time.
TSL_DELEGATION_REVOKED	agent	yes	no	The principal revoked the delegation before action.
TSL_DELEGATION_SCOPE_VIOLATION	agent	yes	no	The action is outside delegated scope.
TSL_DELEGATION_CONSTRAINT_VIOLATION	agent	yes	no	Amount, tool, counterparty, rate, or approval constraints failed.

For the `Fatal` column, `policy` means the verifier policy decides whether the failure blocks the action or only displays a warning.

51 Verifier Implementation

51.1 Pure Verification Algorithm

The verifier MUST be implementable as a pure library with no trusted network call. Network calls can fetch missing state, but fetched data must be independently verified.

```
function verifyTSL(input, policy): VerificationResult {
  result = new VerificationResult()
```

```
  result.schema_valid = validateSchema(input.envelope)
  if !result.schema_valid: return result.fail("TSL_SCHEMA_INVALID")

  canonical = canonicalize(removeSignature(input.envelope))
  event_hash = hashDomain("tsl.event_commitment.v1", canonical)

  identity = resolveTrustID(input.envelope.sender)
  key = identity.getKey(input.envelope.signing_key_id)

  result.key_found = key != null
  result.key_active = keyActiveAt(key, input.envelope.timestamp)
  result.not_revoked = !revokedAt(key, input.envelope.timestamp)
  result.signature_valid = verifySignature(key.public_key, event_hash, input.envelope.
    signature)

  result.commitment_hash = hash(event_hash || input.envelope.signature)

  if input.message_disclosure exists:
    result.content_commitment_matches = verifyContentCommitment(
      input.message_disclosure.raw_message,
      input.message_disclosure.content_salt,
      input.envelope.content_commitment
    )

  result.included_in_log = verifyInclusion(input.proof)
  result.checkpoint_valid = verifyCheckpoint(input.checkpoint)
  result.checkpoint_matches_proof = input.proof.root == rootForKind(input.checkpoint, input.
    proof.tree_kind)
  result.checkpoint_settled = verifySettlement(input.checkpoint)

  return result.finalize(policy)
}
```

51.2 Verification Result Semantics

A result can be cryptographically valid but not trusted. The implementation MUST separate these states:

Field	Meaning
`cryptographic_validity`	Signature, key, commitment, inclusion, checkpoint, and revocation checks.
`content_match`	Raw disclosed content matches the commitment. Optional when content is undisclosed.
`settlement_status`	Whether checkpoint is settled, pending, or unavailable.
`risk_assessment`	Optional signed scoring result. Not required for cryptographic validity.
`verdict`	Human-facing summary generated from policy and checks.

52 Smart Contract Implementation

Contracts settle registry facts and checkpoint roots. They do not compute trust scores and do not store raw messages.

52.1 Checkpoint Registry Storage

```
struct Checkpoint {
    uint64 epochStartMs;
    uint32 epochDurationMs;
    bytes32 shard;
    bytes32 eventRoot;
    bytes32 receiptRoot;
    bytes32 attestationRoot;
    bytes32 revocationRoot;
    uint64 eventCount;
    uint64 receiptCount;
    bytes32 previousCheckpoint;
    bytes32 relayId;
    uint64 submittedAt;
}

mapping(bytes32 => Checkpoint) public checkpointsByHash;
mapping(bytes32 => bytes32) public checkpointHashByEpochShard;
```

The key for `checkpointHashByEpochShard` is:

$$K = \text{Hash}(\text{uint64be}(epochStartMs) \parallel shard). \tag{114}$$

52.2 Checkpoint Submission

```
function submitCheckpoint(CheckpointInput calldata input, bytes calldata relaySignature)
    external {
    require(isAuthorizedRelay(input.relayId), "UNAUTHORIZED_RELAY");

    bytes32 key = keccak256(abi.encodePacked(input.epochStartMs, input.shard));
    require(checkpointHashByEpochShard[key] == bytes32(0), "CHECKPOINT_EXISTS");

    bytes32 checkpointHash = hashCheckpoint(input);
    require(verifyRelaySignature(input.relayId, checkpointHash, relaySignature), "
    BAD_RELAY_SIG");

    checkpointsByHash[checkpointHash] = Checkpoint({
        epochStartMs: input.epochStartMs,
        epochDurationMs: input.epochDurationMs,
        shard: input.shard,
        eventRoot: input.eventRoot,
        receiptRoot: input.receiptRoot,
        attestationRoot: input.attestationRoot,
        revocationRoot: input.revocationRoot,
        eventCount: input.eventCount,
        receiptCount: input.receiptCount,
```

```
        previousCheckpoint: input.previousCheckpoint,
        relayId: input.relayId,
        submittedAt: uint64(block.timestamp)
    });

    checkpointHashByEpochShard[key] = checkpointHash;
    emit CheckpointSubmitted(checkpointHash, input.epochStartMs, input.shard, input.eventRoot
    );
}
```

52.3 Token-Free Core

Core contracts MUST NOT require a native token. Fee policy, staking, rewards, and governance may be added through separate modules.

```
interface IFeePolicy {
    function authorizeAction(address actor, bytes32 actionType, bytes32 actionId) external
    returns (bool);
}

interface IRelayStakingPolicy {
    function isAuthorizedRelay(bytes32 relayId) external view returns (bool);
}
```

The default MVP policy MAY be an allowlist or multisig-controlled relay registry. A later token-based policy can replace it without changing event commitments, receipts, Merkle proofs, or verification rules.

53 Identity Resolution and Revocation

53.1 Resolution Algorithm

```
function resolveTrustID(trustId, atTime): ResolvedIdentity {
  method = parseMethod(trustId)
  doc = methodResolver(method).resolve(trustId)
  registryState = settlementBackend.getIdentityState(trustId)
  revocations = settlementBackend.getRevocations(trustId)

  activeKeys = []
  for key in doc.verification_methods:
    if key.created_at <= atTime and (key.expires_at is null or atTime < key.expires_at):
      if not isRevoked(key, revocations, atTime):
        activeKeys.append(key)

  return { trust_id: trustId, document: doc, active_keys: activeKeys, revocations }
}
```

53.2 Revocation Precedence

Revocation MUST take precedence over local cache, stale identity documents, or old proofs. If a key is revoked effective at time t_r, then:

- events signed before t_r MAY remain historically valid,
- events signed at or after t_r MUST fail key validity,
- clients SHOULD show compromise warnings for a configurable window before t_r if the revocation reason is compromise.

54 Scoring Provider Implementation

Scoring is optional and signed. It must not be required for protocol verification.

54.1 Feature Extraction Contract

```
export interface FeatureExtractor {
  extract(input: {
    subject: TrustID;
    verifiedEvents: VerifiedEventSummary[];
    verifiedReceipts: VerifiedReceiptSummary[];
    attestations: VerifiedAttestationSummary[];
    revocationState: RevocationSummary;
    localContext?: LocalVerifierContext;
  }): Promise<FeatureVectorV1>;
}

export interface FeatureVectorV1 {
  type: "tsl.feature_vector.v1";
  subject: TrustID;
  computed_at: RFC3339;
  identity_age_days: number;
  active_key_age_days: number;
  signed_event_count: number;
  reciprocal_receipt_count: number;
  unique_counterparty_count: number;
  trusted_neighbor_ratio_bps: number;
  attestation_quality_bps: number;
  temporal_consistency_bps: number;
  revocation_risk_bps: number;
  local_relationship_bps?: number;
}
```

54.2 Reference Score

Scores SHOULD be stored as integer basis points from 0 to 10000.

```
score_bps =
  2000 * crypto_validity_bps / 10000 +
  1500 * identity_age_bps / 10000 +
  1500 * reciprocity_bps / 10000 +
  1500 * trusted_neighbor_ratio_bps / 10000 +
  1000 * receipt_quality_bps / 10000 +
  1000 * attestation_quality_bps / 10000 +
  1000 * temporal_consistency_bps / 10000 +
   500 * local_relationship_bps / 10000
```

The signed assessment MUST include the model version, feature disclosure policy, evidence commitment, expiration, and provider signature.

55 Security, Abuse Controls, and Rate Limits

55.1 Replay Protection

Relays MUST reject duplicate $(sender, signing\_key\_id, nonce)$. Clients MUST generate a new 256-bit random nonce for every event.

55.2 Rate Limits

Recommended MVP limits:

Actor / Action	Default Limit
New identity event submissions	100 per hour until identity has settled checkpoint history.
Established identity submissions	10,000 per hour, adjustable by relay policy.
Negative public attestations	10 per day per issuer, stricter for low-continuity issuers.
Proof requests	1,000 per hour per IP or API key for hosted API; unlimited for local verification.
Checkpoint submissions	One per relay per epoch-shard.

55.3 Negative Attestation Controls

A public negative attestation MUST include:

- issuer identity,
- subject identity,
- claim class,
- evidence commitment,
- expiration,
- appeal pointer or policy pointer,
- issuer signature.

The protocol SHOULD distinguish `private_warning`, `provider_risk_flag`, and `public_negative_claim`.

55.4 Privacy Requirements

1. Receiver identities SHOULD be blinded by default.
2. Content commitments MUST use salts unless the content is intentionally public.

3. Metadata commitments MUST use salts.
4. Public logs MUST NOT include raw platform identifiers by default.
5. Clients MUST warn users before disclosing raw messages or exact counterparties.

56 Deployment Specification

56.1 Docker Compose MVP

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: tsl
      POSTGRES_USER: tsl
      POSTGRES_PASSWORD: tsl_dev_only
    ports: ["5432:5432"]

  redis:
    image: redis:7
    ports: ["6379:6379"]

  relay-node:
    build: ./services/relay-node
    environment:
      DATABASE_URL: postgres://tsl:tsl_dev_only@postgres:5432/tsl
      QUEUE_URL: redis://redis:6379
      TSL_RELAY_ID: did:tsl:relay:dev
      TSL_EPOCH_MS: 300000
    ports: ["8080:8080"]

  log-node:
    build: ./services/log-node
    environment:
      DATABASE_URL: postgres://tsl:tsl_dev_only@postgres:5432/tsl
      QUEUE_URL: redis://redis:6379
    ports: ["8081:8081"]

  resolver-node:
    build: ./services/resolver-node
    environment:
      DATABASE_URL: postgres://tsl:tsl_dev_only@postgres:5432/tsl
      SETTLEMENT_RPC_URL: http://anvil:8545
    ports: ["8082:8082"]

  verifier-api:
    build: ./services/verifier-api
    ports: ["8083:8083"]

  anvil:
    image: ghcr.io/foundry-rs/foundry:latest
    command: anvil --host 0.0.0.0
    ports: ["8545:8545"]
```

56.2 Production Topology

A production deployment SHOULD use at least:

- three relay nodes across independent regions,
- three log nodes with replicated storage,
- one checkpoint submitter per settlement backend,
- at least two auditor nodes run by independent operators,
- read-only verifier replicas,
- offline signing keys or HSM-backed keys for relay checkpoint signatures,
- immutable object storage for closed log segments.

56.3 Environment Variables

```
TSL_NODE_ROLE=relay|log|resolver|verifier|auditor|checkpoint_submitter
TSL_NETWORK=devnet|testnet|mainnet
TSL_RELAY_ID=did:tsl:relay:...
TSL_RELAY_PRIVATE_KEY_URI=env|file|kms|hsm
TSL_DATABASE_URL=postgres://...
TSL_QUEUE_URL=nats://...|redis://...|kafka://...
TSL_EPOCH_MS=300000
TSL_SHARD_PREFIX_BITS=16
TSL_SETTLEMENT_BACKEND=eip155:8453
TSL_SETTLEMENT_RPC_URL=https://...
TSL_REQUIRE_SETTLEMENT=true
TSL_LOG_SEGMENT_BUCKET=s3://...
```

57 Test Vectors and Compliance

A compliant implementation MUST pass canonicalization, hashing, signature, Merkle, and verification test vectors.

57.1 Canonicalization Test

These two objects MUST canonicalize to identical bytes:

```
{"b":2,"a":1}
{
  "a": 1,
  "b": 2
}
```

Expected canonical form:

```
{"a":1,"b":2}
```

57.2 Deterministic Event Test Vector

The following vector fixes a seed, payload, canonical form, signature, commitment, and single-leaf Merkle root. Implementations MUST reproduce these values when using the reference suite.

```
private_key_seed_hex:
000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f

public_key_hex:
03a107bff3ce10be1d70dd18e74bc09967e4d6309ba50d5f1ddc8664125531b8

content_salt_hex:
1111111111111111111111111111111111111111111111111111111111111111

content_message_utf8:
hello-tsl

content_commitment_hex:
0x50f62d6063e0d92d02fd4fdafb4b10c38bf271ebb5e14afd037d35b0b35f6b95
```

Canonical unsigned event payload:

```
{"content_commitment":"0x50f62d6063e0d92d02fd4fdafb4b10c38bf271ebb5e14afd037d35b0b35f6b95","
    disclosure_policy":"commitment_only","event_class":"message","nonce":"0
    x2222222222222222222222222222222222222222222222222222222222222222","sender":"did:tsl:test:
    alice","signing_key_id":"#test-key-1","timestamp":"2026-05-25T00:01:00Z","type":"tsl.
    event_commitment.v1"}
```

Expected hashes and signature:

```
event_hash_hex:
0xcf5cb36e4596ed4c446f2d24504407369a1fc4862928e86c340ec5270fcc3267

signature_hex:
0xd3187ac9861b87a3b5f871c9ae9a6426ce0c1e49cee1978c767bf99eff6c94467
b6955cd9821c2a7e3bfcf945b576e49d81deccb4e7c8b0624917fd794f1ff08

commitment_hash_hex:
0x174c377613f1fa94acc95d32408095c27330f5dfa088ee40cdcb81a503b25bb5

single_leaf_merkle_root_hex:
0xc09632a2beaaf0c4702e673a7a1661673c80be478f1136b60677f38c5bb5914f
```

57.3 End-to-End Test Scenario

1. Generate identity from deterministic test seed.
2. Create event commitment with fixed nonce and timestamp.
3. Canonicalize event without signature.
4. Sign event hash.
5. Compute commitment hash.
6. Append to Merkle tree.
7. Generate inclusion proof.
8. Build checkpoint.

9. Verify signature, key state, inclusion proof, checkpoint, and revocation state.

57.4 Compliance Matrix

Requirement	Test
Canonical serialization	Same object with reordered keys produces same bytes.
Signature verification	Known vector signature verifies; altered field fails.
Replay protection	Duplicate sender/key/nonce is rejected.
Revocation	Event after effective revocation fails.
Merkle proof	Inclusion proof verifies against root; altered leaf fails.
Checkpoint conflict	Two roots for same epoch-shard trigger audit finding.
Privacy	Raw message absent from public commitment and checkpoint tables.
Token independence	Event validity tests pass with no fee policy or token module.

58 Implementation Milestones

58.1 Milestone 1: Core Protocol Library

Deliverables:

- JSON Schemas for all v1 objects.
- Canonicalization library.
- Hash and signature utilities.
- TrustID generation and resolution abstraction.
- Event, receipt, attestation, and revocation builders.
- Verification library with test vectors.

Acceptance criteria:

- Unit tests pass across TypeScript and Rust.
- Test vectors are identical across languages.
- Invalid signatures, revoked keys, and tampered commitments fail deterministically.

58.2 Milestone 2: Relay and Log Node

Deliverables:

- Commitment intake API.
- Validation pipeline.
- Durable queue.
- Sharded append-only Merkle log.
- Inclusion proof generation.
- Checkpoint builder.

Acceptance criteria:

- 1 million test commitments can be appended and proven on a single development cluster.
- Inclusion proof verification succeeds from an offline verifier.
- Duplicate nonce replay is rejected.

58.3 Milestone 3: Settlement Contracts

Deliverables:

- TrustID registry.
- Revocation registry.
- Checkpoint registry.
- Provider registry.
- Foundry or Hardhat tests.

Acceptance criteria:

- Checkpoints can be submitted and queried.
- Conflicting checkpoints are rejected.
- Revoked keys fail verifier tests.
- No token contract is required for core verification.

58.4 Milestone 4: Verifier and Proof Link

Deliverables:

- CLI verifier.
- Web reference verifier.
- Proof link format.
- Browser extension proof detection.

Acceptance criteria:

- A proof generated by one client verifies in another client.
- Verification works if hosted API is offline but proof data is available.
- Disclosed message content matches content commitment.

58.5 Milestone 5: Optional Scoring Provider

Deliverables:

- Reference feature extractor.
- Transparent weighted scoring model.
- Signed trust assessment object.
- Explanation generator.

Acceptance criteria:

- Scoring output is signed and expires.
- Verifier can display score but does not require it for cryptographic validity.
- User can choose not to request a score.

59 Token Compatibility Without Token Dependency

TSL is token-compatible but not token-dependent. The core protocol MUST work without a native token. A future token MAY coordinate relay staking, auditor rewards, provider bonds, proof-generation fees, grants, and governance. Token ownership MUST NOT be interpreted as trustworthiness.

Layer	Token Relationship
Core trust layer	No token dependency. Events, receipts, revocations, proofs, and verification remain valid without token logic.
Settlement layer	May use existing chain gas or sponsored submission.
Economic layer	May later add fiat, credits, stablecoins, or native token fees.
Governance layer	May later add token voting, but protocol safety should retain security-council emergency controls.

A future token module must be added as a separate adapter:

```
export interface EconomicsAdapter {
  quote(action: MeteredAction): Promise<Quote>;
  authorize(actor: TrustID, action: MeteredAction, quote: Quote): Promise<Authorization>;
  settle(authorization: Authorization): Promise<UsageReceipt>;
}
```

The verifier MUST NOT check token balances when determining cryptographic validity.

60 The Founder Narrative

The story should be told simply:

> The internet used to trust accounts. Then it trusted platforms. Now AI can fake both the content and the account layer. The only thing that remains expensive to fake is continuity: a long-running, cryptographically signed, reciprocally witnessed trajectory through a real graph.
>
> TSL is building Proof of Continuity: a trust settlement layer for humans, businesses, wallets, software, and AI agents.
>
> Applications will not own trust. They will carry it.

61 Appendix A: Formal Cryptographic Sketch

Let m be a raw message, μ be private metadata, r be the receiver identity, and n be a nonce. Let canon be canonical serialization.

$$c_m = \mathrm{Hash}(\mathrm{canon}(m) \parallel s_m), \tag{115}$$
$$c_\mu = \mathrm{Hash}(\mathrm{canon}(\mu) \parallel s_\mu), \tag{116}$$
$$c_r = \mathrm{Hash}(\mathrm{canon}(r) \parallel s_r). \tag{117}$$

An event payload is:

$$e_t = (\mathrm{sid}_s, c_r, c_m, c_\mu, h_{t-1}, \tau, n, \rho), \tag{118}$$

where ρ is the disclosure policy.

The event hash is:

$$h_t = \mathrm{Hash}(\mathrm{canon}(e_t)). \tag{119}$$

The sender signature is:

$$\sigma_t = \mathrm{Sign}_{sk_s}(h_t). \tag{120}$$

Verification checks:

$$\mathrm{Verify}_{pk_s}(h_t, \sigma_t) = 1. \tag{121}$$

The submitted commitment is:

$$C_t = \mathrm{Hash}(h_t \parallel \sigma_t). \tag{122}$$

A batch root is:

$$R_E = \mathrm{MerkleRoot}(C_1, C_2, \ldots, C_N). \tag{123}$$

A verifier checks inclusion proof π_i:

$$\mathrm{VerifyMerkle}(C_i, R_E, \pi_i) = 1. \tag{124}$$

The final verification result is:

$$V = \mathrm{Verify}_{pk_s}(h_t, \sigma_t) \wedge \mathrm{VerifyMerkle}(C_t, R_E, \pi_t) \wedge \mathrm{KeyActive}(\mathrm{sid}_s, pk_s, \tau) \wedge \mathrm{CheckpointSettled}(R_E). \tag{125}$$

62 Appendix B: Standards Alignment

TSL should align with existing standards where useful without depending on any one standard as a bottleneck.

- **DIDs:** TrustID resolution can map to DID documents, verification methods, and service endpoints. See W3C DID Core [1].
- **Verifiable Credentials:** Trust assessments, organization membership, and professional attestations can be represented as verifiable credentials. See W3C VC Data Model 2.0 [2].
- **Account abstraction:** Smart accounts can support recovery, delegated signing, paymasters, and scoped agent keys. See ERC-4337 [3].
- **Transparency logs:** Append-only Merkle logs and consistency proofs can borrow concepts from Certificate Transparency. See RFC 9162 [4].
- **Layer-2 settlement:** High-volume commitments should be batched and settled through L2s or app-rollups rather than written one-by-one to L1. See Ethereum scaling documentation [5].

63 Appendix C: Default Product Copy

63.1 Website Hero

Proof of Continuity for the AI Internet.
TSL lets humans, businesses, wallets, and AI agents prove they are the same trustworthy actor over time, across any app or transport, without exposing private messages.

63.2 Investor One-Liner

We are building the trust-envelope layer for online communication: every message, transaction, API call, and agent action can carry portable cryptographic proof of continuity.

63.3 Developer One-Liner

Add `tsl.verify()` to any app and get signature validation, key status, inclusion proofs, revocation checks, receipts, and explainable trust assessments.

63.4 Consumer One-Liner

Know whether the person, business, wallet, or AI agent contacting you has real continuity, not just a convincing profile.

64 Implementation Completeness Addendum: Buildable Product Specification

This addendum is normative for implementation planning. The preceding sections define the research architecture, protocol objects, privacy model, trust-scoring science, graph geometry, threat model, and

product thesis. This section defines when those ideas become buildable, testable, versioned, auditable, and release-candidate-compliant.

Architecture-to-Implementation Bridge

A concept in this specification is not implementation-complete until it has: (1) a machine-readable schema, (2) deterministic validation rules, (3) a canonicalization rule, (4) at least one valid example, (5) at least three invalid examples, (6) a test vector, and (7) a migration rule for future versions.

64.1 Scope of This Addendum

The goal of this addendum is not to add more theory. It converts the hyper-specific research architecture into the artifacts engineers need to build the product.

Area	Required Output
Schemas	JSON Schema files for every normative object family.
Algorithms	Deterministic pseudocode for feature extraction, scoring, calibration, graph construction, drift, Sybil assessment, leakage scoring, and agent authorization.
Test vectors	Fixed inputs and expected outputs for every object, algorithm, and failure mode.
Versioning	Coexistence rules for v1 and v2 objects, unknown versions, deprecation, and migration.
UX	Required verifier states, warning semantics, consent prompts, appeal flows, and agent review screens.
Operations	Provider onboarding, auditor onboarding, SLOs, incident escalation, key ceremonies, monitoring, and runbooks.
Governance	Open-source scorer release process, model cards, benchmark gates, community review, and alternative-provider compatibility.
Conformance	Release-candidate levels so the product can ship staged subsets without pretending to implement the entire research roadmap.

64.2 Normative Consistency Patch v4

This version resolves implementation ambiguities introduced by the broader v2 architecture. The following rules are now normative:

1. Signed and hashed protocol examples MUST NOT contain JSON floating-point numbers.
2. Basis-point fields use the suffix `_bps`; milli-count fields use `_milli`; minor currency units use `_minor_units`.
3. `delegation_policy.v2` uses `policy_id`; `agent_action.v2` uses `issued_at` and `parameters_commitment`.
4. `sybil_assessment.v1` uses `risk_score_bps` and `risk_label`; `drift_report.v1` uses `drift_score_bps`, `drift_label`, and `action`.
5. Portable verification is anchored by `proof_bundle.v1`.

6. API and verifier errors MUST use the canonical error-code registry.
7. Research equations are non-canonical unless translated into deterministic fixed-point algorithms and test vectors.

64.3 Spec-to-Artifact Traceability Matrix

Every normative object introduced in this specification MUST map to a schema file, a validator, at least one valid example, invalid examples, and a deterministic test vector before it can be considered implementation-complete.

```
# Each object below requires:
#   1. specs/json-schema/<schema>
#   2. a validator in core-ts and core-rust
#   3. examples/valid/<object>.json
#   4. examples/invalid/<object>/*.json
#   5. test-vectors/<object>/<case>.json

tsl.identity.v1
  schema: identity.v1.schema.json
  vector: identity.v1/valid-local.json

tsl.event_commitment.v1
  schema: event_commitment.v1.schema.json
  vector: event_commitment.v1/deterministic-message.json

tsl.receipt_commitment.v1
  schema: receipt_commitment.v1.schema.json
  vector: receipt_commitment.v1/replied.json

tsl.revocation.v1
  schema: revocation.v1.schema.json
  vector: revocation.v1/compromise.json

tsl.batch_checkpoint.v1
  schema: batch_checkpoint.v1.schema.json
  vector: batch_checkpoint.v1/single-leaf.json

tsl.inclusion_proof.v1
  schema: inclusion_proof.v1.schema.json
  vector: inclusion_proof.v1/single-leaf.json

tsl.scoring_profile.v2
  schema: scoring_profile.v2.schema.json
  vector: scoring_profile.v2/valid-transparent.json

tsl.feature_definition.v2
  schema: feature_definition.v2.schema.json
  vector: feature_definition.v2/identity-age.json

tsl.domain_policy.v1
  schema: domain_policy.v1.schema.json
  vector: domain_policy.v1/agent-payments.json

tsl.evidence_coverage.v1
```

```
  schema: evidence_coverage.v1.schema.json
  vector: evidence_coverage.v1/high-coverage.json

tsl.trust_assessment.v2
  schema: trust_assessment.v2.schema.json
  vector: trust_assessment.v2/likely-trusted.json

tsl.metadata_fingerprint_commitment.v1
  schema: metadata_fingerprint_commitment.v1.schema.json
  vector: metadata_fingerprint_commitment.v1/pairwise.json

tsl.graph_profile.v2
  schema: graph_profile.v2.schema.json
  vector: graph_profile.v2/default.json

tsl.graph_feature_vector.v1
  schema: graph_feature_vector.v1.schema.json
  vector: graph_feature_vector.v1/small-graph.json

tsl.sybil_assessment.v1
  schema: sybil_assessment.v1.schema.json
  vector: sybil_assessment.v1/cluster-b2.json

tsl.drift_report.v1
  schema: drift_report.v1.schema.json
  vector: drift_report.v1/dormant-reactivation.json

tsl.attestation.v2
  schema: attestation.v2.schema.json
  vector: attestation.v2/appealed-negative.json

tsl.model_card.v2
  schema: model_card.v2.schema.json
  vector: model_card.v2/reference-scorer.json

tsl.evaluation_report.v1
  schema: evaluation_report.v1.schema.json
  vector: evaluation_report.v1/promotion-pass.json

tsl.delegation_policy.v2
  schema: delegation_policy.v2.schema.json
  vector: delegation_policy.v2/invoice-agent.json

tsl.agent_action.v2
  schema: agent_action.v2.schema.json
  vector: agent_action.v2/inside-scope.json
```

64.4 Mandatory Artifact Tree

The repository MUST be structured so the LaTeX specification, schemas, examples, tests, algorithms, and conformance levels remain synchronized.

```
tsl/
  specs/
```

```
  latex/
    Trust_Signature_Layer_full_implementation_v3.tex
  json-schema/
    identity.v1.schema.json
    event_commitment.v1.schema.json
    receipt_commitment.v1.schema.json
    revocation.v1.schema.json
    batch_checkpoint.v1.schema.json
    inclusion_proof.v1.schema.json
    proof_bundle.v1.schema.json
    scoring_profile.v2.schema.json
    feature_definition.v2.schema.json
    domain_policy.v1.schema.json
    evidence_coverage.v1.schema.json
    trust_assessment.v2.schema.json
    metadata_fingerprint_commitment.v1.schema.json
    graph_profile.v2.schema.json
    graph_feature_vector.v1.schema.json
    sybil_assessment.v1.schema.json
    drift_report.v1.schema.json
    attestation.v2.schema.json
    model_card.v2.schema.json
    evaluation_report.v1.schema.json
    delegation_policy.v2.schema.json
    agent_action.v2.schema.json
  openapi/
    relay-api.v1.yaml
    verifier-api.v1.yaml
    resolver-api.v1.yaml
    scoring-provider-api.v1.yaml
    auditor-api.v1.yaml
  examples/
    valid/
    invalid/
  test-vectors/
    canonicalization/
    signatures/
    merkle/
    verifier/
    scoring-v0/
    graph-v0/
    sybil-v0/
    drift-v0/
    metadata-fingerprints-v0/
    delegation-v0/
algorithms/
  reference-scorer-v0.md
  graph-construction-v0.md
  drift-baseline-v0.md
  sybil-simulation-v0.md
  calibration-v0.md
  leakage-score-v0.md
  delegation-authorization-v0.md
conformance/
  tsl-rc0.md
  tsl-rc1.md
```

```
    tsl-rc2.md
    tsl-rc3.md
    tsl-rc4.md
    tsl-mainnet.md
```

64.5 Machine-Readable Schema Requirements

A v2 object defined only in prose, equations, or illustrative JSON is not implementation-complete. Each normative object schema MUST include:

1. a stable `$id`;
2. a fixed `type` discriminator;
3. explicit required fields;
4. `additionalProperties: false` unless a declared extension namespace is present;
5. canonical integer units for scores, durations, timestamps, costs, percentages, probabilities, and basis-point values;
6. no floating-point values in signed payloads;
7. explicit enum values for labels, domains, object states, failure codes, claim classes, and governance statuses;
8. a deterministic canonicalization rule before hashing or signing;
9. at least one valid example;
10. at least three invalid examples: missing required field, malformed signature/hash, and unknown non-extension field;
11. a migration note describing how this object interacts with prior versions.

Signed Payload Rule

Signed payloads MUST NOT contain floating-point values. Probabilities, scores, rates, and weights MUST be encoded as integers in basis points, parts per million, or explicitly specified fixed-point units.

64.6 Schema Skeletons for New v2 Object Families

The following schema skeletons are normative patterns. The repository schemas MAY add narrower constraints, but MUST NOT weaken these requirements.

64.6.1 Schema Pattern: `tsl.proof_bundle.v1`

```
{
  "$id": "https://spec.tsl.network/schemas/proof_bundle.v1.schema.json",
  "type": "object",
  "required": ["type", "bundle_id", "created_at", "envelope", "proof", "checkpoint", "
    identity", "redaction_manifest"],
  "additionalProperties": false,
  "properties": {
    "type": { "const": "tsl.proof_bundle.v1" },
    "bundle_id": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
```

```
    "created_at": { "type": "string", "format": "date-time" },
    "envelope": { "type": "object" },
    "proof": { "type": "object" },
    "checkpoint": { "type": "object" },
    "identity": { "type": "object" },
    "receipts": { "type": "array", "items": { "type": "object" } },
    "attestations": { "type": "array", "items": { "type": "object" } },
    "revocations": { "type": "array", "items": { "type": "object" } },
    "assessment": { "type": ["object", "null"] },
    "assessment_v2": { "type": ["object", "null"] },
    "zk_proofs": { "type": "array", "items": { "type": "object" } },
    "delegations": { "type": "array", "items": { "type": "object" } },
    "audit_findings": { "type": "array", "items": { "type": "object" } },
    "governance_policy": { "type": "object" },
    "redaction_manifest": {
      "type": "object",
      "required": ["raw_content_included", "exact_counterparties_included", "
   metadata_fields_redacted"],
      "additionalProperties": false,
      "properties": {
        "raw_content_included": { "type": "boolean" },
        "exact_counterparties_included": { "type": "boolean" },
        "metadata_fields_redacted": { "type": "array", "items": { "type": "string" } }
      }
    }
  }
}
```

64.6.2 Schema Pattern: `tsl.scoring_profile.v2`

```
{
  "$id": "https://spec.tsl.network/schemas/scoring_profile.v2.schema.json",
  "type": "object",
  "required": [
    "type",
    "profile_id",
    "provider",
    "domain",
    "model_family",
    "model_version",
    "feature_registry_commitment",
    "normalization_profile_commitment",
    "weight_profile_commitment",
    "calibration_profile_commitment",
    "threshold_policy_commitment",
    "privacy_policy_commitment",
    "evaluation_report_commitment",
    "issued_at",
    "valid_after",
    "expires_at",
    "signature"
  ],
  "additionalProperties": false,
  "properties": {
```

```
    "type": { "const": "tsl.scoring_profile.v2" },
    "profile_id": { "type": "string", "minLength": 1 },
    "provider": { "type": "string", "pattern": "^did:tsl:" },
    "domain": {
      "enum": [
        "anti_phishing",
        "marketplace_trust",
        "agent_delegation",
        "opensource_maintainer",
        "professional_identity",
        "customer_support",
        "local_verifier"
      ]
    },
    "model_family": {
      "enum": [
        "transparent_weighted_logistic",
        "calibrated_tree_ensemble",
        "graph_risk_model",
        "local_rule_policy",
        "human_review_hybrid"
      ]
    },
    "model_version": { "type": "string", "pattern": "^[0-9]+\\.[0-9]+\\.[0-9]+" },
    "feature_registry_uri": { "type": "string" },
    "feature_registry_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "normalization_profile_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "weight_profile_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "calibration_profile_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "threshold_policy_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "privacy_policy_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "evaluation_report_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "appeal_policy_uri": { "type": "string" },
    "issued_at": { "type": "string", "format": "date-time" },
    "valid_after": { "type": "string", "format": "date-time" },
    "expires_at": { "type": "string", "format": "date-time" },
    "signature": { "type": "string", "pattern": "^0x[0-9a-f]+$" }
  }
}
```

64.6.3 Schema Pattern: `tsl.feature_definition.v2`

```
{
  "$id": "https://spec.tsl.network/schemas/feature_definition.v2.schema.json",
  "type": "object",
  "required": [
    "type",
    "feature_id",
    "name",
    "family",
    "value_type",
    "unit",
    "range",
    "normalization_method",
```

```
    "privacy_class",
    "spoofing_cost_class",
    "missing_value_policy",
    "attack_notes"
  ],
  "additionalProperties": false,
  "properties": {
    "type": { "const": "tsl.feature_definition.v2" },
    "feature_id": { "type": "string" },
    "name": { "type": "string" },
    "family": {
      "enum": [
        "cryptographic",
        "temporal",
        "graph",
        "receipts",
        "attestations",
        "behavioral_drift",
        "adversarial_proximity",
        "local_context",
        "delegation"
      ]
    },
    "value_type": { "enum": ["boolean", "integer", "bps", "count", "duration_ms", "enum"] },
    "unit": { "type": "string" },
    "range": {
      "type": "object",
      "required": ["min", "max"],
      "properties": {
        "min": { "type": "integer" },
        "max": { "type": "integer" }
      }
    },
    "normalization_method": {
      "enum": ["identity", "log1p_cap", "winsorized_z", "percentile_rank", "piecewise_linear",
     "binary"]
    },
    "privacy_class": { "enum": ["public", "commitment", "aggregate", "pairwise", "local_only",
     "forbidden"] },
    "spoofing_cost_class": { "enum": ["low", "medium", "high", "very_high"] },
    "missing_value_policy": { "enum": ["zero", "neutral", "abstain", "domain_default", "
    provider_default"] },
    "attack_notes": { "type": "array", "items": { "type": "string" } }
  }
}
```

64.6.4 Schema Pattern: `tsl.domain_policy.v1`

```
{
  "$id": "https://spec.tsl.network/schemas/domain_policy.v1.schema.json",
  "type": "object",
  "required": [
    "type",
    "domain",
```

```
    "policy_version",
    "requires_settlement",
    "min_coverage_bps",
    "max_assessment_age_seconds",
    "false_positive_cost_class",
    "false_negative_cost_class",
    "sparse_identity_default",
    "thresholds"
  ],
  "additionalProperties": false,
  "properties": {
    "type": { "const": "tsl.domain_policy.v1" },
    "domain": { "type": "string" },
    "policy_version": { "type": "string" },
    "requires_settlement": { "type": "boolean" },
    "requires_delegation_check": { "type": "boolean" },
    "requires_content_opening": { "type": "boolean" },
    "min_coverage_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
    "max_assessment_age_seconds": { "type": "integer", "minimum": 1 },
    "false_positive_cost_class": { "enum": ["low", "medium", "high", "critical"] },
    "false_negative_cost_class": { "enum": ["low", "medium", "high", "critical"] },
    "sparse_identity_default": { "enum": ["unknown", "unknown_caution", "require_step_up", "
    deny_high_value_action"] },
    "thresholds": {
      "type": "object",
      "required": ["trusted_bps", "likely_trusted_bps", "medium_bps", "suspicious_bps", "
    high_risk_bps"],
      "additionalProperties": false,
      "properties": {
        "trusted_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
        "likely_trusted_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
        "medium_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
        "suspicious_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
        "high_risk_bps": { "type": "integer", "minimum": 0, "maximum": 10000 }
      }
    }
  }
}
```

64.6.5 Schema Pattern: `tsl.trust_assessment.v2`

```
{
  "$id": "https://spec.tsl.network/schemas/trust_assessment.v2.schema.json",
  "type": "object",
  "required": [
    "type",
    "assessment_id",
    "subject",
    "issuer",
    "domain",
    "scoring_profile_id",
    "model_version",
    "gate_result",
    "label",
```

```
    "coverage_bps",
    "reason_codes",
    "risk_codes",
    "issued_at",
    "expires_at",
    "signature"
  ],
  "additionalProperties": false,
  "properties": {
    "type": { "const": "tsl.trust_assessment.v2" },
    "assessment_id": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "subject": { "type": "string", "pattern": "^did:tsl:" },
    "issuer": { "type": "string", "pattern": "^did:tsl:" },
    "domain": { "type": "string" },
    "scoring_profile_id": { "type": "string" },
    "model_version": { "type": "string" },
    "gate_result": {
      "type": "object",
      "required": ["schema_valid", "signature_valid", "key_active", "not_revoked"],
      "additionalProperties": false,
      "properties": {
        "schema_valid": { "type": "boolean" },
        "canonicalization_valid": { "type": "boolean" },
        "signature_valid": { "type": "boolean" },
        "key_active": { "type": "boolean" },
        "not_revoked": { "type": "boolean" },
        "included_in_log": { "type": "boolean" },
        "checkpoint_valid": { "type": "boolean" },
        "settlement_satisfied": { "type": "boolean" },
        "delegation_valid": { "type": "boolean" }
      }
    },
    "score_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
    "confidence_interval_bps": {
      "type": "array",
      "minItems": 2,
      "maxItems": 2,
      "items": { "type": "integer", "minimum": 0, "maximum": 10000 }
    },
    "coverage_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
    "label": {
      "enum": [
        "trusted",
        "likely_trusted",
        "medium_trust",
        "unknown_caution",
        "insufficient_evidence",
        "suspicious",
        "high_risk",
        "cryptographic_failure",
        "revoked_or_compromised",
        "settlement_missing"
      ]
    },
    "reason_codes": { "type": "array", "items": { "type": "string" } },
    "risk_codes": { "type": "array", "items": { "type": "string" } },
```

```
    "feature_vector_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "evidence_coverage_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "privacy_disclosure_level": { "enum": ["none", "aggregate_only", "pairwise", "selective",
     "public"] },
    "appeal_uri": { "type": "string" },
    "issued_at": { "type": "string", "format": "date-time" },
    "expires_at": { "type": "string", "format": "date-time" },
    "signature": { "type": "string", "pattern": "^0x[0-9a-f]+$" }
  }
}
```

64.6.6 Schema Pattern: `tsl.metadata_fingerprint_commitment.v1`

```
{
  "$id": "https://spec.tsl.network/schemas/metadata_fingerprint_commitment.v1.schema.json",
  "type": "object",
  "required": [
    "type",
    "subject",
    "scope_class",
    "scope_commitment",
    "bucket_profile",
    "fingerprint_commitment",
    "created_at_bucket",
    "expires_at",
    "disclosure_policy",
    "signature"
  ],
  "additionalProperties": false,
  "properties": {
    "type": { "const": "tsl.metadata_fingerprint_commitment.v1" },
    "subject": { "type": "string", "pattern": "^did:tsl:" },
    "scope_class": { "enum": ["local_only", "pairwise_verifier", "provider_ephemeral", "
    public_commitment"] },
    "scope_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "bucket_profile": { "type": "string" },
    "fingerprint_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "salt_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "created_at_bucket": { "type": "string" },
    "expires_at": { "type": "string", "format": "date-time" },
    "disclosure_policy": { "enum": ["local_only", "selective", "zk_only", "
    public_commitment_only"] },
    "signature": { "type": "string", "pattern": "^0x[0-9a-f]+$" }
  }
}
```

64.6.7 Schema Pattern: `tsl.graph_profile.v2` and `tsl.graph_feature_vector.v1`

```
{
  "$id": "https://spec.tsl.network/schemas/graph_profile.v2.schema.json",
  "type": "object",
  "required": [
```

```
    "type",
    "profile_id",
    "edge_weight_profile",
    "temporal_decay_profile",
    "community_detection",
    "seed_sets",
    "negative_edge_policy",
    "privacy_policy"
  ],
  "additionalProperties": false,
  "properties": {
    "type": { "const": "tsl.graph_profile.v2" },
    "profile_id": { "type": "string" },
    "edge_weight_profile": { "type": "string" },
    "temporal_decay_profile": { "type": "string" },
    "community_detection": {
      "type": "object",
      "required": ["algorithm", "resolution_bps", "min_cluster_size"],
      "additionalProperties": false,
      "properties": {
        "algorithm": { "enum": ["leiden", "louvain", "connected_components", "none"] },
        "resolution_bps": { "type": "integer", "minimum": 1 },
        "min_cluster_size": { "type": "integer", "minimum": 1 },
        "edge_weight_floor_bps": { "type": "integer", "minimum": 0, "maximum": 10000 }
      }
    },
    "seed_sets": { "type": "object" },
    "negative_edge_policy": { "type": "object" },
    "privacy_policy": { "type": "object" }
  }
}

{
  "$id": "https://spec.tsl.network/schemas/graph_feature_vector.v1.schema.json",
  "type": "object",
  "required": [
    "type",
    "subject",
    "graph_profile_id",
    "computed_at",
    "weighted_degree_bps",
    "reciprocity_bps",
    "counterparty_hhi_bps",
    "counterparty_entropy_bps",
    "effective_counterparty_count_milli",
    "seed_escape_bps",
    "adversarial_proximity_bps",
    "privacy_disclosure_level"
  ],
  "additionalProperties": false,
  "properties": {
    "type": { "const": "tsl.graph_feature_vector.v1" },
    "subject": { "type": "string", "pattern": "^did:tsl:" },
    "graph_profile_id": { "type": "string" },
    "computed_at": { "type": "string", "format": "date-time" },
    "weighted_degree_bps": { "type": "integer", "minimum": 0 },
```

```
    "reciprocity_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
    "counterparty_hhi_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
    "counterparty_entropy_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
    "effective_counterparty_count_milli": { "type": "integer", "minimum": 0 },
    "seed_escape_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
    "adversarial_proximity_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
    "privacy_disclosure_level": { "enum": ["aggregate_only", "pairwise", "local_only", "
    public"] }
  }
}
```

64.6.8 Schema Pattern: `tsl.sybil_assessment.v1` and `tsl.drift_report.v1`

```
{
  "$id": "https://spec.tsl.network/schemas/sybil_assessment.v1.schema.json",
  "type": "object",
  "required": [
    "type",
    "subject",
    "cluster_id_commitment",
    "computed_at",
    "adversary_tier_assumed",
    "cluster_concentration_bps",
    "trusted_escape_bps",
    "internal_receipt_ratio_bps",
    "attack_cost_minor_units",
    "risk_score_bps",
    "risk_label",
    "privacy_level",
    "signature"
  ],
  "additionalProperties": false,
  "properties": {
    "type": { "const": "tsl.sybil_assessment.v1" },
    "subject": { "type": "string", "pattern": "^did:tsl:" },
    "cluster_id_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "computed_at": { "type": "string", "format": "date-time" },
    "adversary_tier_assumed": { "enum": ["B0", "B1", "B2", "B3", "B4", "B5"] },
    "cluster_size_bucket": { "type": "string" },
    "cluster_concentration_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
    "trusted_escape_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
    "internal_receipt_ratio_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
    "creation_sync_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
    "issuer_reuse_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
    "external_diversity_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
    "attack_cost_minor_units": { "type": "integer", "minimum": 0 },
    "risk_score_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
    "risk_label": { "enum": ["low", "medium", "elevated", "high", "insufficient_evidence"] },
    "privacy_level": { "enum": ["cluster_commitment_only", "aggregate_only", "local_only"] },
    "signature": { "type": "string", "pattern": "^0x[0-9a-f]+$" }
  }
}

{
```

```
  "$id": "https://spec.tsl.network/schemas/drift_report.v1.schema.json",
  "type": "object",
  "required": [
    "type",
    "subject",
    "computed_at",
    "baseline_window_days",
    "observation_window_days",
    "drift_score_bps",
    "drift_label",
    "action",
    "reason_codes",
    "signature"
  ],
  "additionalProperties": false,
  "properties": {
    "type": { "const": "tsl.drift_report.v1" },
    "subject": { "type": "string", "pattern": "^did:tsl:" },
    "computed_at": { "type": "string", "format": "date-time" },
    "baseline_window_days": { "type": "integer", "minimum": 1 },
    "observation_window_days": { "type": "integer", "minimum": 1 },
    "drift_score_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
    "drift_label": { "enum": ["stable", "minor", "moderate", "severe", "dormant_reactivation",
     "insufficient_baseline"] },
    "action": { "enum": ["none", "lower_confidence", "step_up", "human_review", "
    temporary_block"] },
    "reason_codes": { "type": "array", "items": { "type": "string" } },
    "signature": { "type": "string", "pattern": "^0x[0-9a-f]+$" }
  }
}
```

64.6.9 Schema Pattern: Governance, Evaluation, and Agent Objects

```
{
  "$id": "https://spec.tsl.network/schemas/model_card.v2.schema.json",
  "type": "object",
  "required": [
    "type", "model_id", "provider", "model_version", "supported_domains",
    "feature_registry_commitment", "evaluation_report_commitment",
    "privacy_report_commitment", "metrics", "limitations", "issued_at", "signature"
  ],
  "additionalProperties": false,
  "properties": {
    "type": { "const": "tsl.model_card.v2" },
    "model_id": { "type": "string" },
    "provider": { "type": "string", "pattern": "^did:tsl:" },
    "model_version": { "type": "string" },
    "supported_domains": { "type": "array", "items": { "type": "string" } },
    "feature_registry_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "evaluation_report_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "privacy_report_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "metrics": {
      "type": "object",
      "required": ["auroc_bps", "auprc_bps", "ece_bps", "p95_latency_ms"],
```

```
      "additionalProperties": false,
      "properties": {
        "auroc_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
        "auprc_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
        "ece_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
        "sparse_identity_false_positive_rate_bps": { "type": "integer", "minimum": 0, "
    maximum": 10000 },
        "p95_latency_ms": { "type": "integer", "minimum": 0 },
        "appeal_reversal_rate_bps": { "type": "integer", "minimum": 0, "maximum": 10000 }
      }
    },
    "limitations": { "type": "array", "items": { "type": "string" } },
    "issued_at": { "type": "string", "format": "date-time" },
    "signature": { "type": "string", "pattern": "^0x[0-9a-f]+$" }
  }
}

{
  "$id": "https://spec.tsl.network/schemas/evaluation_report.v1.schema.json",
  "type": "object",
  "required": [
    "type", "report_id", "model_id", "domain", "dataset_commitments",
    "metrics", "promotion_gate_result", "red_team_result", "issued_at", "signature"
  ],
  "additionalProperties": false,
  "properties": {
    "type": { "const": "tsl.evaluation_report.v1" },
    "report_id": { "type": "string" },
    "model_id": { "type": "string" },
    "domain": { "type": "string" },
    "dataset_commitments": { "type": "array", "items": { "type": "string", "pattern": "^0x
    [0-9a-f]{64}$" } },
    "metrics": {
      "type": "object",
      "required": ["auroc_bps", "auprc_bps", "ece_bps", "coverage_bps", "p95_latency_ms"],
      "properties": {
        "auroc_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
        "auprc_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
        "ece_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
        "coverage_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
        "sparse_identity_fpr_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
        "privacy_leakage_bps": { "type": "integer", "minimum": 0, "maximum": 10000 },
        "p95_latency_ms": { "type": "integer", "minimum": 0 }
      }
    },
    "promotion_gate_result": { "enum": ["pass", "fail", "conditional", "research_only"] },
    "red_team_result": { "enum": ["pass", "fail", "not_run"] },
    "issued_at": { "type": "string", "format": "date-time" },
    "signature": { "type": "string", "pattern": "^0x[0-9a-f]+$" }
  }
}

{
  "$id": "https://spec.tsl.network/schemas/delegation_policy.v2.schema.json",
  "type": "object",
  "required": [
```

```
    "type", "policy_id", "principal", "delegate", "effect", "actions",
    "resources", "constraints", "valid_from", "valid_until", "revocation_pointer", "signature
    "
  ],
  "additionalProperties": false,
  "properties": {
    "type": { "const": "tsl.delegation_policy.v2" },
    "policy_id": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "principal": { "type": "string", "pattern": "^did:tsl:" },
    "delegate": { "type": "string", "pattern": "^did:tsl:" },
    "effect": { "enum": ["allow", "deny"] },
    "actions": { "type": "array", "items": { "type": "string" } },
    "resources": { "type": "array", "items": { "type": "string" } },
    "constraints": { "type": "object" },
    "subdelegation": { "type": "object" },
    "valid_from": { "type": "string", "format": "date-time" },
    "valid_until": { "type": "string", "format": "date-time" },
    "revocation_pointer": { "type": "string" },
    "signature": { "type": "string", "pattern": "^0x[0-9a-f]+$" }
  }
}

{
  "$id": "https://spec.tsl.network/schemas/agent_action.v2.schema.json",
  "type": "object",
  "required": [
    "type", "action_id", "agent", "principal", "action", "resource",
    "parameters_commitment", "delegation_chain_root", "issued_at", "nonce", "signature"
  ],
  "additionalProperties": false,
  "properties": {
    "type": { "const": "tsl.agent_action.v2" },
    "action_id": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "agent": { "type": "string", "pattern": "^did:tsl:" },
    "principal": { "type": "string", "pattern": "^did:tsl:" },
    "action": { "type": "string" },
    "resource": { "type": "string" },
    "tool": { "type": "string" },
    "parameters_commitment": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "parameter_disclosure_policy": { "enum": ["commitment_only", "selective", "zk_only", "
    public"] },
    "delegation_chain_root": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "issued_at": { "type": "string", "format": "date-time" },
    "nonce": { "type": "string", "pattern": "^0x[0-9a-f]{64}$" },
    "signature": { "type": "string", "pattern": "^0x[0-9a-f]+$" }
  }
}
```

64.7 Executable Reference Algorithm Registry

Every reference algorithm MUST be deterministic for the same input set, profile version, and clock value. Any implementation that uses randomized sampling, approximate graph algorithms, or nondeterministic ordering MUST declare the seed, ordering rule, and approximation tolerance in the relevant profile.

```
extractFeatureVectorV0
  Sort evidence by canonical hash; ignore objects failing verification;
  output fixed field order.

normalizeFeatureVectorV0
  Use profile-specified integer fixed-point normalization;
  no implementation-specific floats.

computeReferenceScoreV0
  Integer basis-point arithmetic; explicit rounding mode:
  floor after each weighted term.

selectThresholdsV0
  Apply domain-policy thresholds and adverse-evidence rule exactly.

calibrateScoreV0
  Use signed calibration lookup table or monotonic piecewise-linear mapping.

computeConfidenceIntervalV0
  Use deterministic bootstrap seed from evidence commitment or analytic
  interval profile.

constructGraphV0
  Sort edges by timestamp then canonical hash; discard unverifiable edges.

computeGraphMetricsV0
  Use declared graph profile; report approximation tolerance if used.

computeSybilAssessmentV0
  Use declared adversary tier, cost model, cluster profile, and seed sets.

computeDriftReportV0
  Use fixed windows and robust baseline profile; return insufficient
  baseline when needed.

computeMetadataFingerprintCommitmentV0
  Use scoped KDF, bucket profile, HMAC, salt, and commitment rules exactly.

computeLeakageScoreV0
  Use integer-weighted leakage components and policy thresholds.

verifyDelegatedAgentActionV0
  Validate policy signature, time, revocation, action, resource,
  constraints, and chain intersection.
```

64.7.1 Reference Score v0

```
function computeReferenceScoreV0(input, scoringProfile, domainPolicy, atTime):
    TrustAssessmentV2 {
  gate = verifyHardGates(input, domainPolicy, atTime)

  if gate.schema_valid == false:
    return failureAssessment("cryptographic_failure", "TSL_SCHEMA_INVALID", gate)
  if gate.signature_valid == false:
```

```
    return failureAssessment("cryptographic_failure", "TSL_SIGNATURE_INVALID", gate)
  if gate.key_active == false or gate.not_revoked == false:
    return failureAssessment("revoked_or_compromised", "TSL_KEY_REVOKED", gate)
  if domainPolicy.requires_settlement and gate.settlement_satisfied == false:
    return failureAssessment("settlement_missing", "TSL_SETTLEMENT_MISSING", gate)

  features = extractFeatureVectorV0(input.evidence, scoringProfile.feature_registry, atTime)
  coverage = computeEvidenceCoverageV0(features, input.evidence, atTime)

  if coverage.coverage_bps < domainPolicy.min_coverage_bps and features.has_adverse_evidence
    == false:
    return abstainAssessment("insufficient_evidence", gate, coverage)

  normalized = normalizeFeatureVectorV0(features, scoringProfile.normalization_profile)

  weighted_sum_bps = 0
  for item in scoringProfile.weight_profile.features sorted by item.feature_id:
    value_bps = normalized[item.feature_id]
    weight_bps = item.weight_bps
    weighted_sum_bps += floor(value_bps * weight_bps / 10000)

  calibrated_bps = calibrateScoreV0(weighted_sum_bps, scoringProfile.calibration_profile)
  confidence = computeConfidenceIntervalV0(input.evidence, scoringProfile.confidence_profile)
  label = selectLabelV0(calibrated_bps, confidence, coverage, features, domainPolicy)

  return signTrustAssessmentV2({
    subject: input.subject,
    issuer: scoringProfile.provider,
    domain: domainPolicy.domain,
    scoring_profile_id: scoringProfile.profile_id,
    gate_result: gate,
    score_bps: calibrated_bps,
    confidence_interval_bps: confidence,
    coverage_bps: coverage.coverage_bps,
    label: label,
    reason_codes: reasonCodes(features, normalized),
    risk_codes: riskCodes(features),
    issued_at: atTime,
    expires_at: atTime + domainPolicy.max_assessment_age_seconds
  })
}
```

64.7.2 Graph Construction v0

```
function constructGraphV0(events, receipts, attestations, delegations, graphProfile, atTime):
    Graph {
  G = empty directed typed multigraph

  verifiedEvents = []
  for event in events:
    if verifyEvent(event, atTime).cryptographic_validity == true:
      verifiedEvents.append(event)
  sort verifiedEvents by (event.timestamp, canonicalHash(event))
```

```
  for event in verifiedEvents:
    if event.receiver_disclosed == false:
      continue
    G.addEdge({
      src: event.sender,
      dst: event.receiver,
      type: "signed_event",
      timestamp: event.timestamp,
      base_weight_bps: graphProfile.edge_weights.signed_event_bps,
      evidence_hash: canonicalHash(event)
    })

  verifiedReceipts = filterVerifyAndSort(receipts, atTime)
  for receipt in verifiedReceipts:
    linkedSender = senderOf(receipt.event_commitment)
    G.addEdge({
      src: receipt.receiver,
      dst: linkedSender,
      type: receipt.receipt_class,
      timestamp: receipt.timestamp,
      base_weight_bps: graphProfile.edge_weights[receipt.receipt_class],
      evidence_hash: canonicalHash(receipt)
    })

  verifiedAttestations = filterVerifyAndSort(attestations, atTime)
  for attestation in verifiedAttestations:
    if attestation.expires_at <= atTime:
      continue
    G.addEdge({
      src: attestation.issuer,
      dst: attestation.subject,
      type: attestation.attestation_class,
      timestamp: attestation.issued_at,
      base_weight_bps: issuerAdjustedWeight(attestation, graphProfile),
      evidence_hash: canonicalHash(attestation)
    })

  verifiedDelegations = filterVerifyAndSort(delegations, atTime)
  for delegation in verifiedDelegations:
    if delegation.valid_from <= atTime and atTime < delegation.valid_until and not revoked(
    delegation):
      G.addEdge({
        src: delegation.principal,
        dst: delegation.delegate,
        type: "delegation",
        timestamp: delegation.valid_from,
        base_weight_bps: graphProfile.edge_weights.delegation_bps,
        evidence_hash: canonicalHash(delegation)
      })

  applyTemporalDecay(G, graphProfile.temporal_decay_profile, atTime)
  return G
}
```

64.7.3 Sybil Assessment v0

```
function computeSybilAssessmentV0(subject, graph, graphProfile, sybilProfile, atTime):
    SybilAssessmentV1 {
  cluster = communityOf(subject, graph, graphProfile.community_detection)
  internal_mass = sumWeight(edgesWithin(cluster))
  external_mass = sumWeight(edgesLeaving(cluster))
  trusted_escape_mass = sumWeight(edgesFromClusterToTrustedSeeds(cluster, sybilProfile.
    trusted_seed_set))
  adversarial_seed_mass = sumWeight(edgesFromClusterToAdversarialSeeds(cluster, sybilProfile.
    adversarial_seed_set))

  cluster_concentration_bps = bps(internal_mass, internal_mass + external_mass)
  trusted_escape_bps = bps(trusted_escape_mass, internal_mass + external_mass)
  internal_receipt_ratio_bps = bps(receiptMassWithin(cluster), receiptMassTouching(cluster))
  seed_contamination_bps = bps(adversarial_seed_mass, internal_mass + external_mass)

  attack_cost = estimateAttackCostV0(cluster, sybilProfile.cost_model)

  if evidenceMass(cluster) < sybilProfile.min_evidence_mass:
    risk_label = "insufficient_evidence"
  else if cluster_concentration_bps > 8500 and trusted_escape_bps < 500:
    risk_label = "high"
  else if seed_contamination_bps > 2500:
    risk_label = "elevated"
  else if internal_receipt_ratio_bps > 7500 and trusted_escape_bps < 1500:
    risk_label = "medium"
  else:
    risk_label = "low"

  return signSybilAssessmentV1({subject, cluster, metrics, attack_cost, risk_label, atTime})
}
```

64.7.4 Drift Report v0

```
function computeDriftReportV0(subject, featureHistory, driftProfile, atTime): DriftReportV1 {
  baseline = selectWindow(featureHistory, atTime - driftProfile.baseline_days, atTime -
    driftProfile.observation_days)
  observation = selectWindow(featureHistory, atTime - driftProfile.observation_days, atTime)

  if count(baseline) < driftProfile.min_baseline_points:
    return report("insufficient_baseline", 0, "none")

  baselineMedian = robustMedian(baseline)
  baselineCov = robustCovariance(baseline)
  obsMean = mean(observation)

  drift_bps = robustMahalanobisBps(obsMean, baselineMedian, baselineCov, driftProfile.cap_bps
    )

  dormant = daysSinceLastVerifiedEvent(subject, atTime) >= driftProfile.dormant_days
  highValueChange = observationHasHighValueNewAction(observation)
  newDelegation = observationHasNewDelegationPattern(observation)
  adverse = observationHasAdverseEvidence(observation)
```

```
  if dormant and (highValueChange or newDelegation):
    return report("dormant_reactivation", max(drift_bps, 8000), "step_up")
  if adverse and drift_bps >= driftProfile.severe_bps:
    return report("severe", drift_bps, "human_review")
  if drift_bps >= driftProfile.moderate_bps:
    return report("moderate", drift_bps, "lower_confidence")
  if drift_bps >= driftProfile.minor_bps:
    return report("minor", drift_bps, "none")
  return report("stable", drift_bps, "none")
}
```

64.7.5 Metadata Fingerprint Commitment v0

```
function computeMetadataFingerprintCommitmentV0(metadata, masterKey, context, verifierDomain,
     epoch, purpose, salt): FingerprintCommitment {
  assert purpose in ["local_only", "pairwise_verifier", "provider_ephemeral", "
    public_commitment"]
  assert salt.length == 32

  bucketed = bucketizeMetadata(metadata, context.bucket_profile)
  scopeKey = KDF(masterKey, "tsl-fp-v1" || context.id || verifierDomain || epoch || purpose)
  fingerprint = HMAC(scopeKey, "tsl.metadata.fp.v1" || canonicalize(bucketed))
  commitment = HASH("tsl.metadata.commit.v1" || fingerprint || salt)

  return {
    type: "tsl.metadata_fingerprint_commitment.v1",
    scope_class: purpose,
    scope_commitment: HASH(verifierDomain || epoch || purpose),
    bucket_profile: context.bucket_profile,
    fingerprint_commitment: commitment,
    salt_commitment: HASH(salt),
    created_at_bucket: bucketed.time_bucket,
    disclosure_policy: policyForPurpose(purpose)
  }
}
```

64.7.6 Delegated Agent Authorization v0

```
function verifyDelegatedAgentActionV0(action, delegationChain, resolver, atTime):
    AuthorizationResult {
  assert action.type == "tsl.agent_action.v2"
  assert verifySignature(action.agent, canonicalHashWithoutSignature(action), action.
    signature)

  effectiveScope = universalAllowScope()

  for policy in delegationChain ordered from principal to final delegate:
    if verifySignature(policy.principal, canonicalHashWithoutSignature(policy), policy.
    signature) == false:
      return fail("TSL_DELEGATION_SIGNATURE_INVALID")
    if atTime < policy.valid_from or atTime >= policy.valid_until:
```

```
      return fail("TSL_DELEGATION_EXPIRED")
    if resolver.isRevoked(policy.revocation_pointer, atTime):
      return fail("TSL_DELEGATION_REVOKED")
    if policy.delegate != nextActorInChain(policy, delegationChain, action):
      return fail("TSL_DELEGATION_CHAIN_BROKEN")
    effectiveScope = intersectScopes(effectiveScope, policy.scope)

  if scopeAllows(effectiveScope, action.action, action.resource, action.parameters, atTime)
    == false:
    return fail("TSL_DELEGATION_SCOPE_VIOLATION")

  if action.value_minor_units > effectiveScope.max_value_minor_units:
    return fail("TSL_DELEGATION_VALUE_LIMIT_EXCEEDED")

  if effectiveScope.requires_human_approval and action.human_approval_proof is null:
    return fail("TSL_HUMAN_APPROVAL_REQUIRED")

  return pass({effective_scope_commitment: canonicalHash(effectiveScope)})
}
```

64.8 Required Test Vectors

The following test vectors MUST be added before the v2 object set is marked release-candidate.

```
scoring_profile.v2/valid-transparent.json
  Assert: schema validates, canonical hash matches expected value,
  provider signature verifies.

scoring_profile.v2/invalid-float.json
  Assert: rejected because signed scoring profiles MUST NOT use floating-point values.

domain_policy.v1/agent-payments.json
  Assert: requires settlement, delegation check, and value-based step-up.

trust_assessment.v2/cryptographic-failure.json
  Assert: contains no ordinary numeric trust score and preserves failed gate result.

metadata_fingerprint_commitment.v1/pairwise.json
  Assert: same metadata under two verifier domains produces unlinkable commitments.

metadata_fingerprint_commitment.v1/rotation.json
  Assert: same metadata under different epochs produces different commitments
  when rotation is enabled.

graph_feature_vector.v1/small-graph.json
  Assert: degree, reciprocity, HHI, entropy, effective counterparty count,
  and seed escape match expected values.

sybil_assessment.v1/cluster-b2.json
  Assert: synthetic B2 cluster produces expected concentration, internal
  receipt ratio, attack cost, and risk label.

drift_report.v1/dormant-reactivation.json
  Assert: dormant identity with sudden high-risk activity produces step_up.
```

```
attestation.v2/appealed-negative.json
  Assert: appeal changes status without deleting the original signed negative claim.

model_card.v2/reference-scorer.json
  Assert: model card validates and links to evaluation report commitment.

evaluation_report.v1/promotion-pass.json
  Assert: promotion gates are computed deterministically from supplied metrics.

delegation_policy.v2/invoice-agent.json
  Assert: policy canonicalizes and signs deterministically.

agent_action.v2/inside-scope.json
  Assert: agent action verifies as inside delegated scope.

agent_action.v2/outside-scope.json
  Assert: action fails with TSL_DELEGATION_SCOPE_VIOLATION.
```

64.9 Deterministic Test Vector Format

Every test vector directory MUST contain a manifest with the following structure:

```
{
  "type": "tsl.test_vector_manifest.v1",
  "vector_id": "scoring_profile.v2/valid-transparent",
  "spec_version": "3.0.0",
  "object_type": "tsl.scoring_profile.v2",
  "canonicalization": "tsl-json-c14n-v1",
  "hash_suite": "sha256-domain-v1",
  "signature_suite": "ed25519-v1",
  "input_files": ["input.json"],
  "expected": {
    "schema_valid": true,
    "canonical_hash": "0x...",
    "signature_valid": true,
    "error_code": null
  }
}
```

64.10 Versioning, Compatibility, and Migration

64.10.1 Object Version Rules

1. Object versions are part of the signed `type` field.
2. A verifier MUST reject an object whose major version it does not understand unless an explicit forward-compatible extension rule applies.
3. A verifier MUST NOT reinterpret a v1 object as v2 or a v2 object as v1.
4. Minor schema extensions MUST use an `extensions` namespace and MUST NOT alter existing field meaning.

5. Major version changes MAY change required fields, semantics, or verification rules, but MUST use a new `type` discriminator.
6. A signed object MUST remain verifiable under the rules active for its declared object version and verification time policy.

64.10.2 v1 and v2 Coexistence

Case	Required Verifier Behavior
Known v1 object	Verify using v1 rules.
Known v2 object	Verify using v2 rules.
Unknown major version	Reject with `TSL_UNSUPPORTED_OBJECT_VERSION`.
Known object with unknown required field	Reject unless field is inside declared extension namespace.
v1 event with v2 assessment	Allowed if the assessment explicitly references the v1 evidence object hash.
v2 event with v1 verifier	v1 verifier MUST reject as unsupported, not partially verify.
Deprecated object version	Verify historical proofs, but SHOULD warn that new issuance is deprecated.
Revoked object version	Reject new issuance and warn on historical verification according to policy.

64.10.3 Concrete v1-to-v2 Migration Examples

Migration	Required Rule
`trust_assessment.v1` **to** `trust_assessment.v2`	Preserve the v1 assessment hash as `legacy_assessment_hash`; map `score` to `score_bps = score * 100`; map `label` to the domain-policy label; add confidence interval, evidence coverage, scoring profile ID, and expiration.
`attestation.v1` **to** `attestation.v2`	Preserve issuer, subject, claim commitment, visibility, timestamps, and signature evidence; add claim class severity, appeal pointer, evidence class, reversal status, and issuer-quality reference.
`agent_delegation.v1` **to** `delegation_policy.v2`	Convert a monolithic delegation into canonical policy fields: `policy_id`, principal, delegate, resources, actions, constraints, valid window, subdelegation policy, revocation pointer, nonce, and principal signature.
Agent action migration	A v2 `agent_action.v2` MUST reference the relevant delegation chain root and use `issued_at` plus `parameters_commitment`; verifiers MUST reject legacy `timestamp` or singular `parameter_commitment` in v2 actions.
Old proof bundles	Historical bundles remain valid if each embedded object verifies under its declared version. A new verifier MAY wrap old evidence in a `proof_bundle.v1` container without rewriting the signed legacy objects.

64.10.4 Deprecation Policy

State	Meaning
`active`	New issuance and verification are supported.
`deprecated`	Historical verification is supported; new issuance SHOULD warn.
`sunset`	New issuance MUST stop after a published date; historical verification continues.
`revoked`	New issuance is rejected; historical verification shows a security warning.
`research_only`	Object may appear in experiments but MUST NOT be required for product verification.

64.11 Product UX Requirements

Trust UX is part of the security model. The interface MUST not collapse cryptographic validity, trust score, and safety into one misleading green checkmark.

64.11.1 Required Verifier States

State	Required UX Meaning
`valid_proof`	Signature, key, inclusion, checkpoint, and revocation checks passed. This does not imply the actor is safe.
`invalid_signature`	The payload does not verify against the claimed key. Treat as invalid.
`revoked_key`	The signing key was revoked before the relevant event time or is currently revoked.
`unknown_identity`	The identity has insufficient continuity or cannot be resolved under current policy.
`insufficient_evidence`	The score provider abstained because evidence coverage was too low. This MUST NOT be rendered as malicious.
`high_risk`	Risk model or hard security checks indicate elevated risk. UI MUST show specific reasons.
`appealed_negative_claim`	A negative claim exists but has active appeal or reversal context.
`agent_inside_scope`	Agent action was authorized under current delegation policy.
`agent_outside_scope`	Agent action exceeded delegated permissions.

64.11.2 Required UX Flows

A production client SHOULD define user flows for:

1. first-time TrustID creation;
2. proof-link generation;
3. proof-link verification;
4. browser-extension warning display;
5. co-signed receipt request;

6. disclosure consent before revealing raw messages or counterparties;
7. scoring explanation display;
8. key revocation and recovery;
9. negative-attestation appeal;
10. agent delegation review;
11. agent action approval or rejection;
12. model/provider selection;
13. local-only mode and privacy export/delete controls.

UX Safety Rule

A low-evidence identity MUST be displayed as unknown or insufficient evidence, not as suspicious, unless there is affirmative adverse evidence. New users are not Sybils by default.

64.11.3 Proof-Link Verifier Screen Requirements

A proof-link verifier MUST show, at minimum:

1. proof status: valid, invalid, pending settlement, or unavailable;
2. signer TrustID and key status;
3. event class and disclosed fields;
4. whether raw content was opened or only commitment was checked;
5. revocation status at event time and current time;
6. checkpoint status and settlement backend;
7. optional trust assessment provider, model version, score label, confidence, expiration, and appeal link;
8. clear separation between protocol checks and risk assessment;
9. a shareable verification bundle export.

64.11.4 Agent Authorization Review Requirements

For agent actions, the UI MUST show:

- principal identity;
- delegate identity;
- action requested;
- resource affected;
- value or cost bound when applicable;
- policy validity period;
- whether subdelegation was used;
- whether human approval is required;
- exact failure code when authorization fails.

64.12 Operational Requirements

64.12.1 Service-Level Objectives

System Component	MVP SLO Target
Verifier library	Deterministic local verification; no network dependency for already-fetched proof bundles.
Hosted verifier API	99.5% monthly availability during beta.
Relay intake API	99.5% monthly availability during beta.
Proof retrieval API	p95 under 500 ms for settled proof lookup under nominal load.
Checkpoint submission	99% of epoch checkpoints submitted within two epoch durations.
Revocation propagation	Revocation visible to resolver within 60 seconds after settlement confirmation.
Auditor findings	Critical checkpoint conflict findings published within 15 minutes of detection.
Scoring provider	Assessment p95 latency under 750 ms for cached feature inputs; expiration enforced.
Appeal intake	High-impact negative-claim appeal intake acknowledged within one business day.

64.12.2 Provider Onboarding

A scoring provider MUST submit:

1. provider identity and signing key;
2. provider status request: sandbox, active, probation, or research-only;
3. supported domains;
4. model card;
5. scoring profile;
6. evaluation report;
7. privacy leakage report;
8. appeal policy;
9. security contact;
10. incident response contact;
11. signed changelog for each promoted release.

64.12.3 Auditor Onboarding

An auditor MUST publish:

1. auditor TrustID;
2. monitored relays and shards;
3. checkpoint verification policy;
4. log consistency verification method;

5. disclosure policy for findings;
6. signing key and rotation policy;
7. conflict escalation contact.

64.12.4 Required Runbooks

The production repository MUST include runbooks for:

1. relay outage;
2. log-node corruption;
3. checkpoint submission delay;
4. checkpoint conflict;
5. relay signing-key compromise;
6. provider signing-key compromise;
7. user key compromise;
8. malicious negative-attestation campaign;
9. scoring-provider bad release;
10. emergency model rollback;
11. schema migration failure;
12. settlement backend outage;
13. vulnerability disclosure and patch release;
14. data retention and deletion request;
15. privacy incident;
16. auditor false-positive finding;
17. high-volume abuse or spam attack.

64.12.5 Key Ceremonies

Production deployments SHOULD define ceremonies for:

- relay checkpoint signing-key creation;
- relay key rotation;
- provider signing-key creation;
- emergency revocation of provider keys;
- smart-contract admin or governance key rotation;
- auditor key registration;
- disaster recovery restore test.

Each ceremony SHOULD produce a signed ceremony record containing participants, date, key identifiers, action type, artifacts generated, and verification checks performed.

64.13 Open Reference Scoring Governance

The reference scorer SHOULD be open-source and reproducible. The protocol MUST allow alternative scoring providers, but compatibility requires all providers to emit signed assessments using the same

canonical assessment object family.

64.13.1 Reference Scorer License and Release Rules

The reference scorer SHOULD use a permissive open-source license for adoption, plus a clear patent and trademark policy if applicable. Each release MUST include:

1. semantic version;
2. signed source archive commitment;
3. signed model card;
4. feature registry;
5. deterministic test vectors;
6. benchmark results;
7. privacy leakage report;
8. changelog describing feature, weight, threshold, and calibration changes;
9. migration notes from prior scorer versions;
10. reproducible build instructions.

64.13.2 Community Review Process

A reference scorer release SHOULD move through these states:

State	Meaning
`draft`	Proposal open for comments; not used for production labels.
`shadow`	Runs on historical and live data without affecting user-facing labels.
`candidate`	Passed automated tests and awaits governance or maintainer approval.
`recommended`	Default reference scorer for supported domains.
`probation`	Known issue or pending review; labels may warn.
`rejected`	Failed promotion gates or security/privacy review.
`retired`	No longer recommended for new issuance; historical assessments remain verifiable.

64.13.3 Alternative Provider Compatibility

Alternative scoring providers MAY use different models, features, or weights, but MUST disclose:

1. provider identity;
2. model version;
3. supported domains;
4. input evidence classes;
5. output label semantics;
6. calibration method;
7. expiration policy;

8. appeal process for high-impact labels;
9. evaluation report commitment;
10. privacy report commitment.

64.14 Model Promotion Gates

A model MUST NOT become a default scorer unless it passes domain-specific promotion gates. The following are reference gates, not universal constants.

Domain	Gate Bias	Minimum Gate
Anti-phishing	Low false negatives	AUROC at least 9200 bps, AUPRC at least 8500 bps, ECE at most 400 bps, red-team pass.
Marketplace trust	Balanced	AUROC at least 8800 bps, sparse-identity false positive at most 200 bps, appeal reversal monitoring.
Agent payments	Conservative	Zero known delegation-scope bypasses, settlement and revocation tests pass, value-threshold step-up tested.
Open-source maintainer	Low false positives	Drift warnings use step-up by default; maintainer-change false-positive review required.
Professional identity	Privacy and defamation safety	No public high-risk label without adverse evidence and appeal path; leakage report required.

64.15 Implementation Alignment and Conformance Levels

The reference implementation MAY initially implement only a release-candidate subset of this specification. The v2 scoring, graph geometry, metadata fingerprint, Sybil, drift, governance, and agent-delegation systems SHOULD be tracked as separate conformance levels.

Conformance Level	Required Scope	Excluded or Optional Scope
`TSL-RC0`	Identity, event commitment, canonicalization, signatures, proof links, local verification	No graph scoring, no ZK, no Sybil assessment.
`TSL-RC1`	Receipts, Merkle log, inclusion proof, checkpoint registry, revocation registry	No public negative claims, no advanced scoring.
`TSL-RC2`	Reference feature extractor, `TrustAssessmentV2`, model card, evaluation report	Advanced private graph proofs optional.
`TSL-RC3`	Metadata fingerprint commitments, graph profile, Sybil assessment, drift report	ZK graph distance optional.
`TSL-RC4`	Delegation policy v2, agent action v2, inside-scope verifier	Multi-agent ZK proof optional.
`TSL-MAINNET`	Audits, runbooks, provider governance, appeal process, monitoring, production SLOs	Research-only circuits optional.

64.16 Product Requirements Document Annex

The first product milestone SHOULD be a focused proof-link and agent-verification product, not the entire graph/ZK/scoring ecosystem.

64.16.1 MVP User Stories

1. As a user, I can create a TrustID locally or through a smart account.
2. As a user, I can sign a message, claim, or action commitment.
3. As a user, I can generate a portable proof link.
4. As a recipient, I can open a proof link and verify signature, key state, revocation, inclusion, and checkpoint.
5. As a recipient, I can co-sign a receipt for an interaction.
6. As a user, I can revoke a compromised key.
7. As an AI-agent developer, I can create an agent identity and scoped delegation policy.
8. As a verifier, I can determine whether an agent action was inside delegated scope.
9. As an enterprise or marketplace, I can request an optional signed assessment without making scoring necessary for core verification.

64.16.2 MVP Non-Goals

- No global social graph browser.
- No universal human-worth score.
- No mandatory KYC.
- No public exposure of raw messages or exact counterparties by default.
- No ZK graph-distance proof as an MVP dependency.

- No token requirement for core verification.

64.17 Security and Compliance Package Requirements

Before production mainnet or enterprise launch, the project SHOULD maintain the following non-code artifacts:

```
/security/
  threat-model.md
  secure-sdlc.md
  dependency-policy.md
  cryptography-review.md
  key-management-policy.md
  incident-response.md
  vulnerability-disclosure-policy.md
  audit-plan.md
  abuse-response-policy.md
  privacy-threat-model.md
  negative-claims-risk-review.md
  agent-security-test-plan.md

/legal-compliance/
  privacy-policy-draft.md
  terms-of-service-draft.md
  data-processing-addendum-draft.md
  gdpr-ccpa-mapping.md
  automated-decisioning-review.md
  appeal-and-takedown-policy.md
  law-enforcement-request-policy.md
```

This specification is technical architecture, not legal advice. High-impact labels, negative attestations, automated risk assessments, and appeal processes require jurisdiction-specific legal review before production use.

64.18 Next Required Document

The next implementation document SHOULD NOT be another broad architecture paper. The next document SHOULD be:

> **TSL Open Reference Scoring Algorithm: v0 Deterministic Implementation + Test Vectors**

That document MUST define:

1. exact v0 input object set;
2. feature extraction algorithms;
3. normalization formulas;
4. graph construction rules;
5. scoring weights;
6. calibration method;

7. confidence interval method;
8. abstention rules;
9. label threshold rules;
10. deterministic test vectors;
11. benchmark promotion gates;
12. privacy leakage scoring;
13. known limitations.

Final Implementation Direction

The strongest product path is staged: ship proof links, verification, receipts, revocation, logs, checkpoints, and agent delegation first; then add transparent reference scoring; then add graph/Sybil/drift models; then add advanced privacy proofs. Do not make graph scoring, ZK, or global provider governance blocking requirements for the first useful product.

65 Final Summary

TSL is not another messaging app. It is not a KYC company. It is not a single reputation score. It is not "messages on blockchain."

TSL is a trust settlement protocol for the AI internet.

Its key ideas are:

- applications are transports, not trust authorities,
- identities own portable cryptographic continuity,
- events are signed but content remains private by default,
- receipts make interactions mutually witnessed,
- append-only logs make history auditable,
- blockchain checkpoints make state durable and portable,
- graph intelligence makes adversarial behavior legible,
- scoring is plural, signed, explainable, and contestable.

The internet's old trust signals are being commoditized by AI. The new scarce signal is durable, organic, cryptographic continuity.

65.1 Release-Conformance Checklist

A team should not treat this document as one undifferentiated implementation target. The robust path is staged conformance:

Level	Product Meaning
`TSL-RC0`	Proof/signature baseline: canonicalization, identity, event commitments, proof links, and local verification.
`TSL-RC1`	Logs, settlement, and revocation: Merkle inclusion proofs, checkpoint registry, resolver, and revocation checks.
`TSL-RC2`	Scoring profile and assessment v2: machine-readable scoring profiles, evidence coverage, model cards, evaluation reports, and signed assessments.
`TSL-RC3`	Metadata, graph, Sybil, and drift: fingerprint commitments, graph feature vectors, Sybil assessments, and drift reports.
`TSL-RC4`	Agent delegation/action: delegation policies, scoped agent actions, chain-of-authority verification, and agent UX.
`TSL-MAINNET`	Audits, governance, operations, incident response, provider onboarding, appeals, monitoring, and production SLOs.

Guiding Principle

Trust the trajectory, not the profile.

66 Appendix D: Release Candidate Evidence

This appendix summarizes evidence generated by the repository. Long hashes are abbreviated for page layout. Full machine-readable artifacts remain in `deployments/`, `reports/`, and `docs/`.

66.1 Base Sepolia Deployment Evidence

Field	Value
Network	`base-sepolia`
Chain ID	`84532`
Deployer	`0x38953ec02af3Ee73...c3af485D235E`
Checkpoint registry	`0x6d2B5D3FD59AaB9d...262f31D0ec94`
TrustID registry	`0xEB977ec197aC5546...C64FBC2B5083`
Revocation registry	`0x9Cc3e4a9b10ED095...208AaDfeABf5`
Provider registry	`0xDa7703c4084916D9...14ef6cD456FE`
Governance registry	`0x4b6eC8c7a457F4D2...d80FDF8DF343`
Authorized relay hash	`0xd7ed37e2bf220b4a...179f3ef1b20a`
Settlement transaction	`0x839a91ed24b8b1e9...65104f2322b7`
Checkpoint hash	`0x8b2bc8aeca1ca617...86c7bc7eac4c`
Verifier result	`true`
Checkpoint settled	`true`

66.2 ZK Artifact Evidence

Claim	Circuit Hash	Zkey Hash	Verification Key Hash
identity_age_days	0x955c0ffc7cc79a77...76e640e38d21	0x1735467f591ca7af...1db0bd71b0c9	0x992990a8cb0946a2...720afb04c436
reciprocal_receipt_count	0x90d07dfdbed820d3...3b2586f2c562	0xfdc21029bf82a79...eb2b03608169	0x2c6796736cabded...b9c223b6cbbb

ZK Ceremony Warning

Development-only Groth16 setup generated by local snarkjs powers-of-tau contribution. Do not treat these zkeys as production trusted setup artifacts.

66.3 Release Gate Evidence

Gate	Command
Build	'npm run build'
Unit tests	'npm test'
Contracts	'npm run contracts:compile' and 'npm run contracts:test'
ZK	'npm run zk:test' and 'npm run zk:manifest'
Parity	'npm run parity:python' and 'npm run parity:rust'
Browser verifier	'npm run build:web-verifier'
Deployment checks	'npm run deploy:validate' and 'npm run deploy:base:dry-run'

66.4 Evidence File Hashes

File	SHA-256
deployments/base-sepolia.json	0xaa5ca2c0a431ba60...1c86cb27fdd7
reports/base-sepolia-e2e-report.json	0xe009f8b85ae03c68...344b9e1656c5
docs/zk-artifact-manifest.json	0x379f61786c3fb385...0c1f442697f1
docs/release-go-no-go.md	0x438c1925f7008852...68da35fc96f8
docs/npm-audit-risk.md	0x64bcb15385fb14ba...f367aac3569f

66.5 Audit and Custody Notes

The release checklist, audit-prep package, key-custody notes, and npm audit risk record are summarized below:

- Release gates are documented in `docs/release-go-no-go.md`.
- Audit inputs and known limitations are documented in `docs/audit-prep.md`.
- Development signing adapters and fail-closed `kms:/hsm:` boundaries are documented in `docs/key-custody.md`.

- Runtime and development dependency advisories are documented in `docs/npm-audit-risk.md`.

References

[1] World Wide Web Consortium, *Decentralized Identifiers (DIDs) v1.0*, W3C Recommendation. https://www.w3.org/TR/did-core/

[2] World Wide Web Consortium, *Verifiable Credentials Data Model v2.0*, W3C Recommendation, 2025. https://www.w3.org/TR/vc-data-model-2.0/

[3] Ethereum Improvement Proposals, *ERC-4337: Account Abstraction Using Alt Mempool*. https://eips.ethereum.org/EIPS/eip-4337

[4] IETF, *RFC 9162: Certificate Transparency Version 2.0*. https://www.rfc-editor.org/rfc/rfc9162.html

[5] Ethereum.org, *Scaling Ethereum and Layer 2 Networks*. https://ethereum.org/developers/docs/scaling/